OPERATING SYSTEMS
UE22CS242B

Introduction, Computer System Organization

Suresh Jamadagni
Department of Computer Science

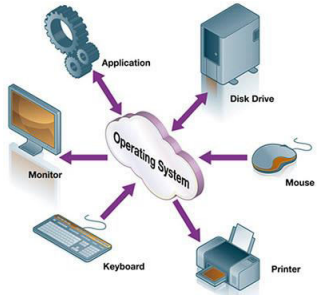
- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Introduction, Computer System Organization

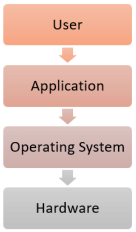
Suresh Jamadagni
Department of Computer Science



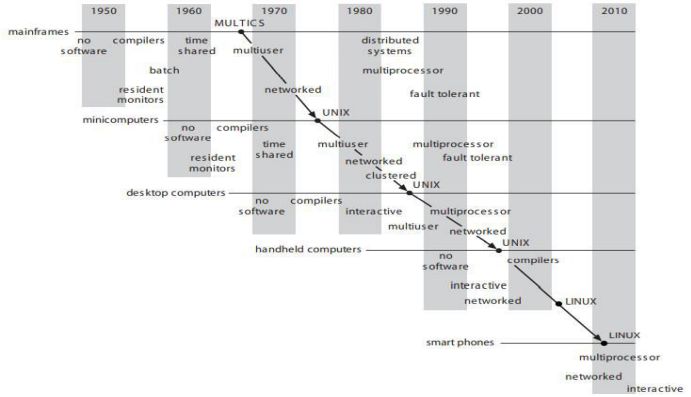
If OS is not developed then how will the application developers access the hardware?

OPERATING SYSTEMS
General Definition

- An Operating System is a program that acts as an intermediary between a user of a computer and the computer hardware
- It provides a user-friendly environment in which a user may easily develop and execute programs. Otherwise, hardware knowledge would be mandatory for computer programming.



OPERATING SYSTEMS
Genesis



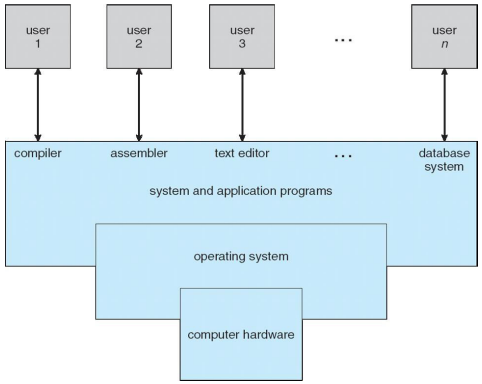
OPERATING SYSTEMS
Operating System Goals

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner
- Manage resources such as
  - Memory
  - Processor(s)
  - I/O Devices

OPERATING SYSTEMS
Why Study Operating Systems?

- Only a small percentage of computer practitioners will be involved in the creation or modification of an operating system.
- However almost all code runs on top of an operating system, and thus knowledge of how operating systems work is crucial to proper, efficient, effective, and secure programming.
- Understanding the fundamentals of operating systems

OPERATING SYSTEMS
Four Components of a Computer System (Abstract view)



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Computer System Structure

- Hardware – provides basic computing resources
  - CPU, memory, I/O devices
- Operating system
  - Controls and coordinates use of hardware among various applications and users
- Application programs – define the ways in which the system resources are used to solve the computing problems of the users
  - Word processors, compilers, web browsers, database systems, video games
- Users
  - People, machines, other computers

OPERATING SYSTEMS
What Operating Systems Do

- Depends on the point of view user and system
- Users want convenience, **ease of use** and **good performance**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy.
  - Maximize resource utilization.
  - Available CPU time, memory, and I/O are used efficiently and that no individual user takes more than her fair share

OPERATING SYSTEMS
What Operating Systems Do

- Users of dedicated systems such as workstations have dedicated resources but frequently use shared resources from servers.
  - resources like file, compute, and print servers are shared.
  - operating system is designed to compromise between individual usability and resource utilization
- Handheld computers are resource poor, optimized for usability and battery life.
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

OPERATING SYSTEMS
System View

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

OPERATING SYSTEMS
Defining Operating System

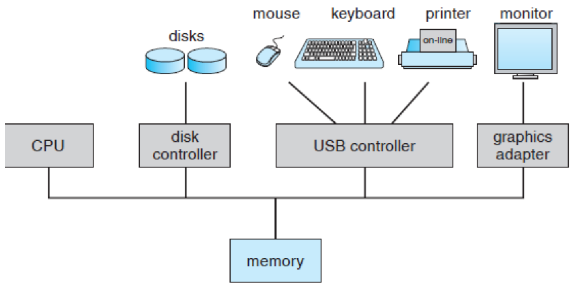
- The OS has many roles and functions
- The fundamental goal of computer systems is to execute user programs and to make solving user problems easier.
- The common functions of controlling and allocating resources are then brought together into one piece of software: the **operating system**
- "The one program running at all times on the computer" is the **kernel**.
- Everything else is either
  - a system program (ships with the operating system) , or
  - an application program.

OPERATING SYSTEMS
Computer System Organization

- Computer-system consists of,
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - The CPU and the device controllers can execute concurrently, competing for memory cycles.
    - memory controller is provided to synchronize access to the memory.

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Computer System Organization



A modern computer system

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Computer System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- Each device controller has registers for action (like “read character from keyboard”) to take
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

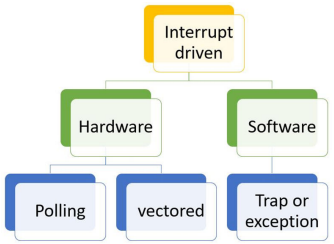
OPERATING SYSTEMS
Computer System Organization

- When the system is booted, the first program that starts running is a Boostrap.
- It is stored in read-only memory (ROM) or electrically erasable programmable read-only memory (EEPROM).
- Bootstrap is known by the general term firmware, within the computer hardware.
- It initializes all aspects of the system, from CPU registers to device controllers to memory contents.
- The bootstrap program must know how to load the operating system and how to start executing that system.
- The bootstrap program must locate and load into memory the operating system kernel.
- The first program that is created is **init**, after the OS is booted. It waits for the occurrence of event.

© copyright Silberschatz, Galvin and Gagne, 2013

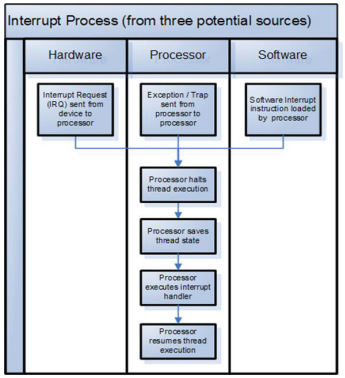
OPERATING SYSTEMS
Common Functions of Interrupts

- An operating system is **interrupt driven**
- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request

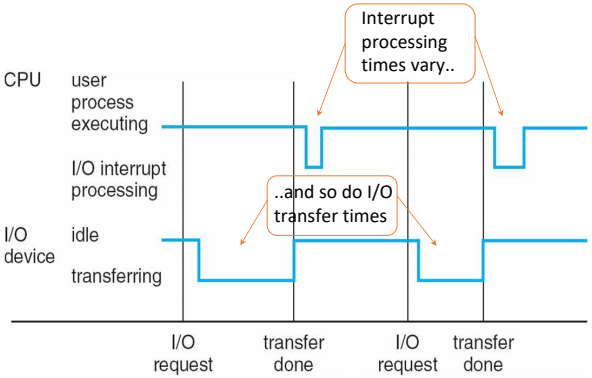


OPERATING SYSTEMS
Interrupt Handling

- The operating system saves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
  - polling
  - vectored interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt



OPERATING SYSTEMS
Interrupt Timeline for a single process doing output



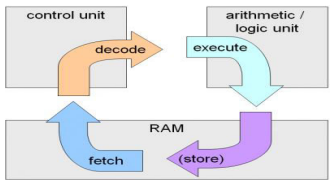
OPERATING SYSTEMS
Storage Structure

- Main memory – only large storage media that the CPU can access directly (**Random access memory** and typically **volatile)**
  - Implemented with semiconductor technology called DRAM
- Computers use other forms of memory like ROM, EEPROM
- Smart phones have EEPROM to store factory installed programs.

OPERATING SYSTEMS
Program Execution Model

Image source :https://computersciencewiki.org/index.php/The_machine_instruction_cycle

- Typical instruction execution



- The processor fetches instructions from memory, decodes and executes them.
- The Fetch, Decode and Execute cycles are repeated until the program terminates.
- This is called the Von Neumann model of computing.

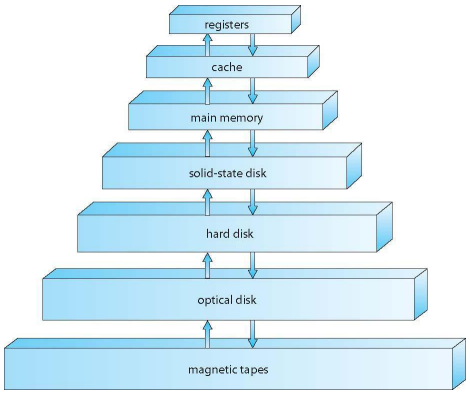
OPERATING SYSTEMS
Storage Structure

- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
  - Various technologies and becoming more popular
  - Flash memory used in camera's PDA's

OPERATING SYSTEMS
Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage

OPERATING SYSTEMS
Storage-Device Hierarchy



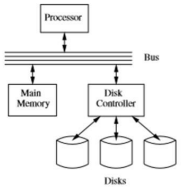
© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem

OPERATING SYSTEMS
I/O Structure

- Storage is a type of I/O device
- A large portion of operating-system code is dedicated to managing I/O
  - As reliability and performance of a system is the main concern.
- General-purpose computer system consists of CPUs and multiple device controllers that are connected through a common bus.
- Each device controller is in charge of a specific type of device.
- **Device Driver** for each device controller to manage I/O
  - Provides uniform interface between controller and kernel
- **Small Computer-Systems Interface (SCSI)** controller enables to connect more devices.



OPERATING SYSTEMS
I/O Structure

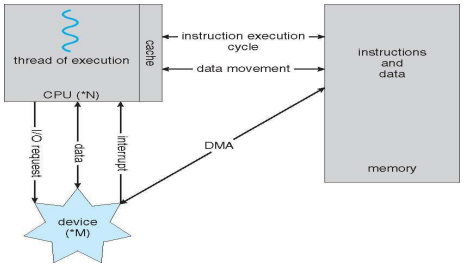
- A device controller maintains some local buffer storage and a set of special-purpose registers.
- The device controller is responsible for moving the data between the peripheral devices.

OPERATING SYSTEMS
I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
  - Wait instruction idles the CPU until the next interrupt
  - Wait loop (contention for memory access)
  - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
  - **System call** – request to the OS to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state
  - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt.

OPERATING SYSTEMS
Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte



THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

Computer-System Architecture, OS Structure and Operations

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Computer-System Architecture, OS Structure & Operations

Suresh Jamadagni
Department of Computer Science

- Most systems use a single general-purpose processor
  - Most systems have other special-purpose processors as well.
  - Device specific processors like disk, keyboard, graphic controller
  - Special-purpose processors run a limited number of instructions
  - Special-purpose processors are low-level components built into the hardware
  - Managed by OS.
  - OS monitors the status.
- Example Disk controller microprocessor
  - Receives sequence of requests from CPU.
  - Implements its own disk queue and scheduling algorithm
  - Relieves the main CPU of the overhead of disk scheduling.

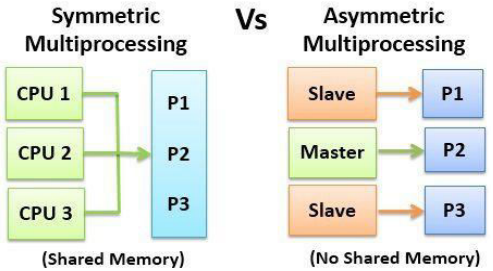
OPERATING SYSTEMS
Computer-System Architecture

- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    - **Increased throughput**
    - **Economy of scale**
    - **Increased reliability** – graceful degradation or fault tolerance

OPERATING SYSTEMS
Multiprocessors

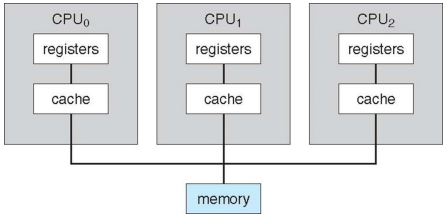
Two types of Multiprocessor Systems

1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
2. **Symmetric Multiprocessing** – each processor performs all tasks



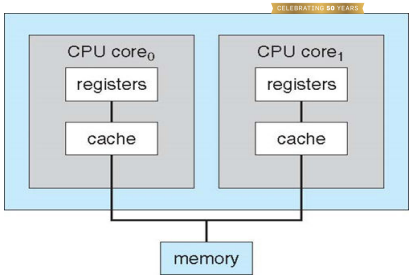
OPERATING SYSTEMS
Symmetric Multiprocessing Architecture

- In SMP all processors are peers; no boss–worker relationship exists between processors.
- Each processor has its own set of registers, as well as a private or local cache.
- All processors share physical memory.



OPERATING SYSTEMS
Multi-Core Design

- A recent trend in CPU design is to include multiple computing cores on a single chip. Such multiprocessor systems are termed **multicore**.
- More efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication.
- One chip with multiple cores uses significantly less power than multiple single-core chips.



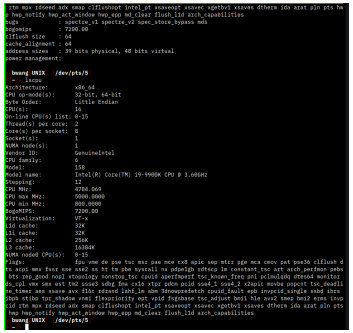
A dual-core design with two cores placed on the same chip

Command to know the number of cores, cache details
$cat /proc/cpuinfo|more

OPERATING SYSTEMS
Multi-Core Design

Command to know the number of cores, cache details

$more /proc/cpuinfo



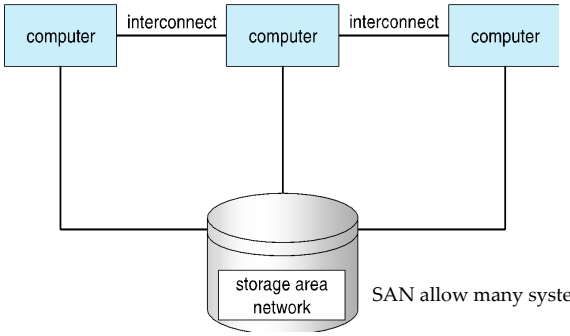
OPERATING SYSTEMS
Blade servers

- **Blade servers** are a recent development in which multiple processor boards, I/Oboards, and networking boards are placed in the same chassis.
  - blade-processor board boots independently and runs its own operating system.
  - Some blade-server boards are multiprocessor as well, which blurs the lines between types of computers.
  - In essence, these servers consist of multiple independent multiprocessor systems.

OPERATING SYSTEMS
Clustered Systems

- Like multiprocessor systems, but multiple systems working together
  - Usually sharing storage via a **storage-area network (SAN)**
  - Provides a **high-availability** service which survives failures
    - **Asymmetric clustering** has one machine in hot-standby mode
    - **Symmetric clustering** has multiple nodes running applications, monitoring each other
  - Some clusters are for **high-performance computing (HPC)**
    - Applications must be written to use **parallelization**
  - Some have **distributed lock manager** (**DLM**) to avoid conflicting operations (Ex: when multiple hosts access the same data on shared storage)

OPERATING SYSTEMS
Clustered Systems

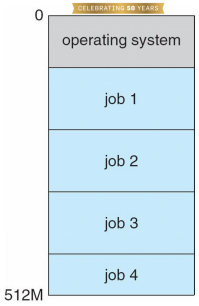


SAN allow many systems to attach to a pool of storage

General structure of a clustered system.

OPERATING SYSTEMS
Operating-System Structure - Multiprogramming

- **Multiprogramming** (**Batch system**) needed for efficiency
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
  - When it has to wait (for I/O for example), OS switches to another job



OPERATING SYSTEMS
Operating-System Structure - Multitasking

- **Timesharing** (**multitasking**) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory ⍰ **process**
  - If several jobs ready to run at the same time ⍰ **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run
  - **Virtual memory** allows execution of processes not completely in memory

OPERATING SYSTEMS
Operating-System Operations

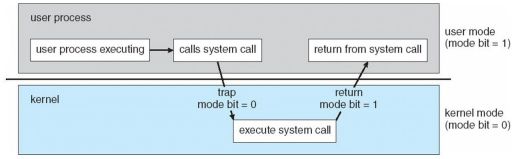
- **Interrupt driven** (hardware and software)
  - Hardware interrupt by one of the devices
  - Software interrupt (**exception** or **trap**):
    - Software error (e.g., division by zero)
    - Request for operating system service
    - Other process problems include infinite loop, processes modifying each other or the operating system

OPERATING SYSTEMS
Dual-Mode and Multimode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
  - i.e. **virtual machine manager** (**VMM**) mode for guest **VMs**

Transition from user to kernel mode

- When a trap or interrupt occurs, hardware switches from user mode to kernel mode (changes the state of the mode bit to 0).
- When the request is fulfilled, the system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.



Timer

- Timer to prevent infinite loop / process hogging resources
  - Timer is set to interrupt the computer after a specified period (fixed 1/60 sec or variable 1 msec to 1 sec)
  - A **variable timer** is generally implemented by a fixed-rate clock and a counter.
  - Operating system sets the counter (privileged instruction)
  - Every time the clock ticks, the counter is decremented.
  - When counter reaches zero, an interrupt occurs
  - Timer can be used to prevent a user program from running too long (terminate the program)

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

Kernel Data Structures and Computing Environments

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Kernel Data Structures

Suresh Jamadagni
Department of Computer Science

Array:

- An array is a simple data structure in which each element can be accessed directly.
- Main Memory constructed with array.
- How the data is accessed?
- Items with multiple bytes are accessed as item number × item size
- But what about storing an item whose size may vary?
- what about removing an item if the relative positions of the remaining items must be preserved?

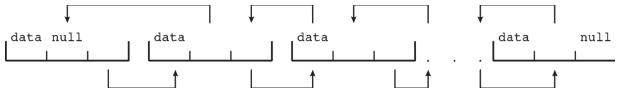
- Standard programming data structures are used extensively in OS

Singly linked list

- The items in a list must be accessed in a particular order.
- common method for implementing this structure is a linked list

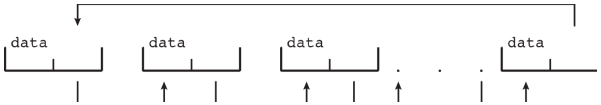
- In a **singly linked list,** each item points to its successor.

Doubly linked list



In a **doubly linked list**, a given item can refer either to its predecessor or to its successor.

Circular linked list



In a **circularly linked list**, the last element in the list refers to the first element, rather than to null.

Advantages:

- Linked lists accommodate items of varying sizes.
- Allow easy insertion and deletion of items

Disadvantages:

- Performance for retrieving a specified item in a list of size n is linear — O(n) , as it requires potentially traversing all n elements in the worst case.

Usage:

- Lists are used by the some of the kernel algorithms
- Constructing more powerful data structures such as stacks and queues

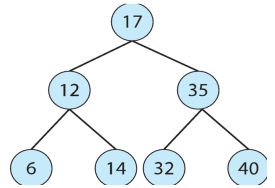
Stack - a sequentially ordered data structure that uses **LIFO** principle for adding and removing items

- OS often uses a stack when involving function calls.
- Parameters, local variables and the return address are **push**ed onto the stack when a function is called
- Return from the function call **pop**s those items off the stack

Queue - a sequentially ordered data structure that uses **FIFO** principle for adding and removing items

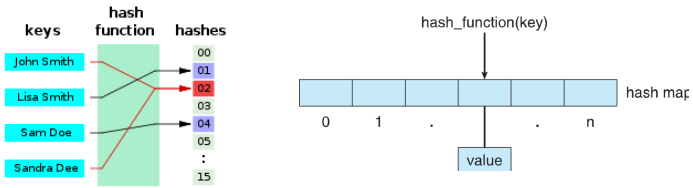
- Tasks waiting to be run on an available CPU are organized in queues
- Print jobs sent to a printer are printed in the order of submission

- Data structure used to represent data hierarchically.
- Data values in a tree structure are linked through parent–child relationships
- **Binary search tree**
  - ordering between 2 children: left <= right
  - Search performance is *O(n)*
- **Balanced binary search tree –** (a tree containing n items has at most lg n levels)
  - Search performance is *O(lg n)*
  - Used by Linux for selecting which task to run next (CPU-Scheduling algorithm)



OPERATING SYSTEMS
Hash Functions and Maps

- Hash functions can result in the same output value for 2 inputs
- **Hash function** can be used to implement a **hash map**
  - Maps or associates key:value pairs using a hash function
  - Search performance is *O(1)*



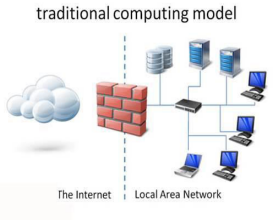
OPERATING SYSTEMS
Bitmaps

Bitmap - string of *n* binary digits representing the status of *n* items

- Availability of each resource is indicated by the value of a binary digit
  - 0 – resource is available
  - 1 – resource is unavailable
- Value of the **i**th position in the bitmap is associated with the **i**th resource
  - Example: bitmap 001011101 shows resources 2, 4, 5, 6, and 8 are unavailable; resources 0, 1, 3, and 7 are available
- Commonly used to represent the availability of a large number of resources (disk blocks)

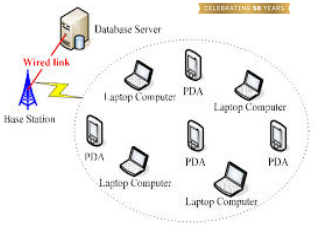
OPERATING SYSTEMS
Computing Environments – Traditional

- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers** (**thin clients**) are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks



OPERATING SYSTEMS
Computing Environments – Mobile

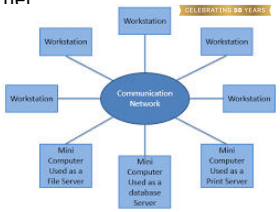
- Handheld smartphones, tablets, etc
- What is the functional difference between them and a "traditional" laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like ***augmented reality***
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**



OPERATING SYSTEMS
Computing Environments – Distributed

Distributed computing

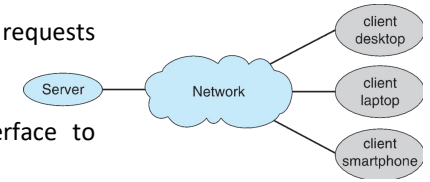
- Collection of separate, possibly heterogeneous systems networked together
  - **Network** is a communications path, **TCP/IP** most common
    - **Local Area Network** (**LAN**)
    - **Wide Area Network** (**WAN**)
    - **Metropolitan Area Network** (**MAN**)
    - **Personal Area Network** (**PAN**)
- **Network Operating System** provides features between systems across network
  - Communication scheme allows systems to exchange messages
  - Illusion of a single system



OPERATING SYSTEMS
Computing Environments – Client-Server

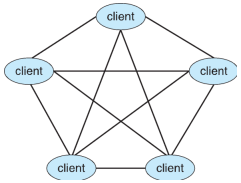
Client-Server Computing

- Dumb terminals replaced by smart PCs
- Many systems now **servers**, responding to requests generated by **clients**
  - **Compute-server system** provides an interface to client to request services (i.e., database)
  - **File-server system** provides interface for clients to store and retrieve files



OPERATING SYSTEMS
Computing Environments – Peer-to-Peer

- Another model of distributed system
  - P2P does not distinguish clients and servers
  - Instead all nodes are considered peers
  - May each act as client, server or both
  - Node must join P2P network
    - Registers its service with central lookup service on network, or
    - Broadcast request for service and respond to requests for service via ***discovery protocol***
  - Examples include Napster and Gnutella, **Voice over IP** (**VoIP**) such as Skype



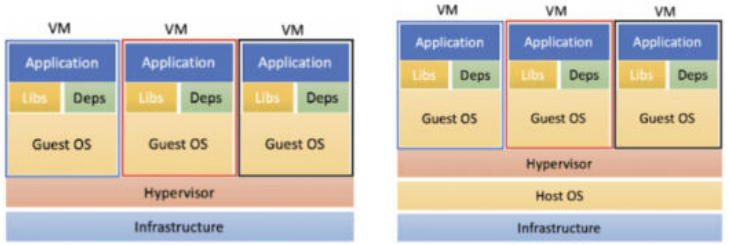
OPERATING SYSTEMS
Computing Environments – Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services

OPERATING SYSTEMS
Computing Environments – Virtualization

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - QA testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host

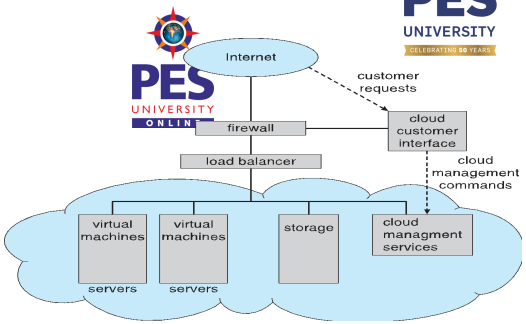
OPERATING SYSTEMS
Computing Environments – Virtualization



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Computing Environments – Cloud Computing

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
  - Internet connectivity requires security like firewalls
  - Load balancers spread traffic across multiple applications



OPERATING SYSTEMS
Computing Environments – Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for it functionality.
  - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
  - **Public cloud** – available via Internet to anyone willing to pay
  - **Private cloud** – run by a company for the company's own use
  - **Hybrid cloud** – includes both public and private cloud components

- Software as a Service (SaaS) – one or more applications available via the Internet (i.e., word processor)
- Platform as a Service (PaaS) – software stack ready for application use via the Internet (i.e., a database server)
- Infrastructure as a Service (IaaS) – servers or storage available over Internet (i.e., storage available for backup use)

- Real-time embedded systems most prevalent form of computers
  - Vary considerable, special purpose, limited purpose OS, real-time OS
  - Use expanding
- Many other special computing environments as well
  - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
  - Processing ***must*** be done within constraint
  - Correct operation only if constraints met

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

Operating-System Services, Design and Implementation

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

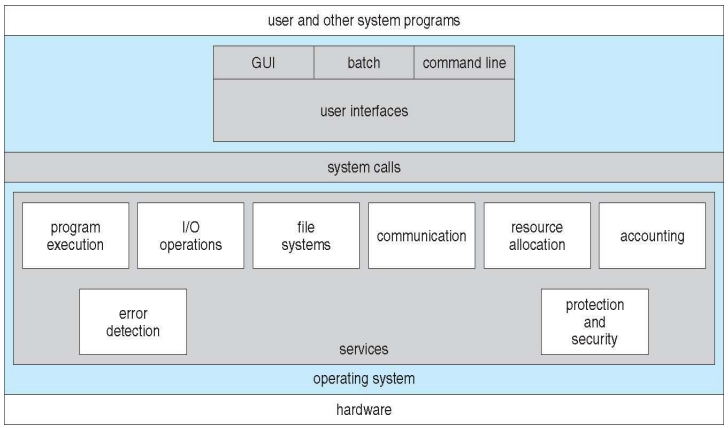
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Operating-System Services

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
A View of Operating System Services



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Services

- Operating systems provide an environment for execution of programs and services to programs and users
- Set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - **Command-Line (CLI) -** Command interpreters (shells)
    - **Graphics User Interface (GUI)**
    - **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device

- **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)

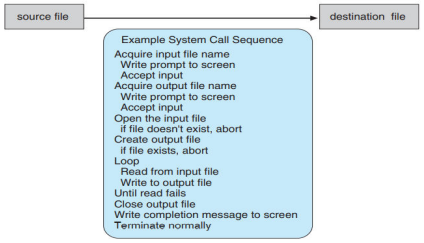
- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources

- **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all access to system resources is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

OPERATING SYSTEMS
System calls

- **System calls** provide an interface to the services made available by an operating system.
- These calls are generally available as routines writt in a higher level programming language or assembl language
- Example: Sequence of system calls used for copying contents from one file to another



OPERATING SYSTEMS
Types of System calls

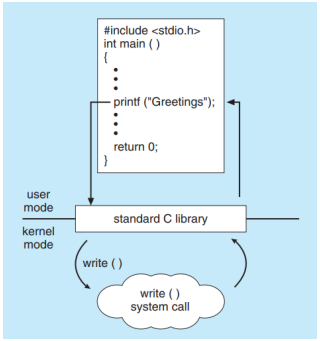
System calls can be grouped roughly into six major categories:

1. Process control
2. File manipulation
3. Device manipulation
4. Information maintenance
5. Communications
6. Protection

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

OPERATING SYSTEMS
Example of a system call

- A C program invokes the printf() statement
- The C library intercepts this call and invokes the necessary system call **write()** in the operating system
- The C library takes the value returned by write() and passes it back to the user program



OPERATING SYSTEMS
Operating-System Design and Implementation

Design goals

- Start the design by defining goals and specifications
- Affected by choice of hardware and the type of system
- Requirements can be categorized as User goals and System goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

OPERATING SYSTEMS
Operating-System Design and Implementation

- Important principle to separate
  Policy: ***What*** will be done?
  Mechanism: ***How*** to do it?
- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer to prevent a user program from running too long)
- Specifying and designing an OS is highly creative task of **software engineering**

OPERATING SYSTEMS
Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
chandravvahebbi@pes.edu

OPERATING SYSTEMS

Process Management

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

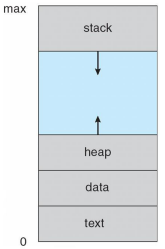
Process Concept

Suresh Jamadagni
Department of Computer Science

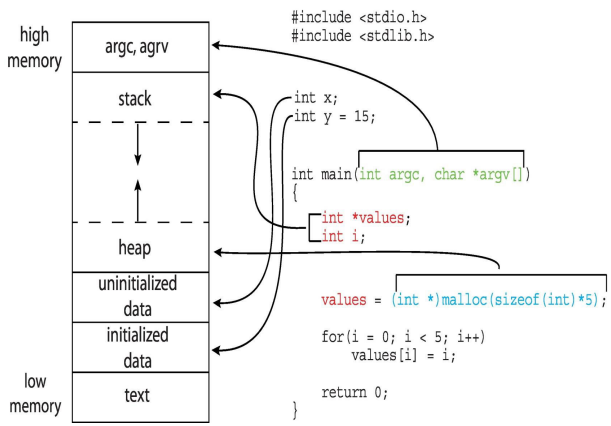
- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program

Structure of a process in memory

- The program code, also called **text section.**
  - Includes current activity including **program counter**, processor registers
- **Stack** containing temporary data
  - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time



OPERATING SYSTEMS
Memory Layout of a C Program



OPERATING SYSTEMS
Memory Layout of a process

```
suresh@PES1UG19CS553:~/OS$ gcc -o q1b.exe q1b.c
suresh@PES1UG19CS553:~/OS$ ./q1b.exe 5
Process id: 23059
```

```
suresh@PES1UG19CS553:~$ pmap 23059
23059:   ./q1b.exe 5
000055cefc914000      4K r-x-- q1b.exe
000055cefcb14000      4K r---- q1b.exe
000055cefcb15000      4K rw--- q1b.exe
000055cefe6e7000    132K rw---   [ anon ]
00007f26ef253000   1948K r-x-- libc-2.27.so
00007f26ef43a000   2048K ----- libc-2.27.so
00007f26ef63a000     16K r---- libc-2.27.so
00007f26ef63e000      8K rw--- libc-2.27.so
00007f26ef640000     16K rw---   [ anon ]
00007f26ef644000    164K r-x-- ld-2.27.so
00007f26ef857000      8K rw---   [ anon ]
00007f26ef86d000      4K r---- ld-2.27.so
00007f26ef86e000      4K rw--- ld-2.27.so
00007f26ef86f000      4K rw---   [ anon ]
00007ffda2b15000    136K rw---   [ stack ]
00007ffda2bae000     12K r----   [ anon ]
00007ffda2bb1000      8K r-x--   [ anon ]
ffffffffff600000      4K --x--   [ anon ]
 total             4524K
```

OPERATING SYSTEMS
Memory Layout of a process

```
mnskrishna@LAPTOP-OEJNE4JS ~/OS
$ ps
    PID    PPID    PGID     WINPID   TTY          UID    STIME COMMAND
   1676       1    1676      11056  ?         197609 12:36:00 /usr/bin/mintty
   1677    1676    1677      24928  pty0      197609 12:36:01 /usr/bin/bash
   1722    1677    1722      20492  pty0      197609 13:12:21 /usr/bin/ps
```

```
mnskrishna@LAPTOP-OEJNE4JS /
$ cd /proc/1676

mnskrishna@LAPTOP-OEJNE4JS /proc/1676
$ ls -l
total 0
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 cmdline
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 ctty
lrwxrwxrwx 1 mnskrishna None 0 Jan 20 13:16 cwd -> /home/mnskrishna
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 environ
lrwxrwxrwx 1 mnskrishna None 0 Jan 20 13:16 exe -> /usr/bin/mintty
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 exename
dr-xr-xr-x 2 mnskrishna None 0 Jan 20 12:36 fd
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 gid
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 maps
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 mountinfo
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 mounts
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 pgid
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 ppid
lrwxrwxrwx 1 mnskrishna None 0 Jan 20 13:16 root -> /
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 sid
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 stat
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 statm
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 status
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 uid
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 winexename
-r--r--r-- 1 mnskrishna None 0 Jan 20 13:16 winpid
```

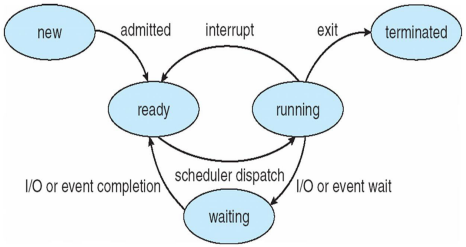
OPERATING SYSTEMS
Memory Layout of a process

```
mnskrishna@LAPTOP-OEJNE4JS /proc/1676
$ cat maps
00010000-00020000 rw-s 00000000 0000:0000 0                    [win heap 1 default shared]
00020000-00022000 r--s 00000000 0000:0000 0
00030000-0004D000 r--s 00000000 0000:0000 0
00050000-00054000 r--s 00000000 0000:0000 0
00060000-00063000 r--s 00000000 0000:0000 0
00070000-00072000 rw-p 00000000 0000:0000 0
00080000-00149000 r--s 00000000 FE1D:7B2C 3096224745518844     /cygdrive/c/Windows/System32/locale.nls
00150000-00152000 r--s 00000000 0000:0000 0
00160000-00161000 r--s 00000000 0000:0000 0
00170000-00173000 rw-p 00000000 0000:0000 0                    [win heap 0 default grow]
00173000-001A2000 ===p 00003000 0000:0000 0                    [win heap 0 default grow]
001B0000-001B4000 r--s 00000000 0000:0000 0
007B0000-009A9000 ===p 00000000 0000:0000 0                    [stack (tid 4688)]
009A9000-009AC000 rw-g 001F9000 0000:0000 0                    [stack (tid 4688)]
009AC000-009B0000 rw-p 001FC000 0000:0000 0                    [stack (tid 4688)]
00DA0000-00DC0000 r--s 00000000 0000:0000 0
00DC0000-00FA0000 ===s 00020000 0000:0000 0
00FA0000-01121000 r--s 00000000 0000:0000 0
01130000-0124B000 r--s 00000000 0000:0000 0
0124B000-02531000 ===s 0011B000 0000:0000 0
02540000-02739000 ===p 00000000 0000:0000 0                    [stack (tid 11584)]
02739000-0273C000 rw-g 001F9000 0000:0000 0                    [stack (tid 11584)]
0273C000-02740000 rw-p 001FC000 0000:0000 0                    [stack (tid 11584)]
02740000-02768000 r--s 00000000 FE1D:7B2C 281474977443112      /cygdrive/c/Windows/System32/C_932.NLS
02770000-0279D000 r--s 00000000 FE1D:7B2C 281474977443073      /cygdrive/c/Windows/System32/C_20932.NLS
027A0000-027D1000 r--s 00000000 FE1D:7B2C 281474977443114      /cygdrive/c/Windows/System32/C_949.NLS
```

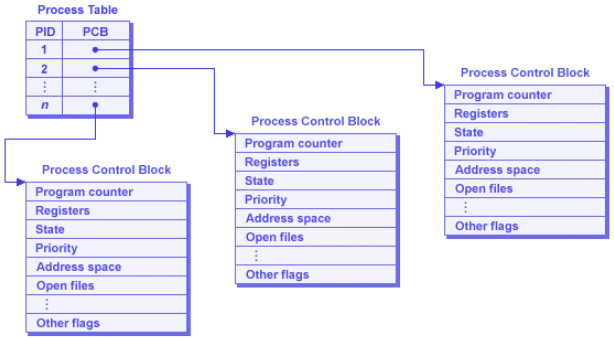
OPERATING SYSTEMS
Process State

As a process executes, it changes **state**

- **New**: The process is being created
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Ready**: The process is waiting to be assigned to a processor
- **Terminated**: The process has finished execution



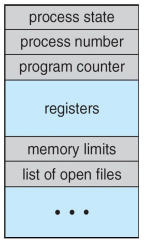
OPERATING SYSTEMS
Process Control Block (PCB)



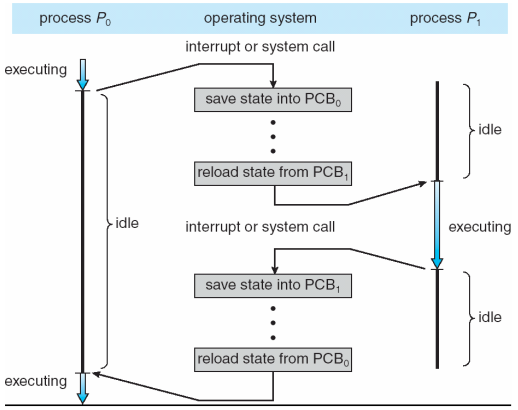
OPERATING SYSTEMS
Process Control Block (PCB)

Each process is represented in the operating system by a Process Control Block (also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



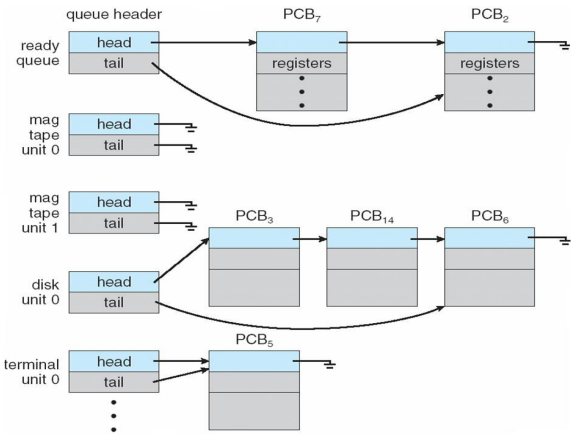
OPERATING SYSTEMS
CPU switch from process to process



OPERATING SYSTEMS
Process Scheduling

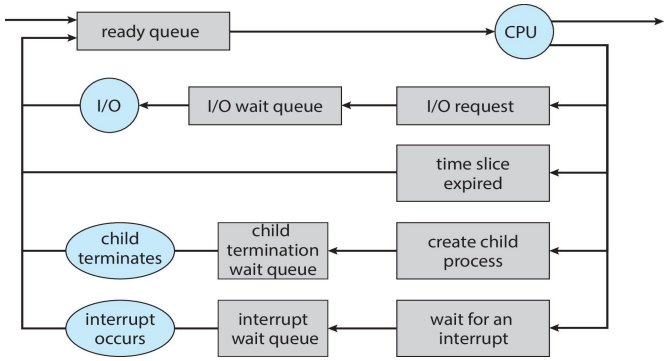
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

OPERATING SYSTEMS
Ready Queue And Various I/O Device Queues



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Representation of Process Scheduling



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Schedulers

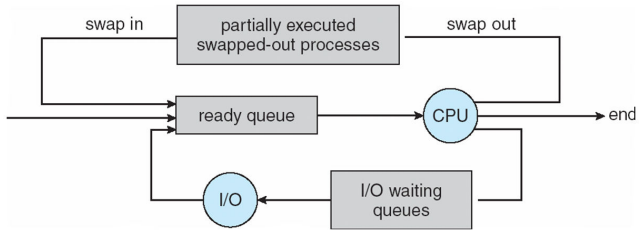
- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**

OPERATING SYSTEMS
Schedulers

- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

OPERATING SYSTEMS
Representation of Process Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

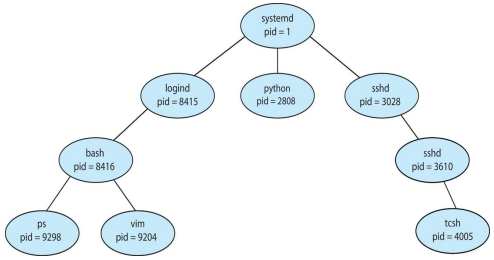
OPERATING SYSTEMS
Operations on Processes

System must provide mechanisms for:

- process creation
- process termination

OPERATING SYSTEMS
Process Creation

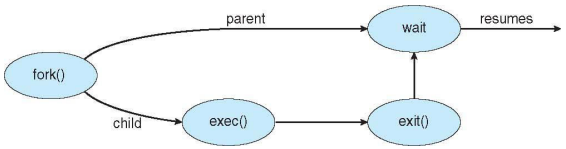
- **Parent** process creates **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Process creation using fork()

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **fork()** system call creates new process
  - **exec()** system call used after a **fork()** to replace the process' memory space with a new program



OPERATING SYSTEMS
C Program forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      execlp("/bin/ls","ls",NULL);
   }
   else { /* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
   }

   return 0;
}
```

OPERATING SYSTEMS
Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
  - Returns status data from child to parent (via **wait()**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call**.** The call returns status information and the pid of the terminated process
  - **pid = wait(&status);**
- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait** , process is an **orphan**

OPERATING SYSTEMS
Process Termination

- **What happens if parent process terminates before child process**
- **What happens if child process terminates before parent(i.e When the parent process in sleep)**
- **How to know the state of the process?**

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

Operating Systems

UE22CS242B

Suresh Jamadagni
Department of Computer Science and Engineering

Operating Systems

System calls for Process Management

Suresh Jamadagni
Department of Computer Science and Engineering

Process Identifiers

- Every process has a unique **process ID**, a non-negative integer
- The process ID is the only well-known identifier of a process that is always unique
- It is often used as a piece of other identifiers, to guarantee uniqueness.
- Although unique, process IDs are reused. As processes terminate, their IDs become candidates for reuse.
- Most UNIX systems implement algorithms to delay reuse, so that newly created processes are assigned IDs different from those used by processes that terminated recently

fork()

- An existing process can create a new one by calling the **fork** function

```
#include <unistd.h>
pid_t fork(void);
```

- The new process created by fork is called the child process
- Return value in the child is 0
- Return value in the parent is the process ID of the new child
- Return value is -1 on error
- The child is a copy of the parent. The child gets a copy of the parent's data space, heap, and stack

exit()

- A process can terminate normally in five ways
  1. Executing a return from the main function
  2. Calling the **exit** function.
  3. Calling the _exit or _Exit function.
  4. Executing a return from the start routine of the last thread in the process.
  5. Calling the pthread_exit function from the last thread in the process

A process can terminate abnormally in three ways

  1. Calling abort
  2. When the process receives certain signals
  3. The last thread responds to a cancellation request

- Regardless of how a process terminates, the same code in the kernel is eventually executed.
- This kernel code closes all the open descriptors for the process, releases the memory that it was using

OPERATING SYSTEMS
wait() and waitpid()

- A process that calls wait or waitpid can
  - Block, if all of its children are still running
  - Return immediately with the termination status of a child, if a child has terminated and is waiting for its termination status to be fetched
  - Return immediately with an error, if it doesn't have any child processes
- If the process is calling wait because it received the SIGCHLD signal, wait will return immediately. But if we call it at any random point in time, it can block.

```
#include <sys/wait.h>
pid_t wait(int *statloc);
pid_t waitpid(pid_t pid, int *statloc, int options);
```

Both return: process ID on success, 0 if state hasn't changed or −1 on failure

OPERATING SYSTEMS
waitid()

- waitid() allows a process to specify which children to wait for.
- Instead of encoding this information in a single argument combined with the process ID or process group ID, two separate arguments are used

```
#include <sys/wait.h>
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

- Returns: 0 if OK, −1 on error

OPERATING SYSTEMS
exec()

- When a process calls one of the exec functions, that process is completely replaced by the new program, and the new program starts executing at its main function.
- The process ID does not change across an exec, because a new process is not created;
- exec merely replaces the current process — its text, data, heap, and stack segments — with a brand-new program from disk.

```
#include <unistd.h>
int execl(const char *pathname, const char *arg0, ... /* (char *)0 */ );
int execv(const char *pathname, char *const argv[]);
int execle(const char *pathname, const char *arg0, ...
          /* (char *)0, char *const envp[] */ );
int execve(const char *pathname, char *const argv[], char *const envp[]);
int execlp(const char *filename, const char *arg0, ... /* (char *)0 */ );
int execvp(const char *filename, char *const argv[]);
int fexecve(int fd, char *const argv[], char *const envp[]);
```

- Return: −1 on error, no return on success

OPERATING SYSTEMS
getpid() and getppid()

```
#include <unistd.h>
pid_t getpid(void);

pid_t getppid(void);
```

- **getpid**() returns the process ID (PID) of the calling process.

- **getppid**() returns the process ID of the parent of the calling process.
- This will be either the ID of the process that created this process using **fork**() or if that process has already terminated, the ID of the process to which this process has been re-parented

OPERATING SYSTEMS
Race condition

- A race condition occurs when multiple processes are trying to do something with shared data and the final outcome depends on the order in which the processes run.
- The fork function is a lively breeding ground for race conditions, if the logic after the fork either explicitly or implicitly depends on whether the parent or child runs first after the fork.
- In general, which process runs first cannot be predicted
- A process that wants to wait for a child to terminate must call one of the wait functions.
- If a process wants to wait for its parent to terminate, a loop of the following form could be used:

```
while (getppid() != 1)
    sleep(1);
```

- The problem with this type of loop, called polling, is that it wastes CPU time, as the caller is awakened every second to test the condition.
- To avoid race conditions and to avoid polling, some form of signaling is used between multiple processes

PES UNIVERSITY
CELEBRATING 50 YEARS

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS
UE22CS242B
CPU Scheduling

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

CPU Scheduling

Suresh Jamadagni
Department of Computer Science

CPU Scheduling - Basic Concepts

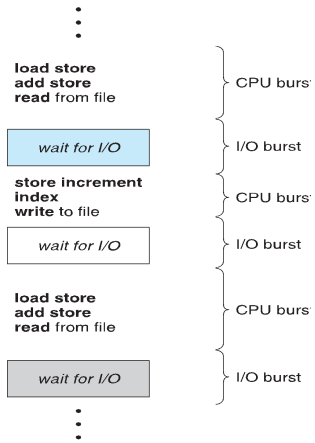
- In a system with a single CPU core, only one process can run at a time. Others must wait until the CPU is free and can be rescheduled.
- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- Several processes are kept in memory at one time.
- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues.
- Every time one process has to wait, another process can take over use of the CPU. On a multicore system, this concept of keeping the CPU busy is extended to all processing cores on the system.

CPU Scheduling - Basic Concepts (Contd)

- A process is executed until it must wait, typically for the completion of some I/O request.
- In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished. With multiprogramming, we try to use this time productively.
- Scheduling of this kind is a fundamental operating-system function.

Alternating Sequence of CPU and I/O bursts

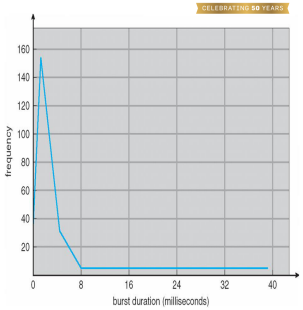
- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Histogram of CPU-burst Times

- The durations of CPU bursts have been measured extensively. Although they vary greatly from process to process and from computer to computer, they tend to have a frequency curve similar to that shown in the Figure.
- An I/O-bound program typically has many short CPU bursts. A CPU-bound program might have a few long CPU bursts. This distribution can be important when implementing a CPU-scheduling algorithm.



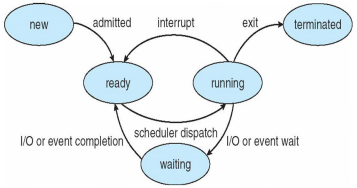
© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
CPU Scheduler

- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
  - Queue may be ordered in various ways
  - It can be FIFO, Priority, queue, tree, unordered linked list
  - The records in the queue are PCB's of the processes

OPERATING SYSTEMS
Preemptive Scheduling

- CPU scheduling decisions may take place when a process
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates
- Scheduling under 1 and 4 is **non-preemptive**
- All other scheduling is **preemptive**
  - Consider access to shared data
  - Consider preemption while in kernel mode
  - Consider interrupts occurring during crucial OS activities
- Scheduling algorithms used in windows3.x, non-preemptive
- Win 95 onwards used preemptive
- Preemptive Scheduling Algorithm is used in Macintosh OS



OPERATING SYSTEMS
Preemptive vs Non-Preemptive Scheduling

- Unfortunately, pre-emptive scheduling can result in race conditions when data are shared among several processes. Ex: While one process is updating the shared data, it is pre-empted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state.

- A pre-emptive kernel requires mechanisms such as mutex locks to prevent race conditions when accessing shared kernel data structures. Most modern operating systems are now fully pre-emptive when running in kernel mode.

OPERATING SYSTEMS
Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

OPERATING SYSTEMS
Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process (performance metric)
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

OPERATING SYSTEMS
Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Scheduling Algorithms

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
FCFS and SJF Scheduling

Suresh Jamadagni
Department of Computer Science

Process	Burst Time
P1	24
P2	3
P3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3

  The Gantt Chart for the schedule is:

  Waiting time for P_1 = 0; P_2 = 24; P_3 = 27

- Average waiting time: (0 + 24 + 27)/3 = 17

FCFS Scheduling (Cont.)

- Suppose that the processes arrive in the order: P_2 , P_3 , P_1

  The Gantt Chart for the schedule is:

- Waiting time for $P_1 = 6$; $P_2 = 0$, $P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3

- ***The average waiting time under an FCFS policy is generally not minimal and may vary substantially if the processes' CPU burst times vary greatly***

FCFS Scheduling (Cont.)

- **Convoy effect** - short process behind long process
  - Consider one CPU-bound and many I/O-bound processes
- FCFS scheduling algorithm is non-preemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU
- FCFS algorithm is thus particularly troublesome for time-sharing systems, where it is important that each process to get a share of the CPU at regular intervals.
- It is not desirable to allow one process to keep the CPU for an extended period

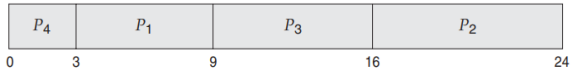
Shortest-Job-First (SJF) Scheduling

- Associate with each process the **length of its next CPU burst**
  - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
  - The difficulty is knowing the length of the next CPU request
  - Compute an approximation of the length of the next CPU burst

Example of SJF Scheduling

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

- SJF scheduling chart



- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

Note: If FCFS scheduling is used, average waiting time = (0 + 6 + 14 + 21) / 4 = 10.25 ms.

OPERATING SYSTEMS
Determining Length of Next CPU Burst

- Can be estimated using the length of previous CPU bursts, using exponential averaging

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n$$

where τ_{n+1} is the predicted value for the next CPU burst,

The parameter α controls the relative weight of recent and past history in the prediction .

Commonly, α = 1/2, so recent history and past history are equally weighted

The value of t_n contains most recent information

The value τ_n stores the past history

- Preemptive version is called **shortest-remaining-time-first**

OPERATING SYSTEMS
Determining Length of Next CPU Burst

Calculate the exponential averaging with $T_1 = 10$, α = 0.5 and the algorithm is SJF with previous runs as 8, 7, 4, 16.

Initially $T_1 = 10$ and α = 0.5 and the run times given are 8, 7, 4, 16 as it is shortest job first,

So the possible order in which these processes would serve will be 4, 7, 8, 16 since SJF is a non-preemptive technique.

So, using formula: $T_2 = \alpha * t_1 + (1-\alpha)T_1$

we have,

$T_2 = 0.5*4 + 0.5*10 = 7$, here $t_1 = 4$ and $T_1 = 10$

$T_3 = 0.5*7 + 0.5*7 = 7$, here $t_2 = 7$ and $T_2 = 7$

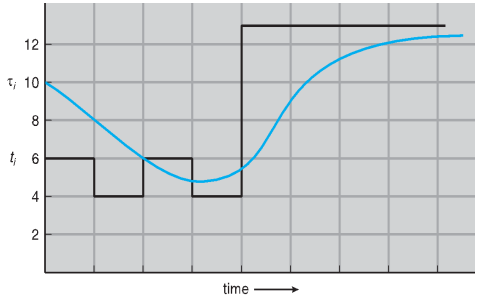
$T_4 = 0.5*8 + 0.5*7 = 7.5$, here $t_3 = 8$ and $T_3 = 7$

$T_5 = 0.5*16 + 0.5*7.5 = 11.8$, here $t_4 = 16$ and $T_4 = 7.5$

So the prediction for 5th burst time will be $T_5 = 11.8$

Src: https://www.geeksforgeeks.org/shortest-job-first-cpu-scheduling-with-predicted-burst-time/

OPERATING SYSTEMS
Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	…
"guess" (τ_i)	10	8	6	6	5	9	11	12	…

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n$$

OPERATING SYSTEMS
Examples of Exponential Averaging

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n$$

- α =0
  - $\tau_{n+1} = \tau_n$
  - Most recent CPU burst does not count
- α =1
  - $\tau_{n+1} = \alpha\, t_n$
  - Only the actual last CPU burst counts
- If we expand the formula, we get:
  - $\tau_{n+1} = \alpha\, t_n + (1-\alpha)\alpha\, t_{n-1} + \ldots$
  - $+ (1-\alpha)^j \alpha\, t_{n-j} + \ldots$
  - $+ (1-\alpha)^{n+1} \tau_0$
- Since both α and (1 - α) are less than or equal to 1, each successive term has less weight than its predecessor

© copyright Silberschatz, Galvin and Gagne, 2013

More examples

Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use non-preemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process	Burst Time
P_1	8
P_2	4
P_3	1

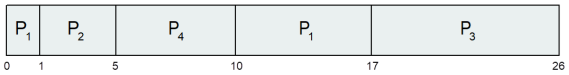
- What is the average wait time for these processes with the FCFS scheduling algorithm?
  Average wait time = (0 + 8 + 12)/3 = 6.67
- What is the average wait time for these processes with the SJF scheduling algorithm?
  Average wait time = (5 + 1 + 0)/3 = 2

Example of Shortest-remaining-time-first

- Preemptive SJF Scheduling is sometimes called SRTF
- Now we add the concepts of varying arrival times and preemption to the analysis

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive* SJF Gantt Chart



- Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5 msec

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

Scheduling Algorithms

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Priority and Round Robin Scheduling

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority)
  - Preemptive
  - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem ≡ **Starvation** – low priority processes may never execute
- Solution ≡ **Aging** – as time progresses increase the priority of the process

OPERATING SYSTEMS
Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority Scheduling Gantt chart

- Average waiting time = (6 + 0 + 16 + 18 + 1) / 5 = 41/5 = 8.2

OPERATING SYSTEMS
Round-Robin Scheduling

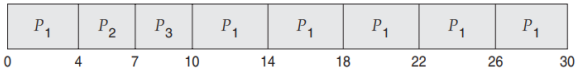
- Round-robin (RR) scheduling algorithm is designed especially for timesharing systems.
- It is similar to FCFS scheduling, but preemption is added to enable the system to switch between processes
- A small unit of time, called a time quantum or time slice, is defined.
- A time quantum is generally from 10 to 100 milliseconds in length
- The ready queue is treated as a circular queue
- The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum

OPERATING SYSTEMS
Round-Robin Scheduling Example

- Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Process	Burst Time
P_1	24
P_2	3
P_3	3

- RR scheduling Gantt chart using a time quantum of 4 milliseconds



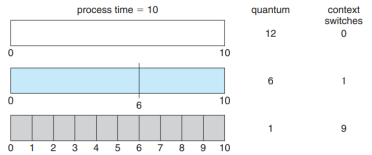
  - P1 waits for 6 milliseconds (10 - 4)
  - P2 waits for 4 milliseconds
  - P3 waits for 7 milliseconds.
- The average waiting time = (6 + 4 + 7)/3 = 5.66 milliseconds

OPERATING SYSTEMS
Round-Robin Scheduling Performance

- If there are n processes in the ready queue and the time quantum is q, then each process gets 1/n of the CPU time in chunks of at most q time units.
- Each process must wait no longer than (n − 1) × q time units until its next time quantum.
  - For example, with five processes and a time quantum of 20 milliseconds, each process will get up to 20 milliseconds every 100 milliseconds.
- Performance of the RR algorithm depends heavily on the size of the time quantum
- If the time quantum is extremely large, the RR policy is the same as the FCFS policy
- If the time quantum is extremely small, the RR approach can result in a large number of context switches

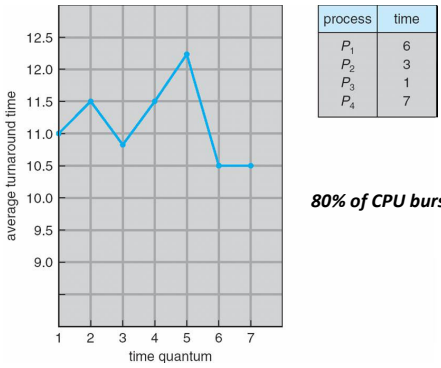
OPERATING SYSTEMS
Example of Round-Robin Scheduling Performance

- Consider only one process of 10 time units.
- If the quantum is 12 time units, the process finishes in less than 1 time quantum, with no overhead.
- If the quantum is 6 time units, the process requires 2 quanta, resulting in one context switch.
- If the time quantum is 1 time unit, then nine context switches will occur, slowing the execution of the process accordingly



- In practice, most modern systems have time quanta ranging from 10 to 100 milliseconds.
- The time required for a context switch is typically less than 10 microseconds. Thus, the context-switch time is a small fraction of the time quantum.

Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than the time quantum

© copyright Silberschatz, Galvin and Gagne, 2013

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Scheduling Algorithms

Suresh Jamadagni
Department of Computer Science

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

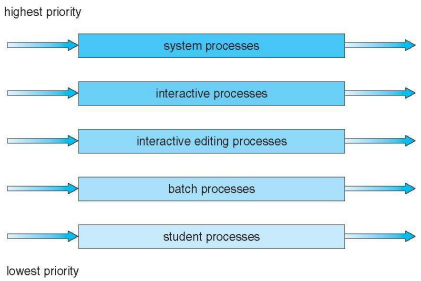
OPERATING SYSTEMS

Multi-level Queue & Feedback Queue Scheduling

Suresh Jamadagni
Department of Computer Science

- Ready queue is partitioned into separate queues:
  - **foreground** (interactive)
  - **background** (batch)
- Process permanently assigned to a queue
- Each queue has its own scheduling algorithm:
  - foreground – RR
  - background – FCFS
- In addition, scheduling must be done between the queues
  - Fixed priority scheduling; (serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS

- Each queue has absolute priority over lower-priority queues
- No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty
- If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.

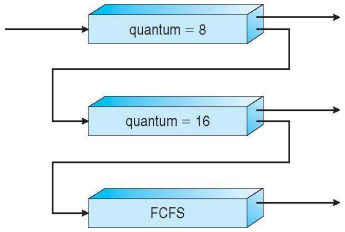


- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to determine when to upgrade a process
  - Method used to determine when to demote a process
  - Method used to determine which queue a process will enter when that process needs service

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS



- Scheduling
  - A new job enters queue Q_0 which is served FCFS
    - When it gains CPU, job receives 8 milliseconds
    - If it does not finish in 8 milliseconds, job is moved to queue Q_1
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds
    - If it still does not complete, it is preempted and moved to queue Q_2

OPERATING SYSTEMS
Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS

- The scheduler first executes all processes in queue 0.
- Only when queue 0 is empty will it execute processes in queue 1.
- Processes in queue 2 will be executed only if queues 0 and 1 are empty.
- A process that arrives for queue 1 will preempt a process in queue 2.
- A process in queue 1 will in turn be preempted by a process arriving in queue 0.

OPERATING SYSTEMS
Multiple-processor Scheduling

If multiple CPUs are available, **load sharing** becomes possible, but scheduling problems become correspondingly more complex.

1. **Asymmetric multiprocessing**
   - CPU scheduling in a multiprocessor system has all scheduling decisions, I/O processing, and other system activities handled by a single processor—the master server.
   - The other processors execute only user code.
   - Asymmetric multiprocessing is simple because only one processor accesses the system data structures, reducing the need for data sharing.
2. **Symmetric multiprocessing (SMP)**
   - Each processor is self-scheduling. All processes may be in a common ready queue, or each processor may have its own private queue of ready processes.
   - All modern operating systems support SMP

OPERATING SYSTEMS
Processor Affinity

- When a process has been running on a specific processor, the data most recently accessed by the process populate the cache of the processor.
- Successive memory accesses by the process are often satisfied in cache memory.
- If the process migrates to another processor, the contents of cache memory must be invalidated for the first processor and the cache for the second processor must be repopulated.
- Because of the high cost of invalidating and repopulating caches, most SMP systems try to avoid migration of processes from one processor to another and instead attempt to keep a process running on the same processor. This is known as **processor affinity**
- **Soft affinity** - OS will attempt to keep a process on a single processor, but it is possible for a process to migrate between processors
- **Hard affinity** – OS provides system calls for process to specify a subset of processors on which it may run

OPERATING SYSTEMS
Load Balancing

- On SMP systems, it is important to keep the workload balanced among all processors to fully utilize the benefits of having more than one processor.
- **Load balancing** attempts to keep the workload evenly distributed across all processors in an SMP system.
- Load balancing is typically necessary only on systems where each processor has its own private queue of eligible processes to execute

1. **Push migration** - a specific task periodically checks the load on each processor and if it finds an imbalance, evenly distributes the load by moving (or pushing) processes from overloaded to idle or less-busy processors
2. **Pull migration** occurs when an idle processor pulls a waiting task from a busy processor.

- Push and pull migration need not be mutually exclusive

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Case Study: Linux/ Windows Scheduling Policies.

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Case Study: Linux/ Windows Scheduling Policies

Suresh Jamadagni
Department of Computer Science

- Process Scheduling in Linux
- Linux kernel ran a variation of standard UNIX scheduling algorithm
- It did not support for SMP systems
- It had poor performance for larger processes

- Kernel moved to constant order *O*(1) scheduling time
- Supported SMP systems - processor affinity and load balancing between processors with good performance
- Poor response times for the interactive processes that are common on many desktop computer systems

- Completely Fair Scheduler (CFS) is the default scheduling algorithm.
- Based on Scheduling classes
  - Each class is assigned a specific priority.
  - Using different scheduling classes, the kernel can accommodate different scheduling algorithms
  - Scheduler picks the highest priority task in the highest scheduling class
  - Two scheduling classes are included, others can be added
    1. A scheduling class with CFS algorithm
    2. A real-time scheduling class

Completely Fair Scheduler (CFS)

- CFS scheduler assigns a **proportion of CPU processing time** to each task.
- Proportion calculated based on **nice value** which ranges from -20 to +19
  - A numerically lower nice value indicates a higher relative priority.
  - Tasks with lower nice values receive a higher proportion of CPU processing time than tasks with higher nice values
- Calculates **target latency** – interval of time during which task should run at least once
  - Proportions of CPU time are allocated from the value of targeted latency computed.
  - Target latency can increase if number of active tasks increase

OPERATING SYSTEMS
Linux Scheduling in Version 2.6.23 + (Cont.)

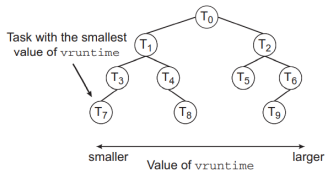
Completely Fair Scheduler (CFS)

- CFS scheduler doesn't directly assign priorities
- CFS scheduler maintains per task **virtual run time** in variable **vruntime**
  - Associated with decay factor based on priority of task – lower priority is higher decay rate
  - Normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time

OPERATING SYSTEMS
Linux Scheduling in Version 2.6.23 + (Cont.)

Completely Fair Scheduler (CFS)

- Each runnable task is placed in a balanced binary search tree whose key is based on the value of **vruntime**
- When a task becomes runnable, it is added to the tree
- When a task is not runnable, it is deleted from the tree
- Navigating the tree to discover the task to run (leftmost node) will require O(lg N) operations (where N is the number of nodes in the tree).



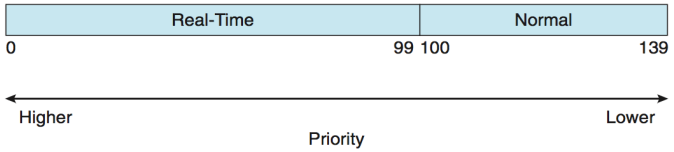
OPERATING SYSTEMS
Linux Scheduling in Version 2.6.23 + (Cont.)

Completely Fair Scheduler (CFS)

- Assume that two tasks have the same nice values.
- One task is I/O-bound and the other is CPU-bound
- The value of **vruntime** will be lower for the I/O-bound task than for the CPU-bound task, giving the I/O-bound task higher priority than the CPU-bound task.
- If the CPU-bound task is executing when the I/O-bound task becomes eligible to run, the I/O-bound task will preempt the CPU-bound task.

OPERATING SYSTEMS
Linux Real Time Scheduling

- Real-time scheduling according to POSIX.1b
  - Real-time tasks have static priorities (0-99)
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139



OPERATING SYSTEMS
Windows Scheduling

- Uses priority based pre-emptive scheduling algorithm
- Scheduler ensures that the **highest-priority thread** will always run
- Windows kernel that handles scheduling is called the **dispatcher**
- Thread selected by the dispatcher runs until it is preempted by a higher-priority thread or until it terminates or until its time quantum ends or until it calls a blocking system call
- Dispatcher uses a 32-level priority scheme
  - **Variable class** is 1-15, **real-time class** is 16-31
  - Priority 0 is memory-management thread
- Queue for each priority class
- If no run-able thread, runs **idle thread**

OPERATING SYSTEMS
Windows Priority Classes

- Windows API identifies several priority classes to which a process can belong
  - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS,NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
  - All are variable except REALTIME
- A thread within a given priority class has a relative priority
  - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Priority class and relative priority combine to give numeric priority
- Base priority is NORMAL within the class
- If quantum expires, priority lowered, but never below base

OPERATING SYSTEMS
Windows Thread Priorities

- A thread within a given priority class also has a relative priority
- Priority of each thread is based on both the priority class it belongs to (top row in the diagram) and its relative priority within that class (left column in the diagram)

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

© copyright Silberschatz, Galvin and Gagne, 2013

Windows Priority Classes (Cont.)

- If wait occurs, priority boosted depending on what was waited for
- Windows distinguishes between foreground process and background processes
- Foreground processes given 3x priority boost
- This priority boost gives the foreground process three times longer to run before a time-sharing preemption occurs.

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

Operating Systems

UE22CS242B

Suresh Jamadagni
Department of Computer Science and Engineering

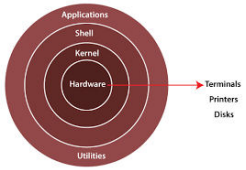
Operating Systems

Bash shell and cron

Suresh Jamadagni
Department of Computer Science and Engineering

OPERATING SYSTEMS
What is a shell?

- The **shell** is the Linux command line interpreter.
- Shell provides an interface between the user and the kernel and executes programs called commands.
- For example, if a user enters ***ls*** then the shell executes the *ls* command.
- The shell can also execute other programs such as applications, scripts, and user programs



OPERATING SYSTEMS
What is a shell?

- Instructions entered in response to the shell prompt have the following syntax: **command [arg1] [arg2] .. [argn]**
- The brackets [] indicate that the arguments are optional. Many commands can be executed with or without arguments
- The shell parses the words or tokens (commandname , options, filesnames[s]) and gets the kernel to execute the commands assuming the syntax is correct.
- Typically, the shell processes the complete line after a carriage return (cr) (carriage return) is entered and finds the program that the command line wants executing.
- The shell looks for the command to execute either in the specified directory if given (./mycommand) or it searches through a list of directories depending on your $PATH variable.

OPERATING SYSTEMS
Various shells

- There are a number of shells available to a Linux user:
  - ***sh*** - This is known as the Borne Shell and is the original shell
  - ***csh, tcsh*** - These are well-known and widely used derivatives of the Borne shell
  - ***ksh*** - The popular Korn shell
  - ***bash*** - The Borne Again SHell is the most popular shell used for linux and developed by GNU

echo $SHELL

```
ubuntu@ip-172-31-41-208:~$ echo $SHELL
/bin/bash
```

OPERATING SYSTEMS
Environment variables

- **Environmental variables** are variables that are defined for the current shell and are inherited by any child shells or processes.
- Environmental variables are used to pass information into processes that are spawned from the shell.
- **Shell variables** are variables that are contained exclusively within the shell in which they were set or defined.
- They are often used to keep track of ephemeral data, like the current working directory.
- By convention, these types of variables are usually defined using all capital letters. This helps users distinguish environmental variables within other contexts.
- We can see a list of all of our environmental variables by using the ***env*** or ***printenv***

OPERATING SYSTEMS
Common Environment variables

- **SHELL**: This describes the shell that will be interpreting any commands you type in. In most cases, this will be bash by default, but other values can be set if you prefer other options.
- **TERM**: This specifies the type of terminal to emulate when running the shell. Different hardware terminals can be emulated for different operating requirements. You usually won't need to worry about this though.
- **USER**: The current logged in user.
- **PWD**: The current working directory.
- **OLDPWD**: The previous working directory. This is kept by the shell in order to switch back to your previous directory by running cd -.
- **PATH**: A list of directories that the system will check when looking for commands. When a user types in a command, the system will check directories in this order for the executable.
- **HOME**: The current user's home directory.

OPERATING SYSTEMS
Common SHELL variables

- **BASHOPTS**: The list of options that were used when bash was executed. This can be useful for finding out if the shell environment will operate in the way you want it to.
- **BASH_VERSION**: The version of bash being executed, in human-readable form.
- **BASH_VERSINFO**: The version of bash, in machine-readable output.

```
ubuntu@ip-172-31-41-208:~$ echo $BASH_VERSION
5.0.17(1)-release
```

OPERATING SYSTEMS
SHELL basics

- Creating shell variables: `ubuntu@ip-172-31-41-208:~$ MY_VAR="Hello world"`
- Accessing the value of any shell or environmental variable:

```
ubuntu@ip-172-31-41-208:~$ echo $MY_VAR
Hello world
```

- Spawning a child shell process

```
ubuntu@ip-172-31-41-208:~$ bash
```

- In the child shell process, the shell variable defined in the parent is not available

```
ubuntu@ip-172-31-41-208:~$ echo $MY_VAR
```

- To terminate the child shell process,

```
ubuntu@ip-172-31-41-208:~$ exit
```

- To pass the shell variables to child shell processes, you need to **export** the variable

```
ubuntu@ip-172-31-41-208:~$ export MY_VAR="Hello world"
ubuntu@ip-172-31-41-208:~$ bash
ubuntu@ip-172-31-41-208:~$ echo $MY_VAR
Hello world
```

OPERATING SYSTEMS
SHELL basics

- Removing a shell variable

```
ubuntu@ip-172-31-41-208:~$ unset MY_VAR
ubuntu@ip-172-31-41-208:~$ echo $MY_VAR

ubuntu@ip-172-31-41-208:~$
```

- For setting environment variables at login, edit the **.profile** file in the **$HOME** directory and add the export command

```
ubuntu@ip-172-31-41-208:~$ cd $HOME
ubuntu@ip-172-31-41-208:~$ vi .profile
```

OPERATING SYSTEMS
SHELL control flow - if

```
if commands; then
    commands
[elif commands; then
    commands...]
[else
    commands]
fi
```

Example: To check if the file exists

```
#!/bin/bash

if [ -f welcome.txt ]; then
   echo "File exists"
else
   echo "File does not exist"
fi
```

OPERATING SYSTEMS
SHELL control flow - loops

```
#!/bin/bash

for i in {1..5}
do
   echo "i = $i"
done
```

Nested loop:

```
#!/bin/bash

for i in {1..5}
do

   for j in {1..3}
   do
      echo "i = $i and j = $j"
   done
done
```

OPERATING SYSTEMS
SHELL control flow – continue and break

```
#!/bin/bash

for i in {1..5}
do

   if [ $i -eq 3 ]
   then
      continue
   fi

   for j in {1..3}
   do
      echo "i = $i and j = $j"
   done
done
```

```
#!/bin/bash

for i in {1..5}
do

   if [ $i -eq 3 ]
   then
      break
   fi

   for j in {1..3}
   do
      echo "i = $i and j = $j"
   done
done
```

OPERATING SYSTEMS
cron

- In Unix and Linux, **cron** is a daemon, which is an unattended program that runs continuously in the background and wakes up (executes) to handle periodic service requests when required. The daemon runs when the system boots
- Cron is a job scheduling utility present in Unix like systems. The **crond** daemon enables cron functionality and runs in background.
- The cron reads the **crontab** (cron tables) for running predefined scripts.
- By using a specific syntax, you can configure a cron job to schedule scripts or other commands to run automatically.
- Checking for jobs scheduled in **cron**

```
ubuntu@ip-172-31-41-208:~$ crontab -l
no crontab for ubuntu
```

OPERATING SYSTEMS
cron

- Starting/checking status of **cron**

```
ubuntu@ip-172-31-41-208:~$ sudo systemctl status cron.service
● cron.service - Regular background program processing daemon
     Loaded: loaded (/lib/systemd/system/cron.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2024-01-24 06:44:21 UTC; 1h 8min ago
```

- Adding jobs to the scheduler
- **crontab -e**: edits crontab entries to add, delete, or edit cron jobs.

```
*  *  *  *  *  sh /path/to/script/script.sh
|  |  |  |  |           |
|  |  |  |  |           Command or Script to Execute
|  |  |  |  |
|  |  |  |  |
|  |  |  |  Day of the Week(0-6)
|  |  |  |
|  |  |  Month of the Year(1-12)
|  |  |
|  |  Day of the Month(1-31)
|  |
|  Hour(0-23)
|
Min(0-59)
```

OPERATING SYSTEMS
cron examples

SCHEDULE	SCHEDULED VALUE
5 0 * 8 *	At 00:05 in August.
5 4 * * 6	At 04:05 on Saturday.
0 22 * * 1-5	At 22:00 on every day-of-week from Monday through Friday.

OPERATING SYSTEMS
cron examples

Executing a program every minute
crontab -l

```
# m h  dom mon dow   command
*/1 * * * * /bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log
```

```
ubuntu@ip-172-31-41-208:~/OS$ tail /var/log/syslog
Jan 24 08:38:01 ip-172-31-41-208 CRON[2923]: (ubuntu) CMD ($HOME/OS/hello.exe)
Jan 24 08:38:01 ip-172-31-41-208 CRON[2920]: (CRON) info (No MTA installed, discarding output)
Jan 24 08:39:01 ip-172-31-41-208 CRON[2927]: (ubuntu) CMD ($HOME/OS/hello.exe)
Jan 24 08:39:01 ip-172-31-41-208 CRON[2926]: (CRON) info (No MTA installed, discarding output)
Jan 24 08:39:09 ip-172-31-41-208 crontab[2911]: (ubuntu) REPLACE (ubuntu)
Jan 24 08:39:09 ip-172-31-41-208 crontab[2911]: (ubuntu) END EDIT (ubuntu)
Jan 24 08:39:41 ip-172-31-41-208 crontab[2932]: (ubuntu) LIST (ubuntu)
Jan 24 08:40:01 ip-172-31-41-208 cron[665]: (ubuntu) RELOAD (crontabs/ubuntu)
Jan 24 08:40:01 ip-172-31-41-208 CRON[2935]: (ubuntu) CMD (/bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log)
Jan 24 08:41:01 ip-172-31-41-208 CRON[2939]: (ubuntu) CMD (/bin/bash $HOME/OS/hello.exe &> $HOME/OS/cron_output.log)
```

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

IPC - Shared Memory & Message Passing

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

IPC - Shared Memory and Message Passing

Suresh Jamadagni
Department of Computer Science

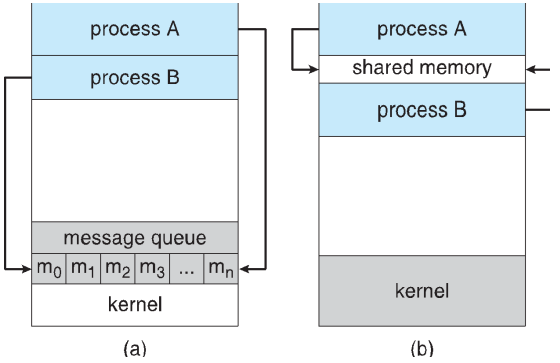
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

OPERATING SYSTEMS
Cooperating Processes

- ***Independent*** process cannot affect or be affected by the execution of another process
- ***Cooperating*** process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - Information sharing
  - Computation speed-up
  - Modularity
  - Convenience

OPERATING SYSTEMS
Communication Models

- Two models of IPC
  a) **Message passing**
  b) **Shared memory**



OPERATING SYSTEMS
Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
  - **unbounded-buffer** places no practical limit on the size of the buffer
    - Consumer may have to wait for new items, but the producer can always produce new items
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Consumer must wait if the buffer is empty; producer must wait if the buffer is full

OPERATING SYSTEMS
Bounded-Buffer – Producer Consumer

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Shared buffer is implemented as a circular array with 2 logical pointers: **in** and **out**
- Buffer is empty when **in == out;** buffer is full when ((**in** + 1) % BUFFER_SIZE) == **out**
- Variable **in** points to the next free position in the buffer; **out** points to the first full position in the buffer
- Solution is correct, but can only use BUFFER_SIZE-1 elements

OPERATING SYSTEMS
Bounded-Buffer – Producer

```
item next_produced;
while (true) {
        /* produce an item in next_produced */
        while (((in + 1) % BUFFER_SIZE) == out)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
}
```

OPERATING SYSTEMS
Bounded-Buffer – Consumer

```
item next_consumed;
while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next_consumed */
}
```

OPERATING SYSTEMS
Interprocess Communication – Shared Memory

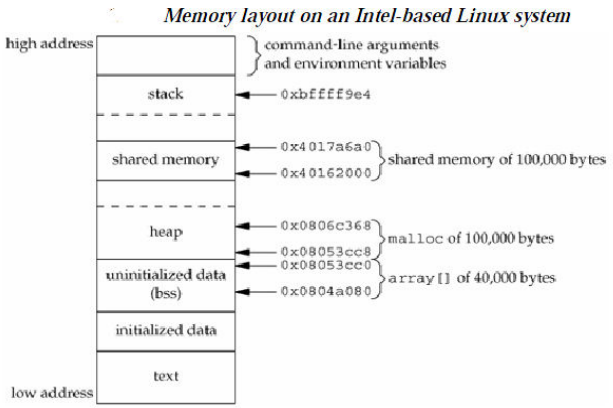
- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

OPERATING SYSTEMS
Shared Memory (Cont.)

- Shared memory allows two or more processes to share a given region of memory.
- Shared memory is the fastest form of IPC, because the data does not need to be copied between the client and the server.
- The only trick in using shared memory is synchronizing access to a given region among multiple processes.
- If the server is placing data into a shared memory region, the client shouldn't try to access the data until the server is done.
- Often, semaphores are used to synchronize shared memory access. (record locking can also be used.)

(A semaphore is a value in an operating system (OS) that controls resource access.)

OPERATING SYSTEMS
Shared Memory (Cont.)



OPERATING SYSTEMS
Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*)
  - **receive**(*message*)
- The *message* size is either fixed or variable

OPERATING SYSTEMS
Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?

OPERATING SYSTEMS
Message Passing (Cont.)

- Implementation of communication link
  - Physical:
    - Shared memory
    - Hardware bus
    - Network
  - Logical:
    - Direct or indirect
    - Synchronous or asynchronous
    - Automatic or explicit buffering

OPERATING SYSTEMS
Direct Communication

- Processes must name each other explicitly:
  - **send** (*P, message*) – send a message to process P
  - **receive**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional

OPERATING SYSTEMS
Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional

OPERATING SYSTEMS
Indirect Communication (Cont.)

- Operations
  - create a new mailbox (port)
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:
- **send**(*A, message*) – send a message to mailbox A
- **receive**(*A, message*) – receive a message from mailbox A

OPERATING SYSTEMS
Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1 sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

OPERATING SYSTEMS
Message Passing - Synchronization

- Message passing may be either blocking or non-blocking
- Blocking is considered synchronous
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- Non-blocking is considered asynchronous
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a rendezvous between the sender and the receiver

OPERATING SYSTEMS
Message Passing - Synchronization (Cont.)

- Producer-consumer becomes trivial

```
 message next_produced;
while (true) {
   /* produce an item in next_produced */
send(next_produced);
}
```

```
message next_consumed;
while (true) {
  receive(next_consumed);

  /* consume the item in next_consumed */
}
```

OPERATING SYSTEMS
Buffering

- Queue of messages attached to the link (direct or indirect); messages reside in a temporary queue.
- Queues can be implemented in one of three ways
  1. Zero capacity – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits
- Zero-capacity case is sometimes referred to as a message system with no buffering; other cases are referred to as systems with automatic buffering

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

UE20CS254

Operating Systems

Suresh Jamadagni
Department of Computer Science
and Engineering

Operating Systems

System calls for Inter Process Communication

Suresh Jamadagni
Department of Computer Science and Engineering

Pipes

- Pipes are the oldest form of UNIX System IPC and are provided by all UNIX systems.
- Pipes have two limitations.
  1. Historically, they have been half duplex (i.e., data flows in only one direction). Some systems now provide full-duplex pipes, but for maximum portability, we should never assume that this is the case.
  2. Pipes can be used only between processes that have a **common ancestor**. Normally, a pipe is created by a process, that process calls fork, and the pipe is used between the parent and the child.

pipe()

- A pipe is created by calling the **pipe()** function.

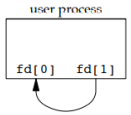
```
#include <unistd.h>
int pipe(int fd[2]);
```

- Return value: 0 if OK, −1 on error
- Two file descriptors are returned through the fd argument:
  1. fd[0] is open for reading
  2. fd[1] is open for writing.
- The output of fd[1] is the input for fd[0]

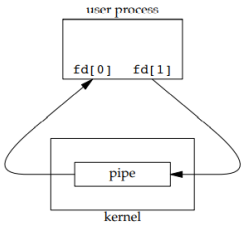
OPERATING SYSTEMS
pipe()

- Two ways to picture a half-duplex pipe

1. The two ends of the pipe connected in a single process

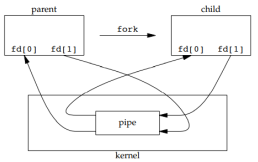


2. The data in the pipe flows through the kernel

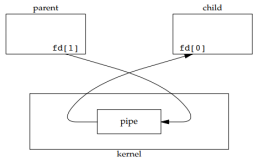


OPERATING SYSTEMS
pipe()

- Normally, a process that calls pipe then calls fork, creating an IPC channel from the parent to the child or vice versa



- For a pipe from the parent to the child, the parent closes the read end of the pipe (fd[0]) and the child closes the write end (fd[1]).
- For a pipe from the child to the parent, the parent closes fd[1], and the child closes fd[0]



Pipe from parent to child

OPERATING SYSTEMS
popen() and pclose()

- A common operation is to create a pipe to another process to either read its output or send it input, the standard I/O library has historically provided the **popen()** and **pclose()** functions
- These two functions handle all the work: creating a pipe, forking a child, closing the unused ends of the pipe, executing a shell to run the command, and waiting for the command to terminate.

```
#include <stdio.h>
FILE *popen(const char *cmdstring, const char *type);

int pclose(FILE *fp);
```

- The function **popen()** does a fork and exec to execute the cmdstring and returns a standard I/O file pointer.
- If type is "r", the file pointer is connected to the standard output of cmdstring.
- If type is "w", the file pointer is connected to the standard input of cmdstring
- The function popen() returns file pointer if OK, NULL on error
- The function **pclose()** closes the standard I/O stream, waits for the command to terminate, and returns the termination status of the shell or -1 on error

OPERATING SYSTEMS
FIFOs

- FIFOs are sometimes called **named pipes**.
- Unnamed pipes can be used only between related processes when a common ancestor has created the pipe.
- With FIFOs, unrelated processes can exchange data
- Creating a FIFO is similar to creating a file

```
#include <sys/stat.h>
int mkfifo(const char *path, mode_t mode);
int mkfifoat(int fd, const char *path, mode_t mode);
```

- Functions mkfifo() and mkfifoat() return 0 if OK, −1 on error
- Once a FIFO has been created using mkfifo() or mkfifoat() , it can be opened using open(). Normal file I/O functions (e.g., close, read, write, unlink) all work with FIFOs.
- There are two uses for FIFOs.
  1. FIFOs are used by shell commands to pass data from one shell pipeline to another without creating intermediate temporary files.
  2. FIFOs are used as rendezvous points in client–server applications to pass data between the clients and the servers

OPERATING SYSTEMS
Message Queues

- A message queue is a linked list of messages stored within the kernel and identified by a message queue identifier
- A new queue is created or an existing queue opened by **msgget**.

```
#include <sys/msg.h>
int msgget(key_t key, int flag);
```

- Returns message queue ID if OK, −1 on error
- New messages are added to the end of a queue by **msgsnd**.

```
#include <sys/msg.h>
int msgsnd(int msqid, const void *ptr, size_t nbytes, int flag);
```

- Every message has a positive long integer type field, a non-negative length, and the actual data bytes all of which are specified to **msgsnd** when the message is added to a queue.
- Messages are fetched from a queue by **msgrcv**.

```
#include <sys/msg.h>
ssize_t msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);
```

- Messages need not be fetched in a first-in, first-out order. Instead, can be fetched based on their type field

OPERATING SYSTEMS
Semaphores

- A semaphore is a counter used to provide access to a shared data object for multiple processes.
- To obtain a shared resource, a process needs to do the following:
  1. Test the semaphore that controls the resource.
  2. If the value of the semaphore is positive, the process can use the resource. In this case, the process decrements the semaphore value by 1, indicating that it has used one unit of the resource.
  3. Otherwise, if the value of the semaphore is 0, the process goes to sleep until the semaphore value is greater than 0. When the process wakes up, it returns to step 1.
- When a process is done with a shared resource that is controlled by a semaphore, the semaphore value is incremented by 1.
- If any other processes are asleep, waiting for the semaphore, they are awakened.

OPERATING SYSTEMS
Semaphores

- First need to obtain a semaphore ID by calling the semget function

```
#include <sys/sem.h>
int semget(key_t key, int nsems, int flag);
```

- Returns: semaphore ID if OK, −1 on error
- **semctl** function is the catchall for various semaphore operations.

```
#include <sys/sem.h>
int semctl(int semid, int semnum, int cmd, ... /* union semun arg */ );
```

- The function **semop** atomically performs an array of operations on a semaphore set

```
#include <sys/sem.h>
int semop(int semid, struct sembuf semoparray[], size_t nops);
```

- The semoparray argument is a pointer to an array of semaphore operations
- Returns 0 if OK, −1 on error

OPERATING SYSTEMS
Shared memory

- Shared memory allows two or more processes to share a given region of memory.
- This is the fastest form of IPC, because the data does not need to be copied between the client and the server
- The challenge in using shared memory is synchronizing access to a given region among multiple processes.
- Semaphores and mutexes are used to synchronize shared memory access
- Function **shmget** is used to obtain a shared memory identifier

```
#include <sys/shm.h>
int shmget(key_t key, size_t size, int flag);
```

- Returns shared memory ID if OK, −1 on error

- Function **shmctl** is the catchall for various shared memory operations

```
#include <sys/shm.h>
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

- Returns 0 if OK, −1 on error
- The cmd argument specifies one of the commands to be performed on the segment specified by shmid
- Once a shared memory segment has been created, a process is attached to this memory segment by calling **shmat**

```
#include <sys/shm.h>
void *shmat(int shmid, const void *addr, int flag);
```

- Returns: pointer to shared memory segment if OK, −1 on error

- Function call **shmdt** will detach a shared memory segment from a process

```
#include <sys/shm.h>
int shmdt(const void *addr);
```

- Returns 0 if OK, −1 on error
- A shared memory segment can be removed by calling **shmctl** with a command of IPC_RMID.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Threads and Concurrency

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Threads and Concurrency

Suresh Jamadagni
Department of Computer Science

- A thread is a sequence of instructions that an operating system (OS) allocates processor time to execute.
- Threads are the basic unit of execution in an OS.

How do threads work?

- A thread is a sequential flow of control within a program.
- A process is a program that is running in a computer.
- A process can have multiple threads running at the same time. This is called multithreading.
- Threads share resources like memory with other threads in the same process.
- The OS manages all the threads in a system.

Types of threads

- User-level threads: These threads are related to user applications.
- Kernel-level threads: These threads are managed by the OS itself.

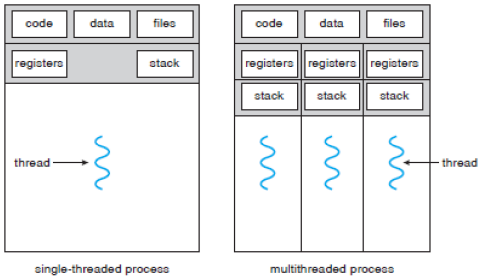
Benefits of threads

- Threads enable concurrent execution, parallelism, and improved system performance.
- For example, you can download and watch a video at the same time using multithreading.

- A Thread is a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems.
- It consists of thread ID, Program counter, a register set and stack
- Shares with other threads of same process its code, data,file descriptors, signals
- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads.
- Application 1: internet browser.
  - numerous tabs open at a given time
  - Multiple threads of execution are used to load content, display animations, play a video, fetch data from a network and so on.

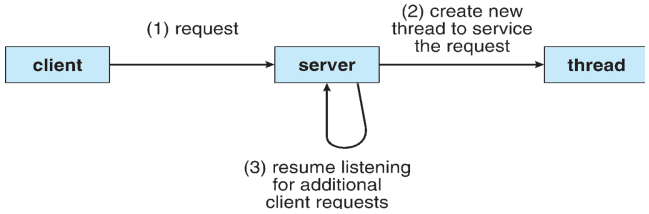
OPERATING SYSTEMS
Single threaded and multithreaded processes

- Application 2: Word processor
  - Thread to accept input
  - A thread for spell checking
  - A thread for grammar checking



OPERATING SYSTEMS
Multithreaded Server Architecture

- Process creation is heavy-weight while thread creation is light-weight
- Process creation if time consuming, resource intensive
- Threads also play a vital role in remote procedure call (RPC) systems
- Can simplify code, increase efficiency
- Kernels are generally multithreaded



OPERATING SYSTEMS
Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked or performing lengthy operations.
  - It is useful in designing user interfaces.
  - A multithreaded Web browser could allow user interaction in one thread while an image was being loaded in another thread.
  - Example: click on the button
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing.
  - Programmer needs to specify the techniques for sharing
  - But threads share the memory and other resources
  - Sharing of resources helps in having many threads within the same address space

OPERATING SYSTEMS
Benefits

- **Economy –** cheaper than process creation, thread switching lower overhead than context switching.
  - In Solaris, for example, creating a process is about thirty times slower than creating a thread, and context switching is about five times slower.
- **Scalability –** process can take advantage of multiprocessor architectures.
  - Threads can run on multiple cores parallelly

OPERATING SYSTEMS
Processes vs Threads

Process	Thread
• Will by default not share memory	• Will by default share memory
• Most file descriptors not shared	• Will share file descriptors
• Don't share filesystem context	• Will share filesystem context
• Don't share signal handling	• Will share signal handling

OPERATING SYSTEMS
Attributes shared by Threads

- process ID and parent process ID;process group ID and session ID;
- controlling terminal;
- process credentials (user and group Ids);open file descriptors;
- record locks created using fcntl();signal dispositions;
- file system–related information: umask, cwd and root directory;
- resource limits;CPU time consumed (as returned by times());
- resources consumed (as returned by getrusage()); nice value (set by setpriority() and nice()).

OPERATING SYSTEMS
Attributes specific to Threads

- thread ID ;signal mask;
- Thread-specific data ;
- the errno variable;
- floating-point environment (see fenv(3));
- stack (local variables and function call linkage information i.e CPU registers saved in the called function's stack frame when one function calls another function and restored for the calling function when the called function returns)
- and a few more …….

OPERATING SYSTEMS
Thread Concepts

- A thread consists of the following information necessary to represent an execution context within a process.
- thread ID that identifies the thread within a process
- Every thread has a thread ID
- set of register values
- stack
- scheduling priority and policy
- signal mask
- Errno variable
- Thread-specific/thread-private data (each thread to access its own separate copy of the data, without worrying about synchronizing access with other threads)

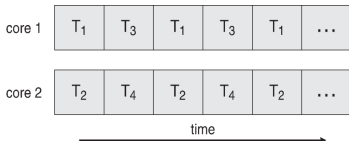
OPERATING SYSTEMS
Thread Concepts (Cont.)

- When a thread is created, there is no guarantee which will run first: the newly created thread or the calling thread.
- The newly created thread has access to the process address space and inherits the calling thread's floating-point environment and signal mask
- The set of pending signals for the thread is cleared.
- The pthread functions usually return an error code when they fail. They don't set errno like the other POSIX functions.

OPERATING SYSTEMS
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**

- **Parallelism on a multi-core system:**



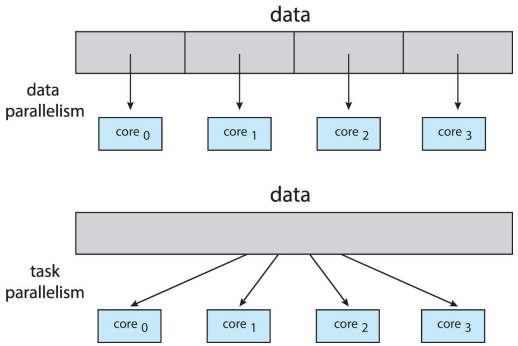
OPERATING SYSTEMS
Multicore Programming

- Multicore or multiprocessor systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

OPERATING SYSTEMS
Multicore Programming (Cont.)

- Types of parallelism
  - Data parallelism – distributes subsets of the same data across multiple cores, same operation on each
  - Task parallelism – distributing threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
  - CPUs have cores as well as ***hardware threads***
  - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core

OPERATING SYSTEMS
Data and Task Parallelism



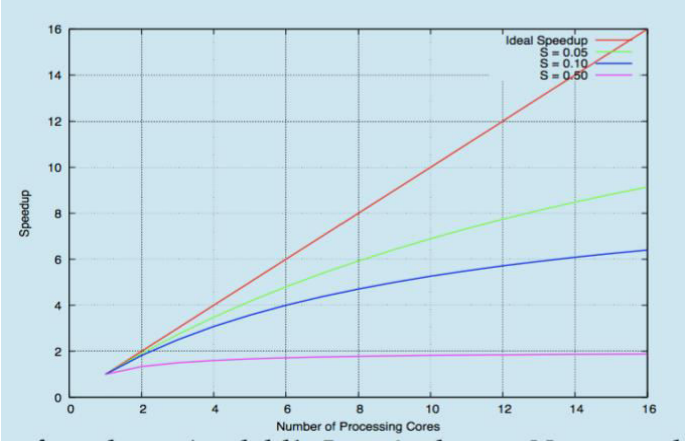
OPERATING SYSTEMS
Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is portion of an application that needs to be done in serial
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

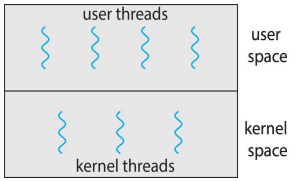
- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*
  Serial portion of an application has disproportionate effect on performance gained by adding additional cores
- But does the law take into account contemporary multicore systems?

OPERATING SYSTEMS
Amdahl's Law



OPERATING SYSTEMS
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general purpose operating systems, including:
  - Windows
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X



THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

Multithreading Models, Thread Creation and Scheduling

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Multithreading Models, Thread Creation and Scheduling

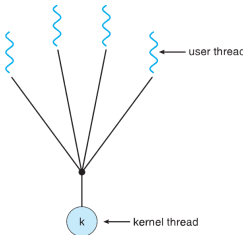
Suresh Jamadagni
Department of Computer Science

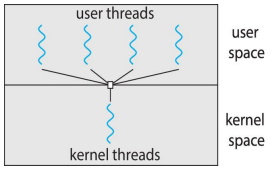
OPERATING SYSTEMS
Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

OPERATING SYSTEMS
Many-to-One

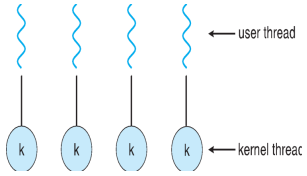
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**

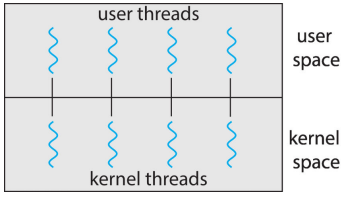




OPERATING SYSTEMS
One-to-One

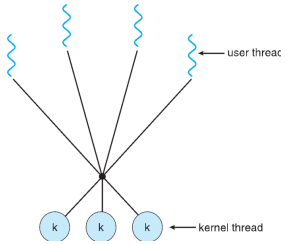
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux
  - Solaris 9 and later

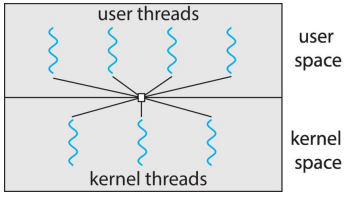




OPERATING SYSTEMS
Many-to-Many Model

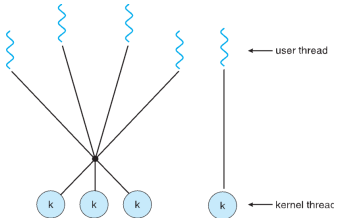
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the *ThreadFiber* package

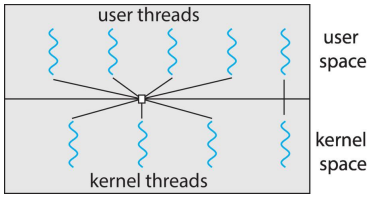




OPERATING SYSTEMS
Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread
- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier





© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

OPERATING SYSTEMS
Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
  pthread_t tid; /* the thread identifier */
  pthread_attr_t attr; /* set of thread attributes */

  if (argc != 2) {
    fprintf(stderr,"usage: a.out <integer value>\n");
    return -1;
  }
  if (atoi(argv[1]) < 0) {
    fprintf(stderr,"%d must be >= 0\n",atoi(argv[1]));
    return -1;
  }
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Pthreads Example

```
  /* get the default attributes */
  pthread_attr_init(&attr);
  /* create the thread */
  pthread_create(&tid,&attr,runner,argv[1]);
  /* wait for the thread to exit */
  pthread_join(tid,NULL);

  printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
  int i, upper = atoi(param);
  sum = 0;

  for (i = 1; i <= upper; i++)
    sum += i;

  pthread_exit(0);
}
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Thread Scheduling

- Distinction between user-level and kernel-level threads
- When threads supported, threads scheduled, not processes
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP.
- Scheduling of user level threads (ULT) to kernel level threads (KLT) via leight-weight process (LWP) by the application developer.
  - **Leightweight Process (LWP) :** Light-weight process are threads in the user space that acts as an interface for the ULT to access the physical CPU resources
- Scheduling of kernel level threads by the system scheduler to perform different unique OS functions.

OPERATING SYSTEMS
Thread Scheduling

- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
  - Known as **process-contention scope (PCS)** since scheduling competition is within the process
  - Typically done via priority set by programmer
- Kernel thread scheduled onto available CPU is **system-contention scope (SCS)** – competition among all threads in system

OPERATING SYSTEMS
Pthread Scheduling

- API allows specifying either PCS or SCS during thread creation
  - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
  - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Can be limited by OS – Linux and Mac OS X only allow PTHREAD_SCOPE_SYSTEM

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM_THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");
      else
         fprintf(stderr, "Illegal scope value.\n");
   }
```

OPERATING SYSTEMS
Pthread Scheduling API

```
/* set the scheduling algorithm to PCS or SCS */
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
  /* create the threads */
  for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i],&attr,runner,NULL);
  /* now join on each thread */
  for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
  /* do some work ... */
  pthread_exit(0);
}
```

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Threads and Concurrency

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Pthreads and Windows Threads

Suresh Jamadagni
Department of Computer Science

- Provides the programmer with an API for creating and managing threads.
- Two primary ways of implementing a thread library.
  - Provide a library entirely in user space with no kernel support.
    - All code and data structures for the library exist in user space.
  - Implement a kernel-level library
    - Code and data structures for the library exist in kernel space.
    - Invoking a function directly by the operating system.

OPERATING SYSTEMS
Thread Libraries: Pthreads

- Three main thread libraries are in use today:
  (1) POSIX Pthreads,
  (2)Win32,
  (3) Java.
- Pthreads, the threads extension of the POSIX standard, may be provided as either a user- or kernel-level library.
- The Win32 thread library is a kernel-level library available on Windows systems.
- The Java thread API allows threads to be created and managed directly in Java programs.

OPERATING SYSTEMS
Thread Libraries: Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

OPERATING SYSTEMS
Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
  pthread_t tid; /* the thread identifier */
  pthread_attr_t attr; /* set of thread attributes */

  if (argc != 2) {
    fprintf(stderr,"usage: a.out <integer value>\n");
    return -1;
  }
  if (atoi(argv[1]) < 0) {
    fprintf(stderr,"%d must be >= 0\n",atoi(argv[1]));
    return -1;
  }
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Pthreads Example

```
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid,&attr,runner,argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
  pthread_join(workers[i], NULL);
```

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS
Windows threads

- Windows implements the Win32 API as its primary API.
- A Windows application runs as a separate process, and each process may contain one or more threads.
- Windows uses the one-to-one mapping.
- Windows also provides support for a **fiber** library, which provides the functionality of the many-to-many model

OPERATING SYSTEMS
Windows threads

- The general components of a thread include:
  - A thread ID uniquely identifying the thread
  - A register set representing the status of the processor.
  - A user stack, employed when the thread is running in user mode, and a kernel stack, employed when the thread is running in kernel mode
  - A private storage area various run-time libraries and dynamic link libraries (DLLs)

OPERATING SYSTEMS
Windows Threads

- Threads are created in theWin32 API using the CreateThread() function the Win32 API.
- Using the WaitForSingleObject() function,which causes the creating thread to block until the child thread has exited

OPERATING SYSTEMS
Windows Multithreaded C Program

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* the thread runs in this separate function */
DWORD WINAPI Summation(LPVOID Param)
{
   DWORD Upper = *(DWORD*)Param;
   for (DWORD i = 0; i <= Upper; i++)
      Sum += i;
   return 0;
}

int main(int argc, char *argv[])
{
   DWORD ThreadId;
   HANDLE ThreadHandle;
   int Param;

   if (argc != 2) {
      fprintf(stderr,"An integer parameter is required\n");
      return -1;
   }
   Param = atoi(argv[1]);
   if (Param < 0) {
      fprintf(stderr,"An integer >= 0 is required\n");
      return -1;
   }
```

Windows Multithreaded C Program (Cont.)

```
   /* create the thread */
   ThreadHandle = CreateThread(
      NULL, /* default security attributes */
      0, /* default stack size */
      Summation, /* thread function */
      &Param, /* parameter to thread function */
      0, /* default creation flags */
      &ThreadId); /* returns the thread identifier */

   if (ThreadHandle != NULL) {
       /* now wait for the thread to finish */
      WaitForSingleObject(ThreadHandle,INFINITE);

      /* close the thread handle */
      CloseHandle(ThreadHandle);

      printf("sum = %d\n",Sum);
   }
}
```

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Mutual Exclusion and Synchronization

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Software Approaches

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Background

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- Illustration of the problem:

  Suppose that we wanted to provide a solution to the consumer-producer problem that fills ***all*** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

OPERATING SYSTEMS
Producer

```
while (true) {
        /* produce an item in next_produced */

        while (counter == BUFFER_SIZE) ;
                /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE; /* Variable in points to the next free position in the buffer */
        counter++;
}
```

OPERATING SYSTEMS
Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE; /* Variable out points to the first full position in the buffer */
        counter--;
        /* consume the item in next consumed */
}
```

OPERATING SYSTEMS
Race Condition

- **counter++** could be implemented as

  register1 = counter
  register1 = register1 + 1
  counter = register1

- **counter--** could be implemented as

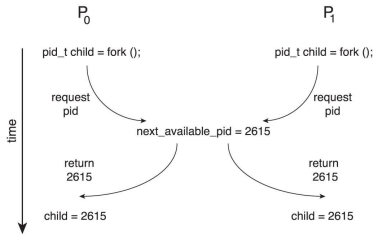
  register2 = counter
  register2 = register2 - 1
  counter = register2

- Consider this execution interleaving with "count = 5" initially:

  S0: producer execute **register1 = counter** {register1 = 5}
  S1: producer execute **register1 = register1 + 1** {register1 = 6}
  S2: consumer execute **register2 = counter** {register2 = 5}
  S3: consumer execute **register2 = register2 – 1** {register2 = 4}
  S4: producer execute **counter = register1** {counter = 6 }
  S5: consumer execute **counter = register2** {counter = 4}

OPERATING SYSTEMS
Race Condition (Another Example)

- Processes P_0 and P_1 are creating child processes using the fork() system call
- Race condition on kernel variable next_available_pid which represents the next available process identifier (pid)



- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable next_available_pid the same pid could be assigned to two different processes!

OPERATING SYSTEMS
Critical Section Problem

- Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

OPERATING SYSTEMS
Critical Section

- General structure of process P_i

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

OPERATING SYSTEMS
Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes

OPERATING SYSTEMS
Critical-Section Handling in OS

Two approaches depending on if kernel is preemptive or non- preemptive

- **Preemptive** – allows preemption of process when running in kernel mode.
  - Preemptive kernels are difficult to design on SMP architectures
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - Essentially free of race conditions in kernel mode
  - It is free from race conditions on kernel data structures
- Preemptive kernel may be more responsive. Suitable for real-time systems.

OPERATING SYSTEMS
Peterson's Solution

- Software-based solution to the Critical Section problem.
- Good algorithmic description of solving the problem
- Assume that the **load** and **store** machine-language instructions are atomic; that is, cannot be interrupted
- **Peterson's Solution** restricted to two process.
- P_i and P_j are two process, j=1-i
- Peterson solution requires two processes share two data items:
  - int turn;
  - Boolean flag[2]
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = *true*** implies that process P_i is ready!

OPERATING SYSTEMS
Algorithm for Process P_i

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j);
            /* critical section */
    flag[i] = false;
            /* remainder section */
}
```

The structure of process P_i in Peterson's solution

Software Solution

- To prove solution is correct, we need show
- Mutual exclusion is preserved

  P_i enters critical section only if:

  turn = i

  and **turn** cannot be both 0 and 1 at the same time
- What about the Progress requirement?
- What about the Bounded-waiting requirement?

Correctness of Peterson's Solution

- Provable that the three CS requirement are met:
  1. Mutual exclusion is preserved

     P_i enters CS only if:

     either **flag[j] = false** or **turn = i**
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

Synchronization

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Principles of concurrency
Synchronization Hardware

Suresh Jamadagni
Department of Computer Science

Code for process i

```
do{
flag[i]=TRUE
turn=j
while(flag[j]&&turn==j);//Do-nop
  critical section
flag[i]=FALSE;
Reminder section
}while(TRUE)
```

Code for process j

```
do{
flag[j]=TRUE
turn=i
while(flag[i]&&turn==i);//Do-nop
  critical section
flag[j]=FALSE;
Reminder section
}while(TRUE)
```


- **Principles of Concurrency**
- relative speed of execution of processes is not predictable.
- system interrupts are not predictable
- scheduling policies may vary

OPERATING SYSTEMS
Synchronization Hardware

- Software based solutions are not guaranteed to work on modern computer architectures
- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**
  - Protecting critical regions via locks.
- synchronization can be done through Lock & Unlock technique
- Locking part is done in the Entry Section. After locking the process enter critical section.
- The process is moved to the Exit Section after it is done with execution in CS.
- Unlock is done in exit section.
- This process is designed in such a way that all the three conditions of the Critical Sections are satisfied

OPERATING SYSTEMS
Synchronization Hardware

- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
    - **Atomic** = non-interruptible
  - Either test memory word and set value
  - Or swap contents of two memory words

OPERATING SYSTEMS
test_and_set Instruction

- Test and Set Lock (TSL) is a synchronization mechanism.
- It uses a test and set instruction to provide the synchronization among the processes executing concurrently.
- It is an instruction that returns the old value of a memory location and sets the memory location value to 1 as a single atomic operation.
- If one process is currently executing a test-and-set, no other process is allowed to begin another test-and-set until the first process test-and-set is finished.

OPERATING SYSTEMS
test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
  {
     boolean rv = *target;
     *target = TRUE;
     return rv:
  }
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

OPERATING SYSTEMS
Solution using test_and_set()

- Shared Boolean variable lock, initialized to FALSE
- Solution:

```
do {
  while (test_and_set(&lock))
    ; /* do nothing */
      /* critical section */
  lock = false;
      /* remainder section */
} while (true);
```

while(Test-and-Set(Lock)); — Entry Section

Critical Section

Lock = 0 — Exit Section

OPERATING SYSTEMS
compare_and_swap Instruction

Definition:

```
int compare _and_swap(int *value, int expected, int new_value) {
  int temp = *value;

  if (*value == expected)
    *value = new_value;
return temp;
}
```

```
do{
while(compare_and_swap(&lock,0,1)!=0);
Critical section
lock=0
Remainder section
}while(true)
```

1. Executed atomically
2. Returns the original value of passed parameter "value"
3. Set the variable "value" the value of the passed parameter "new_value" but only if *value == expected. That is, the swap takes place only under this condition.
4. In the x86 (since 80486) and Itanium architectures this is implemented as compare and exchange (CMPXCHG) instruction

OPERATING SYSTEMS
Solution using compare_and_swap()

- Shared integer "lock" initialized to 0;
- Solution:

```
do {
    while (compare_and_swap(&lock, 0, 1) != 0)
  ; /* do nothing */
  /* critical section */
 lock = 0;
  /* remainder section */
} while (true);
```

Mutual exclusion is satisfied
Do not satisfy bounded waiting requirement

OPERATING SYSTEMS
Bounded-waiting Mutual Exclusion with test_and_set

This test_and_set algorithm satisfies all the critical section requirements
The common data structures are
boolean waiting[n];
boolean lock;

```
do {
  waiting[i] = true;
  key = true;
  while (waiting[i] && key)
    key = test_and_set(&lock);
  waiting[i] = false;

/* critical section */

  j = (i + 1) % n;
  while ((j != i) && !waiting[j])
    j = (j + 1) % n;
  if (j == i)
    lock = false;
  else
    waiting[j] = false;
/* remainder section */
} while (true);
```

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Mutex Locks and Semaphores

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Mutex Locks

- Previous solutions i.e hardware and software solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
  - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
  - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
- This lock therefore called a **spinlock**
- It is problem in real time systems
- Busy waiting wastes CPU cycles

Mutex Locks

Advantages of spinlocks

- no context switch is required when a process must wait on a lock.
- context switch may take considerable time.
- When locks are expected to be held for short times, spinlocks are useful
- These are employed on multiprocessor systems where one thread can "spin" on one processor while another thread performs its critical section on another processor.

Solution to Critical-section Problem Using Locks

```
while (TRUE) {
    acquire lock
      critical section
    release lock
  remainder section
}
```

- **Implementation of acquire and release**
- **acquire() {**
  while (!available)
  ; /* busy wait */
  available = false;
  }
- **release() {**
  available = true;
  }

acquire() and release()

- Solution to a critical section problem using mutex
- **do {**
  acquire lock
  critical section
  release lock
  remainder section
  } while (true);

Scenario 2

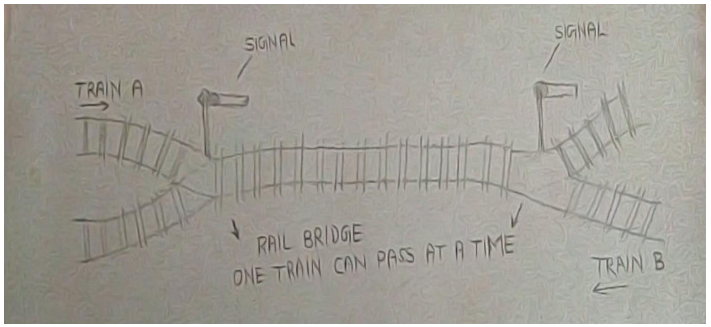


Image src: https://www.linkedin.com/pulse/concurrency-semaphore-real-life-kunal-saxena/

Scenario 1

- Consider a library of an university with 10 rooms
- At a time one room can be used by only one student by informing the front desk for reading.
- Once he completes reading, he has to inform the front desk.
- Person at front desk knows how many rooms are available for use and how many are occupied, how may of them are waiting.
- Once the room is vacant, who will get the chance to occupy the room?

Idea Src:https://en.wikipedia.org/wiki/Semaphore_(programming)

Semaphore

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - **wait()** and **signal()**
    - Originally called **P()** and **V()**
- Definition of the **wait() operation**

```
wait(S) {
   while (S <= 0)
     ; // busy wait
   S--;}
```

- Definition of the **signal() operation**

```
signal(S) {
   S++;
}
```


Semaphore

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can solve various synchronization problems
- Consider P_1 and P_2 that require S_1 to happen before S_2

  Create a semaphore "**synch**" initialized to 0

```
P1:
   S1;
   signal(synch);
P2:
   wait(synch);
   S2;
```

- Can implement a counting semaphore ***S*** as a binary semaphore

Semaphore Implementation

- Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
  - Could now have **busy waiting** in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

OPERATING SYSTEMS
Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- **typedef struct{**

int value;

struct process *list;

} semaphore;

OPERATING SYSTEMS
Semaphore Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
  S->value--;
  if (S->value < 0) {
    add this process to S->list;
    block();
  }
}
signal(semaphore *S) {
  S->value++;
  if (S->value >= 0) {
    remove a process P from S->list;
    wakeup(P);
  } }
```

OPERATING SYSTEMS
Problems with Semaphores

- Incorrect use of semaphore operations:
  - signal (mutex) …. wait (mutex)
  - wait (mutex) … wait (mutex)
  - Omitting of wait (mutex) or signal (mutex) (or both)
- Deadlock and starvation are possible.
- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

```
P0                          P1
     wait(S);                    wait(Q);
     wait(Q);                    wait(S);
...                         ...
     signal(S);                  signal(Q);
     signal(Q);                  signal(S);
```

OPERATING SYSTEMS
Problems with Semaphores (Cont.)

- **Starvation** – **indefinite blocking**
  - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**

➢When several tasks are waiting for the same critical resource, the task which is currently holding this critical resource is given the highest priority among all the tasks which are waiting for the same critical resource.

➢Now after the lower priority task having the critical resource is given the highest priority then the intermediate priority tasks can not preempt this task. This helps in avoiding priority inversion.

➢When the task which is given the highest priority among all tasks, finishes the job and releases the critical resource then it gets back to its original priority value (which may be less or equal).

➢It allows the different priority tasks to share the critical resources.

OPERATING SYSTEMS
Priority Inheritance Protocol

- Consider the scenario with three processes **P1, P2**, and **P3**.
  - **P1** has the highest priority, **P2** the next highest, and **P3** the lowest.
- Assume that **P3** is holding semaphore **S** and that **P1** is waiting for **S** to be released
- Assume that **P2** is assigned the CPU and preempts **P3**
  - **P3** is still holding semaphore **S**
  - **P1** is waiting for **S** to be released
- What has happened is that **P2** - a process with a lower priority than **P1** - has indirectly prevented **P3** from gaining access to the resource.
- To prevent this from occurring, a **priority inheritance protocol** is used.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Classic problems of Synchronization

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

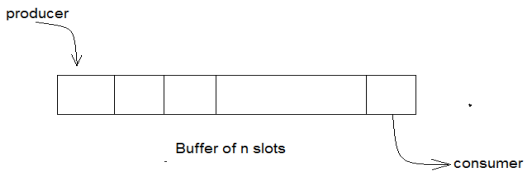
OPERATING SYSTEMS

- **Classical problems of Synchronization**

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n



OPERATING SYSTEMS
Bounded-Buffer Problem (Cont.)

- The structure of the producer process

```
do {
   ...
   /* produce an item in next_produced */
   ...
   wait(empty); //wait until empty > 0 and then decrement 'empty'
   wait(mutex); //acquire lock
   ...
   /* add next produced to the buffer */
   ...
   signal(mutex); //release a lock
   signal(full); //increment full
} while (true);
```

Bounded-Buffer Problem (Cont.)

- The structure of the consumer process

```
do {
  wait(full); // wait until full > 0 and then decrement 'full'
  wait(mutex); // acquire the lock
  ...
  /* remove an item from buffer to next_consumed */
  ...
  signal(mutex); // release the lock
  signal(empty); // increment 'empty'
  ...
  /* consume the item in next consumed */
  ...
} while (true);
```


Readers-Writers Problem

The Problem Statement

There is a shared resource which should be accessed by multiple processes. There are two types of processes in this context. They are **reader** and **writer**. Any number of **readers** can read from the shared resource simultaneously, but only one **writer** can write to the shared resource. When a **writer** is writing data to the resource, no other process can access the resource. A **writer** cannot write to the resource if there are non zero number of readers accessing the resource at that time.

Readers-Writers Problem

- **solution**
- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
  - Data set
  - Semaphore **rw_mutex** initialized to 1(semaphore)
  - Semaphore **mutex** initialized to 1 (mutex)
  - Integer **read_count** initialized to 0

Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
} while (true);
```

Readers-Writers Problem (Cont.)

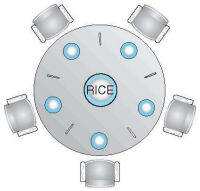
- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
       wait(rw_mutex);
    signal(mutex);
    ...
    /* reading is performed */
    ...
    wait(mutex);
    read count--;
    if (read_count == 0)
       signal(rw_mutex);
    signal(mutex);
} while (true);
```


Readers-Writers Problem Variations

- ***First*** variation – no reader kept waiting unless writer has permission to use shared object
- ***Second*** variation – once writer is ready, it performs the write ASAP
- Both may have starvation leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

Dining-Philosophers Problem



- Philosophers spend their lives alternating thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

- The structure of Philosopher *i*:

```
do {
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );
            // eat
    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );
            // think
} while (TRUE);
```

- What is the problem with this algorithm?

Dining-Philosophers Problem Algorithm (Cont.)

- Deadlock handling
  - Allow at most 4 philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up the chopsticks only if both are available (picking must be done in a critical section).
  - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Synchronization Examples

Suresh Jamadagni
Department of Computer Science

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Synchronization Examples

- Windows
- Linux
- Pthreads
- Solaris

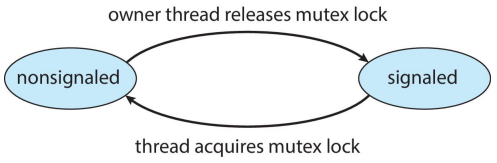
Suresh Jamadagni
Department of Computer Science

Windows Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
  - For reasons of efficiency, kernel ensures that a thread will never be preempted while holding a spinlock.
- Also provides **dispatcher objects** outside the kernel, to synchronize mutex locks, semaphores, events, and timers
  - **Events**
    - An event acts much like a condition variable (i.e notify a waiting thread when a desired condition occurs)
  - Timers notify one or more thread when time expired

Windows Synchronization - Mutex dispatcher object

- Dispatcher objects may be in either a **signaled-state (**object available and a thread will not block) or a **non-signaled state** (object not available, thread will block)
- A Relationship exists between the state of a dispatcher object and the state of a thread.
  - State of a thread changes from ready to waiting and vice-versa



Linux Synchronization

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - Semaphores
  - atomic integers
  - spinlocks
  - reader-writer versions of both
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption

OPERATING SYSTEMS
Linux Synchronization (Cont.)

- Atomic variables
  atomic_t is the data type for atomic integer
- Consider the variables
  atomic_t counter;
  int value;

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

OPERATING SYSTEMS
Pthreads (POSIX) Synchronization

- Pthreads API is OS-independent, widely used on UNIX, Linux, and macOS
- It provides:
  - mutex locks
  - semaphores
  - condition variable
- Non-portable extensions include:
  - read-write locks
  - spinlocks

OPERATING SYSTEMS
POSIX Mutex Locks

- Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

- Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

OPERATING SYSTEMS
POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores (have actual names in the file system) can be shared by multiple unrelated processes
- Unnamed semaphores can be used only by threads belonging to the same process.

OPERATING SYSTEMS
POSIX Named Semaphores

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

OPERATING SYSTEMS
POSIX Unnamed Semaphores

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

OPERATING SYSTEMS
POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor (A high-level abstraction that provides a convenient and effective mechanism for process synchronization) , POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

OPERATING SYSTEMS
POSIX Condition Variables

- Thread waiting for the condition **a == b** to become true:

```
pthread_mutex_lock(&mutex);
while (a != b)
     pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing
- Uses **adaptive mutexes** for efficiency when protecting data from short code segments (< a few 100 instructions)
  - Starts as a standard semaphore spin-lock
  - If lock held, and by a thread running on another CPU, spins
  - If lock held by non-run-state thread (i.e. the thread holding the lock is not currently in run state), block and sleep waiting for signal of lock being released

- Uses **condition variables**
- Uses **readers-writers** locks when longer sections of code need access to data
- Uses **turnstiles** to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock
  - Turnstiles are per-lock-holding-thread, not per-object
- Priority-inheritance per-turnstile gives the running thread the highest of the priorities of the threads in its turnstile

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Deadlocks

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Principles of Deadlocks, Deadlock Characterization

Suresh Jamadagni
Department of Computer Science

- Multiprogramming environment: several processes compete to limited number of resources
- A Process is holding a resource(R1) and is waiting for the resources(R2).
- The resource R2 is held by another process..
- Waiting state of processes will not change, as the requested resource is held by the waiting process.
- This situation is called deadlock

- System consists of finite number of resources
- Resource types $R_1, R_2, \ldots, R_m$
  - *Physical resources: CPU cycles, memory space, I/O devices, printer, tape drives.*
  - *Logical resources: semaphores, mutex locks, files.*
- Each resource type R_i has W_i instances.
- Ex: if the system has 2 CPU's then CPU has 2 instances
- Each process utilizes a resource as follows:
  - **Request :** Process makes request to the resource. Eg. system call like request(), open(), wait(),allocate() etc
  - **Use:** operates on these resources
  - **Release:** process releases the resources. Eg. Using a system call like release(), close(), signal(), free etc.
- Request and release of semaphore, acquire and release of lock on mutex

OPERATING SYSTEMS
Deadlock with physical resources

Example 1

- Consider a system with 3 CD RW drives.
- Suppose 3 processes(p0,p1,p2) are holding one drive each.
- What happens,
  - If a process p0 makes a request for one more drive

Example 2

- Consider a system with one printer one DVD drive.
- Process Pi is holding printer and process Pj is holding DVD drive.
- What happens if
  - Process Pi request DVD and Pj requests printer

Does dead lock occur in example 1 and 2?

OPERATING SYSTEMS
Deadlock with Semaphores

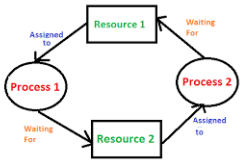
- Data:
  - A semaphore **S1** initialized to 1
  - A semaphore **S2** initialized to 1
- Two processes P1 and P2
- **P1:**

  wait(s1)

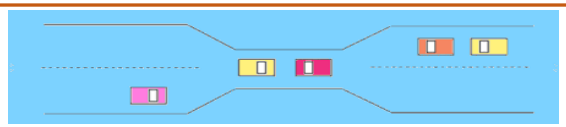
  wait(s2)
- **P2:**

  wait(s2)

  wait(s1)



OPERATING SYSTEMS
Bridge crossing Example



- Traffic only in one direction
- Each section of a bridge can be viewed as a resource
- If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback)
- Several cars may have to be backed up if a deadlock occurs
- Starvation is possible.
- Note – Most OSes do not prevent or deal with deadlocks

OPERATING SYSTEMS
Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource (sharable resources like Read-only files do not require mutually exclusive access and thus cannot be involved in a deadlock.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

OPERATING SYSTEMS
Resource-Allocation Graph

- Deadlocks are described precisely with directed graphs called system resource-allocation graph

A graph consists of set of vertices *V* and a set of edges *E*.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

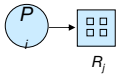
OPERATING SYSTEMS
Resource-Allocation Graph (Cont.)

A set of vertices *V* and a set of edges *E*.

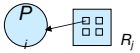
- Process

- Resource Type with 4 instances

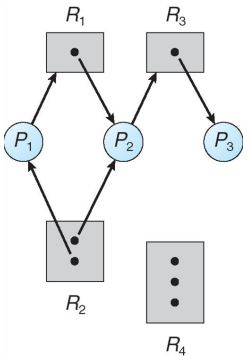
- P_i requests instance of R_j



- P_i is holding an instance of R_j

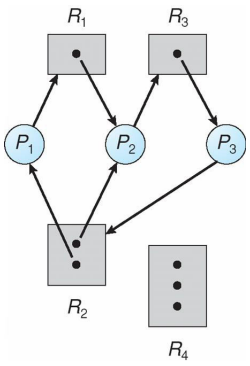


OPERATING SYSTEMS
Example of a Resource-Allocation Graph



Set of process: P1,P2,P3
Set of Resources: R1,R2,R3

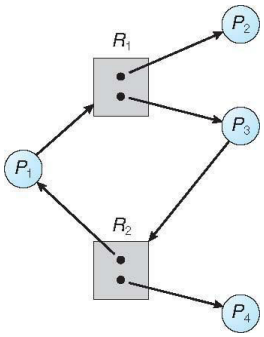
OPERATING SYSTEMS
Resource-Allocation Graph With A Deadlock



P1 R1 P2 R3 P3 R2 P1

P2 R3 P3 R2 P2

OPERATING SYSTEMS
Resource-Allocation Graph With A Cycle but No Deadlock



P1 R1 P3 R2 P1

OPERATING SYSTEMS
Basic Facts

- If graph contains no cycles ⇒ no deadlock
- If graph contains a cycle ⇒
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, the system may or may not be in a deadlocked state

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B
Deadlocks

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Deadlock Prevention, Deadlock Example

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Methods for Handling Deadlocks

- Generally speaking there are three ways of handling deadlocks:
  - Deadlock prevention or avoidance - Do not allow the system to get into a deadlocked state.
  - Allow the system to enter into deadlocked state, detect it, and recover.
  - Ignore the problem all together and pretend that deadlocks never occur in the system.

OPERATING SYSTEMS
Deadlock Prevention

4 Necessary conditions for deadlock to occur

- **Mutual Exclusion**
  - At least one resource must be non sharable
    - Ex. printers and tape drives, mutex locks
  - Sharable resources do not require mutual exclusion
    - Ex . Read-only files
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
  - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
  - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption** –
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
  - Preempted resources are added to the list of resources for which the process is waiting
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** –
  - Each resource will be assigned with a numerical number.
  - A process can request the resources increasing/decreasing order of numbering.

Circular Wait

- Ex., if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.
- Resources={R1,R2,…..Rm}
- Resources are assigned unique number
- Each process request resource in increasing order enumeration
- we define a one-to-one function F: R→N, where N is the set of natural numbers
- Protocol 1: Process makes request for Ri and then for Rj. Resources Rj request is allowed if and only if F(Rj)>F(Ri).
- Protocol 2: Process requesting an instance of resource type Rj, must have released any resource Ri , such that F(Ri)>=F(Rj).
- If these protocols are used then circular wait will not exist.

Circular Wait

If these two protocols are used, then the circular-wait condition cannot hold.

Proof by contradiction:

- We can demonstrate this fact that by assuming that circular wait condition cannot hold
- Consider set of processes P={P0,P1,…….Pn}
- Let us consider process P0 is waiting for resource held by P1, P1 waiting for P2,……..Pn-1 is waiting resource held by Pn, Pn is waiting for resources held by P0.
- Generalizing this, Process Pi is waiting for resources Ri, Ri is held by Pi+1 and it is making request for Ri+1
- We must have F(Ri)<F(Ri+1)
- But this condition means that $F(R0) < F(R1) < \ldots < F(Rn) < F(R0)$.
- By transitivity, $F(R0) < F(R0)$, which is impossible
- Therefore no circular wait.

Circular Wait

Example

- F(tape drive)=1
- F(disk drive)=5
- F(printer)=12
- F(tape drive)<F(printer)

Circular Wait

- Ex., if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.
- Invalidating the circular wait condition is most common.
- Assign each resource (i.e., mutex locks) a unique number.
- Resources must be acquired in order.
- If:
  first_mutex = 1
  second_mutex = 5

  code for **thread_two** could not be written as follows:

Deadlock Example

```
/* thread one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /** * Do some work */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);
   pthread_exit(0);
}
/* thread two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /** * Do some work */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);
   pthread_exit(0);
}
```


Deadlock Example with Lock Ordering

```
void transaction(Account from, Account to, double amount) {
  mutex lock1, lock2;
  lock1 = get_lock(from);
  lock2 = get_lock(to);
  acquire(lock1);
    acquire(lock2);
      withdraw(from, amount);
      deposit(to, amount);
    release(lock2);
  release(lock1);
}
```

Transactions 1 and 2 execute concurrently. Transaction 1 transfers $25 from account A to account B, and Transaction 2 transfers $50 from account B to account A

OPERATING SYSTEMS

Deadlock Detection, Algorithm

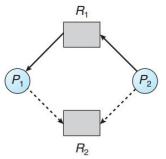
Chandravva Hebbi
Department of Computer Science

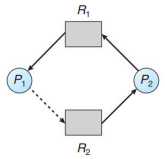
OPERATING SYSTEMS
Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

OPERATING SYSTEMS
Single Instance of Each Resource Type

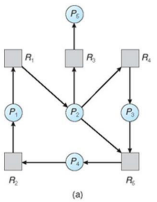
- In a resource allocation graph, a claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge is represented in the graph by a dashed line.
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph



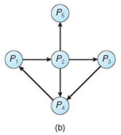


OPERATING SYSTEMS
Resource-Allocation Graph and Wait-for Graph

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph.
- We obtain wait-for graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.
- An edge from P_i to P_j implies that process P_i is waiting for process P_j to release a resource that P_i needs.
- An edge $P_i \rightarrow P_j$ exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q
- A **deadlock exists** in the system if and only if the **wait-for graph contains a cycle**.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph



Resource-Allocation Graph



Corresponding wait-for graph

OPERATING SYSTEMS
Several Instances of a Resource Type

- **Available***:* A vector of length ***m*** indicates the number of available resources of each type. If Available[j] = k, then k instances of resource type R_j are available
- **Max**: An n × m matrix defines the maximum demand of each process. If Max[i][j] = k, then process P_i may request at most k instances of resource type R_j .
- **Allocation***:* An ***n*** **x** ***m*** matrix defines the number of resources of each type currently allocated to each process. If Allocation[i][j] = k, then process P_i is currently allocated k instances of resource type R_j
- **Request***:* An ***n*** **x** ***m*** matrix indicates the current request of each process. If ***Request [i][j] = k***, then process P_i is requesting ***k*** more instances of resource type R_j.

OPERATING SYSTEMS
Detection Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:
   (a) ***Work = Available***
   (b) For ***i* = 1,2, …, *n***, if ***Allocation*$_i$ ≠ 0**, then ***Finish*[i] = *false***; otherwise, ***Finish*[i] = *true***

2. Find an index ***i*** such that both:
   (a) ***Finish*[*i*] == *false***
   (b) ***Request*$_i$ ≤ *Work***

   If no such ***i*** exists, go to step 4

OPERATING SYSTEMS
Detection Algorithm (Cont.)

3. ***Work = Work + Allocation*$_i$**
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish[i] == false***, for some ***i***, 1 ≤ ***i*** ≤ ***n***, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then ***P*$_i$** is deadlocked. If **Finish[i] == true** for all i, then the system is in a safe state

Algorithm requires an order of O(m x n^2) operations to detect whether the system is in deadlocked state

OPERATING SYSTEMS
Example of Detection Algorithm

- Five processes ***P*$_0$** through ***P*$_4$**; three resource types A (7 instances), *B* (2 instances), and *C* (6 instances)
- Snapshot at time ***T*$_0$**:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence <***P*$_0$, *P*$_2$, *P*$_3$, *P*$_1$, *P*$_4$***> will result in ***Finish[i] = true*** for all ***i***

OPERATING SYSTEMS
Example of Detection Algorithm (Cont.)

- ***P*$_2$** requests an additional instance of type ***C***

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

- State of system?
  - Can reclaim resources held by process ***P*$_0$**, but insufficient resources to fulfill other processes; requests
  - Deadlock exists, consisting of processes ***P*$_1$**, ***P*$_2$**, ***P*$_3$**, and ***P*$_4$**

- When, and how often, to invoke depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Deadlocks

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Deadlock Avoidance

Suresh Jamadagni
Department of Computer Science

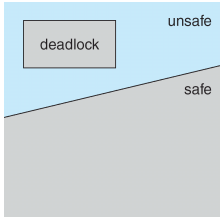
- Deadlock-prevention algorithms, prevent deadlocks by limiting how requests can be made.
- The limits ensure that at least one of the necessary conditions for deadlock cannot occur.
- Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.
- **Deadlock avoidance** requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge of complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process

- The simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.
- A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource allocation ***state*** is defined by the number of available and allocated resources and the maximum demands of the processes.

- A state is ***safe*** if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a **safe sequence**.
- A sequence of processes <*P*1, *P*2, ..., *Pn*> is a safe sequence for the current allocation state if, for each *Pi* , the resource requests that *Pi* can still make can be satisfied by the currently available resources plus the resources held by all *Pj*, with $j < i$.
- In this situation, if the resources that *Pi* needs are not immediately available, then *Pi* can wait until all *Pj* have finished.
- When they have finished, *Pi* can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate.
- When *Pi* terminates, *Pi*+1 can obtain its needed resources, and so on. If no such sequence exists, then the system state is said to be ***unsafe***

OPERATING SYSTEMS
Safe, Unsafe and Deadlock States

- A safe state is not a deadlocked state.
- Conversely, a deadlock state is an unsafe state.
- Not all unsafe states are deadlocks.
- An unsafe state may lead to a deadlock.
- As long as the state is safe, the OS can avoid unsafe states.
- In an unsafe state, the OS cannot prevent processes from requesting resources in such a way that a deadlock occurs.
- The behavior of processes controls unsafe states.
- If a system is in safe state → no deadlocks
- If a system is in unsafe state → possibility of deadlock
- Goal for Avoidance → ensure that a system will never enter an unsafe state.



OPERATING SYSTEMS
Example - Safe, Unsafe and Deadlock States

- Consider a system with twelve magnetic tape drives and three processes: *P*0, *P*1, and *P*2.
- Process *P*0 requires ten tape drives, process *P*1 may need as many as four tape drives, and process *P*2 may need up to nine tape drives.
- Suppose that, at time *t*0, process *P*0 is holding five tape drives, process *P*1 is holding two tape drives, and process *P*2 is holding two tape drives. (Thus, there are three free tape drives.)

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

- At time *t*0, the system is in a safe state. The sequence <*P*1, *P*0, *P*2> satisfies the safety condition. Process *P*1 can immediately be allocated all its tape drives and then return them (the system will then have five available tape drives); then process *P*0 can get all its tape drives and return them (the system will then have ten available tape drives); and finally process *P*2 can get all its tape drives and return them
- At time *t*1, process *P*2 requests and is allocated one more tape drive. The system is no longer in a safe state.
- Since process *P*0 is allocated five tape drives but has a maximum of ten, it may request five more tape drives. If it does so, it will have to wait, because they are unavailable. Similarly, process *P*2 may request six additional tape drives and have to wait, resulting in a deadlock

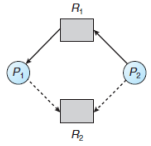
OPERATING SYSTEMS
Deadlock avoidance algorithms

- Avoidance algorithms ensure that the system will never deadlock.
- The idea is simply to ensure that the system will always remain in a safe state.
- Initially, the system is in a safe state. Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait.
- The request is granted only if the allocation leaves the system in a safe state.
- In this scheme, if a process requests a resource that is currently available, it may still have to wait.
- Thus, resource utilization may not be optimal

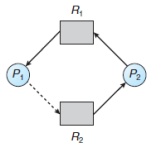
OPERATING SYSTEMS
Resource Allocation Graph algorithm

- In addition to the request and assignment edges in a resource allocation graph, we introduce a new type of edge, called a **claim edge**
- A claim edge *Pi* → *Rj* indicates that process *Pi* may request resource *Rj* at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a dashed line.
- When process *Pi* requests resource *Rj* , the claim edge *Pi* → *Rj* is converted to a request edge. Similarly, when a resource *Rj* is released by *Pi* , the assignment edge *Rj* → *Pi* is reconverted to a claim edge *Pi* → *Rj*
- The resources must be claimed a priori in the system. That is, before process *Pi* starts executing, all its claim edges must already appear in the resource-allocation graph.
- Now suppose that process *Pi* requests resource *Rj*. The request can be granted only if converting the request edge *Pi* → *Rj* to an assignment edge *Rj* → *Pi* does not result in the formation of a cycle in the resource-allocation graph.

OPERATING SYSTEMS
Resource Allocation Graph algorithm example



Resource-allocation graph for deadlock avoidance.



An unsafe state in a resource-allocation graph.

- Consider the above resource-allocation graph.
- Suppose that *P*2 requests *R*2. Although *R*2 is currently free, we cannot allocate it to *P*2, since this action will create a cycle in the graph below.
- A cycle indicates that the system is in an unsafe state.
- If *P*1 requests *R*2, then a deadlock will occur.

OPERATING SYSTEMS
Banker's algorithm

- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.
- Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system.
- We need the following data structures, where *n* is the number of processes in the system and *m* is the number of resource types

OPERATING SYSTEMS
Banker's algorithm data structures

- **Available –** A vector of length *m* indicates the number of available resources of each type. If ***Available***[*j*] equals *k,* then *k* instances of resource type *Rj* are available.
- **Max**. - An *n* × *m* matrix defines the maximum demand of each process. If ***Max***[*i*][*j*] equals *k,* then process *Pi* may request at most *k* instances of resource type *Rj* .
- **Allocation** - An *n* × *m* matrix defines the number of resources of each type currently allocated to each process. If ***Allocation***[*i*][*j*] equals *k,* then process *Pi* is currently allocated *k* instances of resource type *Rj* .
- **Need** - An *n* × *m* matrix indicates the remaining resource need of each process. If ***Need***[*i*][*j*] equals *k,* then process *Pi* may need *k* more instances of resource type *Rj* to complete its task.
- ***Need***[*i*][*j*] = ***Max***[*i*][*j*] − ***Allocation***[*i*][*j*].
- Treat each row in the matrices ***Allocation*** and ***Need*** as vectorsThe vector ***Allocation****i* specifies the resources currently allocated to process *Pi* ; the vector ***Need****i* specifies the additional resources that process *Pi* may still request to complete its task
- These data structures vary over time in both size and value.

OPERATING SYSTEMS
Banker's algorithm for determining whether or not a system is in a safe state

1. Let***Work*** and ***Finish*** be vectors of length *m* and *n,* respectively.
   Initialize ***Work*** = ***Available*** and ***Finish***[*i*] = ***false*** for *i* = 0, 1, ..., *n* − 1.
2. Find an index *i* such that both
   a. ***Finish***[*i*] == ***false***
   b. ***Need****i* ≤***Work***
   If no such *i* exists, go to step 4
3. ***Work*** =***Work*** + ***Allocation****i*
   Finish[*i*] = ***true***
   Go to step 2.
4. If ***Finish***[*i*] == ***true*** for all *i,* then the system is in a safe state.

- This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.

Banker's algorithm for determining whether requests can be safely granted

Let ***Request****i* be the request vector for process *Pi* . If ***Request****i* [*j*] == *k*, then process *Pi* wants *k* instances of resource type *Rj* .

When a request for resources is made by process *Pi* , the following actions are taken:

1. If ***Request****i* ≤ ***Need****i* , go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If ***Request****i* ≤ ***Available,*** go to step 3. Otherwise, *Pi* must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process *Pi* by modifying the state as follows:

   Available = ***Available***–***Request****i* ;

   Allocation*i* = ***Allocation****i* + ***Request****i* ;

   Need*i* = ***Need****i* –***Request****i* ;

If the resulting resource-allocation state is safe, the transaction is completed, and process *Pi* is allocated its resources.

If the new state is unsafe, then *Pi* must wait for ***Request****i* **,** and the old resource-allocation state is restored.

Banker's algorithm example

- Consider a system with five processes *P*0 through *P*4 and three resource types *A, B,* and *C.*
- Resource type *A* has ten instances, resource type *B* has five instances, and resource type *C* has
- seven instances.
- Suppose that, at time *T*0, the following snapshot of the system has been taken:

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Sequence <*P*1, *P*3, *P*4, *P*0, *P*2> satisfies the safety requirement. Hence, we can immediately grant the request of process *P*1.
- A request for (3,3,0) by *P*4 cannot be granted, since the resources are not available.
- A request for (0,2,0) by *P*0 cannot be granted, even though the resources are available, since the resulting state is unsafe.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Signals

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Signals

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Signals

- A signal is used in UNIX systems to notify a process that a particular event has occurred.
- A signal may be received either synchronously or asynchronously depending on the source of and the reason for the event being signaled
- All signals, whether synchronous or asynchronous, follow the same pattern:
  1. A signal is generated by the occurrence of a particular event.
  2. The signal is delivered to a process.
  3. Once delivered, the signal must be handled.

OPERATING SYSTEMS

Signals – synchronous and asynchronous

- Examples of synchronous signal include illegal memory access and division by 0.
- If a running program performs either of these actions, a signal is generated.
- Synchronous signals are delivered to the same process that performed the operation that caused the signal (that is the reason they are considered synchronous).
- When a signal is generated by an event external to a running process, that process receives the signal asynchronously.
- Examples of such signals include terminating a process with specific keystrokes (such as <control><C>) and having a timer expire.
- Typically, an asynchronous signal is sent to another process.

OPERATING SYSTEMS
Signal handlers

- A signal may be ***handled*** by one of two possible handlers:
  1. A default signal handler
  2. A user-defined signal handler
- Every signal has a **default signal handler** that the kernel runs when handling that signal.
- This default action can be overridden by a **user-defined signal handler** that is called to handle the signal.
- Signals are handled in different ways. Some signals (such as changing the size of a window) are simply ignored; others (such as an illegal memory access) are handled by terminating the program.

OPERATING SYSTEMS
Signal handling

- Handling signals in single-threaded programs is straightforward: signals are always delivered to a process.
- Delivering signals is more complicated in multithreaded programs, where a process may have several threads.
- In general, the following options exist:
  1. Deliver the signal to the thread to which the signal applies.
  2. Deliver the signal to every thread in the process.
  3. Deliver the signal to certain threads in the process.
  4. Assign a specific thread to receive all signals for the process.

OPERATING SYSTEMS
Signal handling

- The method for delivering a signal depends on the type of signal generated.
- For example, synchronous signals need to be delivered to the thread causing the signal and not to other threads in the process.
- However, the situation with asynchronous signals is not as clear. Some asynchronous signals—such as a signal that terminates a process (<control><C>) should be sent to all threads.
- The standard UNIX function for delivering a signal is

  kill(pid t pid, int signal)
- This function specifies the process (pid) to which a particular signal (signal) is to be delivered

OPERATING SYSTEMS
Signal handling

- Most multithreaded versions of UNIX allow a thread to specify which signals it will accept and which it will block.
- An asynchronous signal may be delivered only to those threads that are not blocking it.
- Because signals need to be handled only once, a signal is typically delivered only to the first thread found that is not blocking it.
- POSIX Pthreads provides the following function, which allows a signal to be delivered to a specified thread (tid):

  pthread kill(pthread t tid, int signal)

Signals in windows

- Windows does not explicitly provide support for signals, it allows to emulate them using **asynchronous procedure calls (APCs)**.
- The APC facility enables a user thread to specify a function that is to be called when the user thread receives notification of a particular event.
- An APC is roughly equivalent to an asynchronous signal in UNIX
- The APC facility is more straightforward, since an APC is delivered to a particular thread rather than a process

Signals in Linux

- A signal is a kind of software interrupt, used to announce asynchronous events to a process
- **SIGINT** is a signal generated when a user presses Control-C. This will terminate the program from the terminal.
- **SIGALRM** is generated when the timer set by the alarm function goes off.
- **SIGABRT** is generated when a process executes the abort function.
- **SIGSTOP** tells LINUX to pause a process to be resumed later.
- **SIGCONT** tells LINUX to resume the processed paused earlier.
- **SIGSEGV** is sent to a process when it has a segmentation fault.
- **SIGKILL** is sent to a process to cause it to terminate at once.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Main Memory:

Hardware and control structures, OS support, Address translation

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Background

- What is a memory?
  - Memory consists of a large array of bytes, each with its own address.
- Execution of an instruction,
  - Fetch an Instruction from memory, Decode the instruction, operands are fetched(from memory or registers)
  - After the instruction is executed, results are stored back.
- The memory unit(MU) sees the stream of addresses.
- Memory unit does not know how these addresses are generated.
- We will learn how the addresses are generated by the running program.

OPERATING SYSTEMS
Basic Hardware

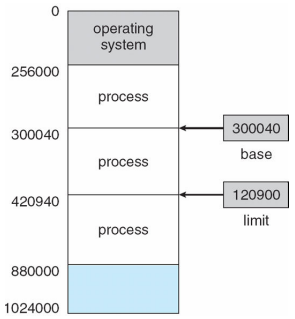
- Program must be brought (from disk) into memory and placed within a ready queue it to be run
- Main memory and registers are only storage CPU can access directly
- The data required for CPU must be made available in the registers.
  - Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
  - Speeds up memory access without any OS control
- Protection of memory required to ensure correct operation

OPERATING SYSTEMS
Basic Hardware

- Protection of OS from its access by user processes is needed
- On multiuser systems user processes must be protected from each other.
- This protection must be provided by the hardware because the operating system doesn't usually intervene between the CPU and its memory accesses.
- Several hardware protection methods will be discussed.
- Protection by using two registers, usually a base and a limit.

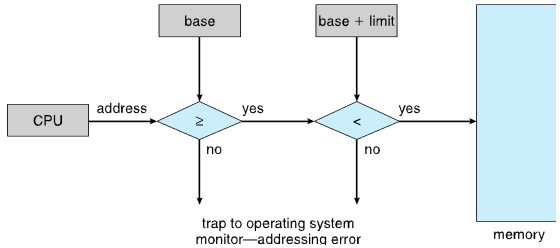
OPERATING SYSTEMS
Basic Hardware

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



OPERATING SYSTEMS
Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



- prevents a user program from modifying the code or data structures of either the operating system or other users.

OPERATING SYSTEMS
Basic Hardware

- The base and limit registers can be loaded only by the operating system.
  - Using special privileged instruction.
  - privileged instructions can be executed only in kernel mode.
  - OS runs in the kernel mode. Only OS can change the register values.
  - This prevents other user programs to modify the register values.

OPERATING SYSTEMS
Address Binding

- Programs on disk as binary executable and are brought to main memory for execution
- Processes may be moved between the disk and memory during execution.
- What are the steps involved in the program execution?
- Most systems allow a user process to reside in any part of the physical memory.
- The address space of the computer may start at 00000 but the first address of the user process need not be 00000
- Addresses in the source program are generally symbolic (Ex. variable count).

OPERATING SYSTEMS
Address Binding

- A compiler typically binds these symbolic addresses to relocatable addresses
  - “14 bytes from the beginning of this module”
- The linkage editor or loader in turn binds the relocatable addresses to absolute addresses
  - Ex. 74014
- Each binding is a mapping from one address space to another.

OPERATING SYSTEMS
Memory-Management Unit (Cont.)

Base-register scheme for address mapping

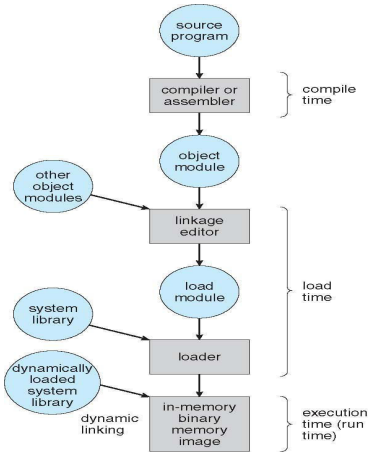
- The base register also referred to as a relocation register.
- The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory .
- Example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000;
- An access to location 346 is mapped to location 14346.
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses

OPERATING SYSTEMS
Memory-Management Unit (Cont.)

The binding of instructions and data to memory addresses

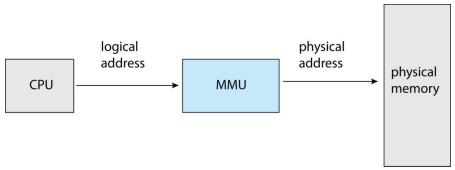
- **Compile time**. If you know at compile time where the process will reside in memory, then **absolute code** can be generated.
  - If the start address changes, it is necessary to recompile once again
  - The MS-DOS .COM-format programs are bound at compile time.
- **Load time**. If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code**.
  - final binding is delayed until load time.
  - If the starting address changes, we need only reload the user code to incorporate this changed value.
- **Execution time**. If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.
  - Special hardware must be available for this scheme to work

OPERATING SYSTEMS
Multistep Processing of a User Program



OPERATING SYSTEMS
Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address



- Many methods possible to accomplish this mapping, will be discussed in the next few lectures.

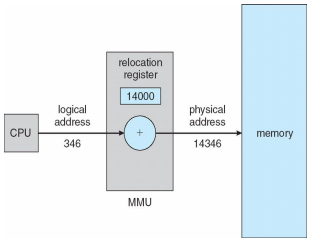
OPERATING SYSTEMS
Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

OPERATING SYSTEMS

Dynamic relocation using a relocation register

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading



OPERATING SYSTEMS

Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- **Dynamic linking** –linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
  - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- Other programs linked before the new library was installed will continue using the older library
- This System also known as **shared libraries**
- Consider applicability to patching system libraries
  - Versioning may be needed

OPERATING SYSTEMS

Static and Dynamic Linking

- A program whose necessary library functions are embedded directly in the program's executable binary file is ***statically*** linked to its libraries
- The main disadvantage of static linkage is that every program generated must contain copies of exactly the same common system library functions
- *Dynamic* linking is more efficient in terms of both physical memory and disk-space usage because it loads the system libraries into memory only once

Demo

- $cc fork.c
- size a.out
- $cc -static fork.c
- size a.out

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Swapping, Memory Allocation , Fragmentation

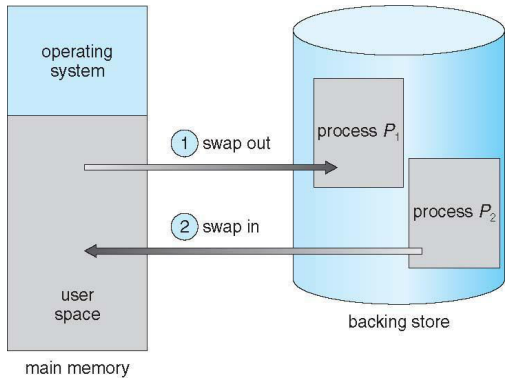
Suresh Jamadagni
Department of Computer Science

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

OPERATING SYSTEMS
Swapping (Cont.)

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold

OPERATING SYSTEMS
Schematic View of Swapping



OPERATING SYSTEMS
Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- Let us consider a user process with size 100MB
- hard disk transfer rate of 50MB/sec
  - Swap out time=100MB/50MB per sec= 2 sec
  - Swap in in time is same as swap out time
  - Total context switch swapping component time = 4 seconds =swap in time + swap out time
- What will be the swap time for process with size 3GB? – 60 secs
- Is it right to swap complete process?
- Can be reduced if size of memory to be swapped is reduced – by knowing how much memory really being used
  - System calls to inform OS of memory use via request_memory() and release_memory()

OPERATING SYSTEMS
Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to process memory space
    - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
    - Swap only when free memory extremely low

OPERATING SYSTEMS
Swapping on Mobile Systems

- Not typically supported
  - Flash memory is used rather than hard disks
    - Storage is a constraint for swapping
    - Limited number of writes are tolerated by flash memory
    - Poor throughput between flash memory and CPU on mobile platform
- Instead of swapping, other methods to free memory if memory is low
  - ***iOS*** asks apps to voluntarily relinquish allocated memory
  - Read-only data are removed system and are loaded if needed
  - Data that have been modified (such as the stack) are never removed.
  - Applications that fail to free up sufficient memory may be terminated by the operating system.

OPERATING SYSTEMS
Swapping on Mobile Systems

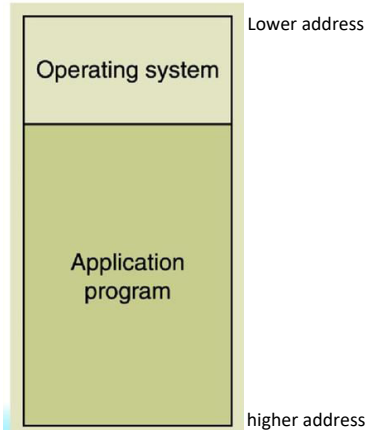
Android OS

- Does not support swapping
- ***Android*** terminates apps if low free memory, but first writes application state to flash for fast restart
- Both OSes support paging which will be discussed later

OPERATING SYSTEMS
Contiguous Allocation

- Main memory must accommodate both OS and user processes
- Memory needs to be allocated efficiently
- Contiguous allocation is one early method
- Main memory usually into two **partitions**:
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single section of memory that is contiguous to the section containing the next process.

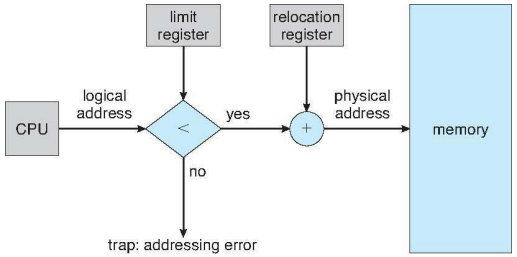
OPERATING SYSTEMS
Contiguous Allocation



OPERATING SYSTEMS
Contiguous Allocation (Cont.)

- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
  - Base register contains value of smallest **physical** address
  - Limit register contains range of **logical** addresses – each logical address must be less than the limit register
  - MMU maps logical address *dynamically*
  - Can then allow actions such as kernel code being **transient** (it comes and goes as needed) and kernel/OS changing size during program execution.

OPERATING SYSTEMS
Hardware Support for Relocation and Limit Registers



When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.

OPERATING SYSTEMS
Memory Protection

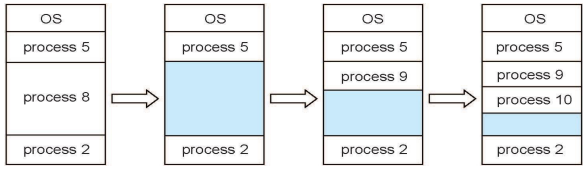
- Every address generated by a CPU is checked against these registers, so possible to protect both the operating system and the other users' programs and data from being modified by this running process .
- The relocation-register scheme provides an effective way to allow the operating system's size to change dynamically.
  - This flexibility is desirable in many situations. For example, the operating system contains **code** and **buffer space** for **device drivers**. If a device driver is not currently in use, it makes little sense to keep it in memory; instead, it can be loaded into memory only when it is needed. Likewise, when the device driver is no longer needed, it can be removed and its memory allocated for other needs.

OPERATING SYSTEMS
Multiple-partition allocation

- Multiple-partition allocation
  - Divide memory into several **fixed-size partitions**.
    - Each partition may contain exactly one process
    - Degree of multiprogramming is bounded by number of partitions.
    - when a partition is free, a process is selected from the input queue and is loaded into the free partition
    - used by the IBM OS/360 operating system,but is no longer in use
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
    - This scheme keeps a table indicating which parts of memory are available/occupied
  - **Hole** – block of available memory; holes of various size are scattered throughout memory

OPERATING SYSTEMS
Multiple-partition allocation (Cont.)

- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Operating system maintains information about:
  a) allocated partitions b) free partitions (hole)
- Considering the size of process as,
- Process 5=100MB,Processes 8=400MB, Process 2=200MB,Process 9=200MB, process 10=100MB, can the process 11=200MB be accommodated after process 5 terminates?



OPERATING SYSTEMS
Partitions

- **Advantages**

1. No Internal Fragmentation
2. No restriction on Degree of Multiprogramming
3. No Limitation on the size of the process

- **Disadvantages**
  - Causes External Fragmentation
  - Difficult to implement

OPERATING SYSTEMS
Dynamic Storage-Allocation Problem and Strategies

How to satisfy a request of size ***n*** from a list of free holes?

- **First-fit**: Allocate the ***first*** hole that is big enough (generally faster)
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization

OPERATING SYSTEMS
Allocations strategies: Examples

Example: Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)? Rank the algorithms in terms of how efficiently they use memory.

Soln:

Partitions	First-Fit
M1=300 KB	P1
185KB	
M2=600 KB	P2
100KB	
M3=350 KB	P4
150KB	
M4=200 KB	
M5=750 KB	P3
392KB	P5
17KB	
M6=125 KB	

Partitions	Best Fit
M1=300 KB	
M2=600 KB	P2
100KB	
M3=350 KB	
M4=200 KB	P4
M5=750 KB	P3
392KB	P5
17KB	
M6=125 KB	P1
10KB	

Partitions	Worst-Fit
M1=300 KB	
M2=600 KB	P3
242KB	
M3=350 KB	P4
150KB	
M4=200 KB	
M5=750 KB	P1
635KB	P2
135KB	
M6=125 KB	

P5 must wait

Processes and their sizes
P1=115 KB P2=500 KB
P3=358 KB P4=200 KB
P5=375 KB

Fragmentation

- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
  - First fit statistical analysis reveals that given *N* allocated blocks (66% for example), another 0.5 *N* blocks (33% for example) lost to fragmentation
  - 1/3 of memory may be unusable -> **50-percent rule**
    - Unusable memory = (0.5N)/(N+0.5N)= 1/3

Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
  - I/O problem
    - Latch job in memory while it is involved in I/O
    - Do I/O only into OS buffers

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Segmentation

Suresh Jamadagni
Department of Computer Science

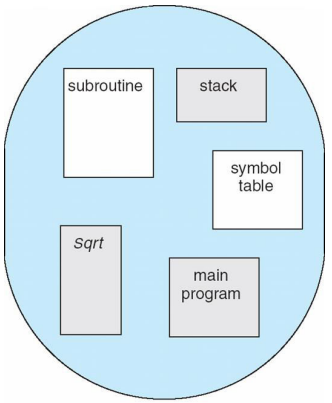
OPERATING SYSTEMS

Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a logical unit such as:
    main program
    procedure
    function
    method
    object
    local variables, global variables
    common block
    stack
    symbol table
    arrays

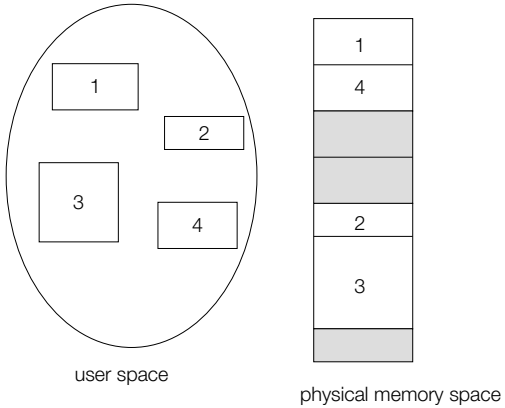
OPERATING SYSTEMS

User's View of a Program



- Segmentation is a memory-management scheme that supports this programmer view of memory
- Each segment has a name and a length.
- Addresses specify segment name + offset within the segment

OPERATING SYSTEMS
Logical View of Segmentation



OPERATING SYSTEMS
Segmentation Basics

- Segmentation is a technique for breaking memory up into logical pieces
- Each "piece" is a grouping of related information
  - data segments for each process
  - code segments for each process
  - data segments for the OS
  - etc.
- Segmentation permits the physical address space of a process to be non-contiguous.
- Like paging, use virtual addresses and use disk to make memory look bigger than it really is
- Segmentation can be implemented with or without paging

OPERATING SYSTEMS
Segmentation Architecture

- Logical address consists of a two tuple:

  <segment-number s, offset d>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
  - **Segment base** – contains the starting physical address where the segments reside in memory
  - **Segment limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;

  segment number ***s*** is legal if ***s*** < **STLR**

OPERATING SYSTEMS
Segmentation Architecture (Cont.)

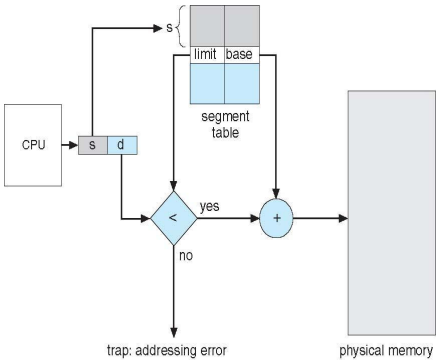
- Protection
  - With each entry in segment table associate:
    - validation bit = 0 ⇒ illegal segment
    - read/write/execute privileges
- Protection bits associated with segments
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem

OPERATING SYSTEMS
Segmentation Hardware

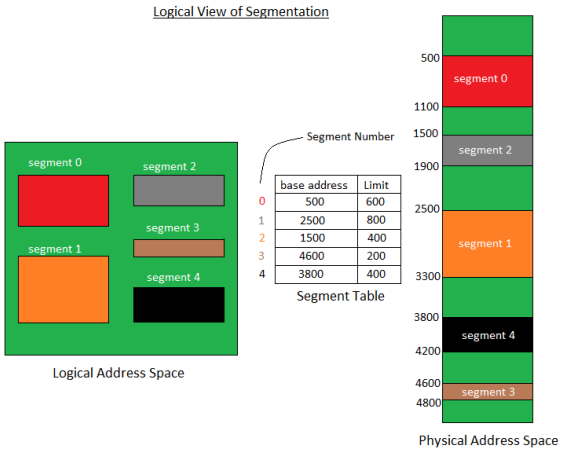
- Sounds very similar to paging
- Big difference – segments can be variable in size
- As with paging, to be effective hardware must be used to translate logical address
- Most systems provide segment registers
- If a reference isn't found in one of the segment registers
  - ➢ trap to operating system
  - ➢ OS does lookup in segment table and loads new segment descriptor into the register
  - ➢ return control to the user and resume

OPERATING SYSTEMS
Segmentation Hardware (Cont.)

- A logical address consists of two parts: a segment number, s, and an offset into that segment, d.
- The segment number is used as an index to the segment table. The offset d of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system (logical addressing attempt beyond end of segment).
- When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte. The segment table is thus essentially an array of base–limit register pairs.



OPERATING SYSTEMS
Segmentation Example



OPERATING SYSTEMS
Segmentation Example (Cont.)

- ❑We have five segments numbered from 0 through 4. The segments are stored in physical memory as shown.
- ❑The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (or base) and the length of that segment (or limit).
- ❑For example, segment 2 is 400 bytes long and begins at location 1500.
  - ❑Thus, a reference to byte 53 of segment 2 is mapped onto location 1500 + 53 = 1553.
- ❑A reference to segment 3, byte 852, is mapped to 4600 (the base of segment 3) + 852 = 5452.
- ❑A reference to byte 722 of segment 0 would result in a trap to the operating system, as this segment is only 600 bytes long.

OPERATING SYSTEMS
Segmentation Comments

- In each segment table entry, we have both the starting address and length of the segment; the segment can thus dynamically grow or shrink as needed.
- But variable length segments introduce external fragmentation and are more difficult to swap in and out.
- It is natural to provide protection and sharing at the segment level since segments are visible to the programmer (pages are not).
- Useful protection bits in segment table entry:
  - read-only/read-write bit
  - Kernel/User bit

OPERATING SYSTEMS
Segmentation Issues

❑ Entire segment is either in memory or on disk

❑ Variable sized segments leads to external fragmentation in memory

❑ Must find a space big enough to place segment into

❑ May need to swap out some segments to bring a new segment in.

OPERATING SYSTEMS
Segmentation: Examples

❑ Consider the following segment table:

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?
a. 0,430
b. 1,10
c. 2,500
d. 3,400
e. 4,112

OPERATING SYSTEMS
Segmentation: Examples

❑ Consider the following segment table:

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?
a. 0,430
b. 1,10
c. 2,500
d. 3,400
e. 4,112

Solution:

a. 0, 430 ⍰ 219 + 430 = 649
b. 1, 10 ⍰ 2300 + 10 = 2310
c. 2, 500 Illegal address since size of segment 2 is 100 and the offset in logical address is 500.
d. 3, 400 ⍰ 1327 + 400 = 1727
e. 4, 112 Illegal address since size of segment 4 is 96 and the offset in logical address is 112.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Paging

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Paging

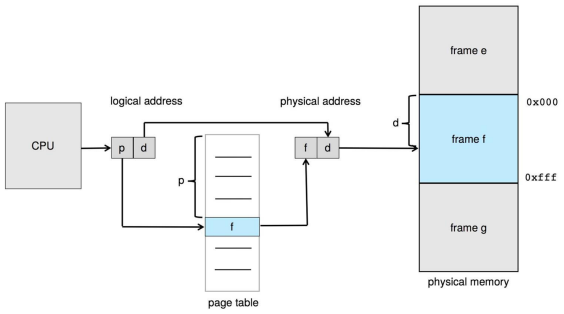
- Segmentation permits Physical address space of a process can be **noncontiguous**
- Paging is another memory-management scheme that offers this advantage.
  - **Avoids external fragmentation** and need for compaction
  - Solves the problem of fitting memory chunks of varying sizes onto the backing store.
- The backing store also has the fragmentation problems

OPERATING SYSTEMS
Paging: Basic Method

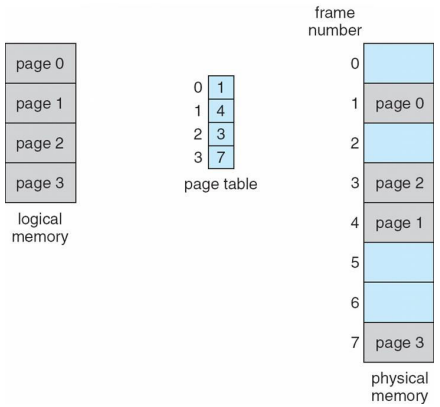
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- **Page size and frame size are defined by the hardware**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation
- Following command displays the page size supported in the system,

```
getconf   PAGESIZE
```

OPERATING SYSTEMS
Paging Hardware

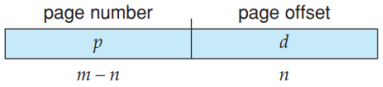


OPERATING SYSTEMS
Paging Model of Logical and Physical Memory



OPERATING SYSTEMS
Address Translation Scheme

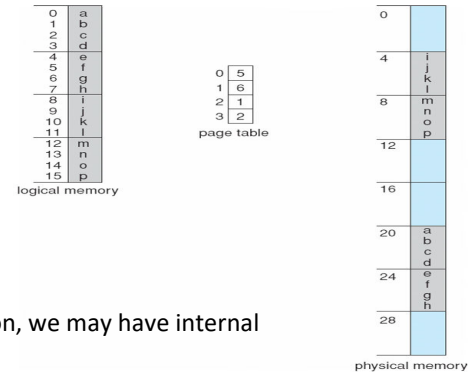
- Address generated by CPU is divided into:
  - **Page number** (*p*) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** (*d*) – combined with base address to define the physical memory address that is sent to the memory unit



  - Page size defined by hardware.
  - Page size varies between 512 bytes and 1 GB per page
  - The logical address space is 2^m and page size 2^n
  - The high-order m − n bits of a logical address designate the page number, and the n low-order bits designate the page offset

OPERATING SYSTEMS
Paging Example

- Logical address: n = 2 and m = 4. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)
- Logical address space = 2^m
- Page size = 2^n bytes



- No external fragmentation, we may have internal fragmentation

OPERATING SYSTEMS
Paging (Cont.)

- Calculating internal fragmentation
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - **Internal fragmentation of 2,048 - 1,086 = 962 bytes**
  - Worst case fragmentation = **1 frame – 1 byte**
  - On average fragmentation = 1 / 2 frame size
  - So small frame sizes desirable?
  - But each page table entry takes memory to track
  - Page sizes growing over time
- Programmer's view and physical memory now very different
- By implementation the user process can only access its own memory

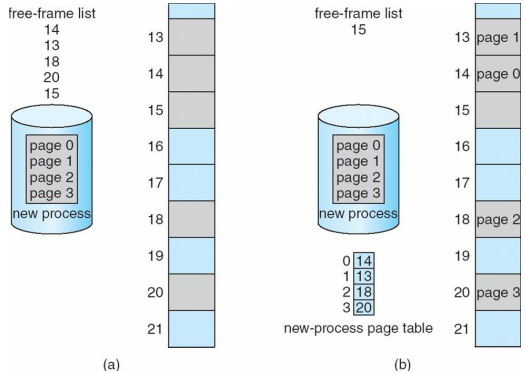
OPERATING SYSTEMS
Paging (Cont.)

- Worst case, a process would need *n* pages plus 1 byte.
  - needs to be allocated *n* + 1 frames, resulting in internal fragmentation of almost an entire frame.
- If the page size is smaller, an overhead is involved with each page table entry
- The overhead can be reduced by increasing the page size
- Page size are 4KB, 8KB
- Some CPU and kernel support multiple page sizes.
- Researchers are now developing support for variable on-the-fly page size.

OPERATING SYSTEMS
Paging (Cont.)

- On a 32-bit CPU, each page-table entry is 4 bytes long, but that size can vary as well.
- A 32-bit entry can point to one of 2^{32} physical page frames.
- If frame size is 4 KB (2^{12}), then a system with 4-byte entries can address 2^{44} bytes (or 16 TB) of physical memory.
- If other information is kept in the page-table entries, it reduces number of bits available to address page frames.
- A system with 32-bit page-table entries may address less physical memory than the possible maximum.

OPERATING SYSTEMS
Free Frames



Before allocation

After allocation

OPERATING SYSTEMS
Paging (Cont.)

- OS manages physical memory, keeps track of which frames are free or allocated using **frame table**.
- OS must be aware that user processes operate in user space, and all logical addresses must be mapped to produce physical addresses.
- **The operating system maintains a copy of the page table for each process**.
- It is used to translate logical addresses to physical addresses
- It is also used by the CPU dispatcher to define the hardware page table when a process is to be allocated the CPU.
- Paging therefore increases the context-switch time.
- How to reduce the context-switch time?
  - Using TLB

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Structure of the page table

Suresh Jamadagni
Department of Computer Science

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

Hardware Support for Page Table

- The hardware implementation of page table can be done by using dedicated registers.
- But the usage of register for the page table is satisfactory only if page table is small.
- If page table contain large number of entries then we can use TLB(translation Look-aside buffer), a special, small, fast look up hardware cache.

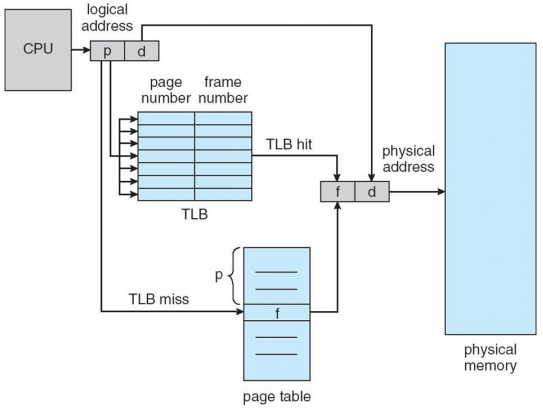
OPERATING SYSTEMS
Hardware Support for Page Table

- Page table is kept in main memory
- **Page-table base register** (**PTBR**) points to the page table
- **Page-table length register** (**PTLR**) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers** (**TLBs**)

OPERATING SYSTEMS
Translation Look-Aside Buffer

- Some TLBs store **address-space identifiers** (**ASIDs**) in each TLB entry
- This identifier is stored with each TLB entry to distinguish between entries loaded for different processes.
- Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered
  - Some entries can be **wired down** for permanent fast access

OPERATING SYSTEMS
Paging Hardware With TLB



OPERATING SYSTEMS
Effective Access Time

- Hit ratio – percentage of times that a page number is found in the TLB
- An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that 10 nanoseconds to access memory.
  - If we find the desired page in TLB then a mapped-memory access take 10 ns
  - Otherwise we need two memory access so it is 20 ns i.e. if we fail to find the page number in the TLB then we must first access memory for the page table and frame number (10 ns) and then access the desired byte in memory (10 nanoseconds), for a total of 20 ns (assuming that a page table lookup takes only one memory access).
- **Effective Access Time** (**EAT**)

  EAT = 0.80 x 10 + 0.20 x 20 = 12 nanoseconds

  implying 20% slowdown in access time
- Consider a more realistic hit ratio of 99%,

  EAT = 0.99 x 10 + 0.01 x 20 = 10.1ns

  implying only 1% slowdown in access time.

OPERATING SYSTEMS
Translation Look-Aside Buffer

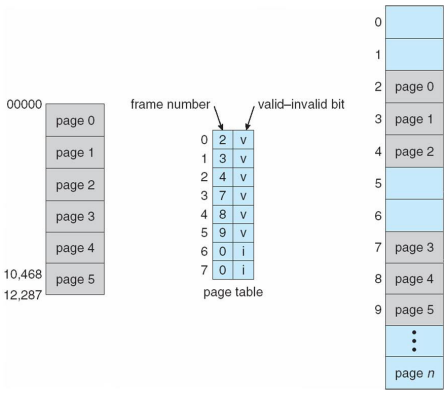
How TLB is different from CPU Cache

- TLB is about 'speeding up address translation for Virtual/logical memory' so that page-table needn't to be accessed for every address.
- CPU Cache is about 'speeding up main memory access latency' so that RAM isn't accessed always by CPU.
- TLB operation comes at the time of address translation by MMU while CPU cache operation comes at the time of memory access by CPU.

OPERATING SYSTEMS
Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
  - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use **page-table length register** (**PTLR**)
- Any violations result in a trap to the kernel

OPERATING SYSTEMS
Valid (v) or Invalid (i) Bit In A Page Table



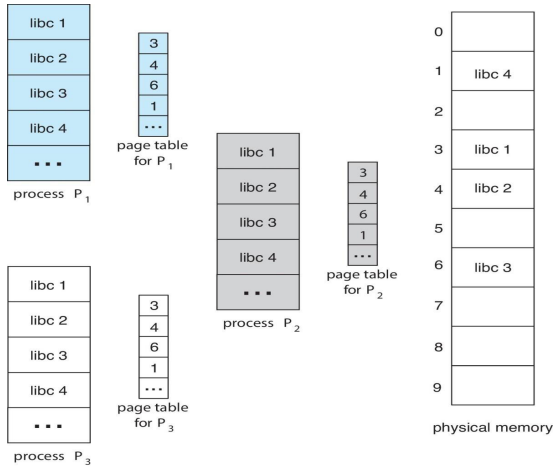
OPERATING SYSTEMS
Shared Pages

- An advantage of paging is the possibility of sharing common code.
  - important in a time-sharing environment
- Consider a system that supports 40 users, each of whom executes a text editor.
- If the text editor consists of 150 KB of code and 50 KB of data space.
- we need 8,000 KB(150 KBX40+50KB X40) to support the 40 users.
- Can this memory space usage be reduced?
  - YES
- If the code is shared, then a total space required will be 2,150 KB

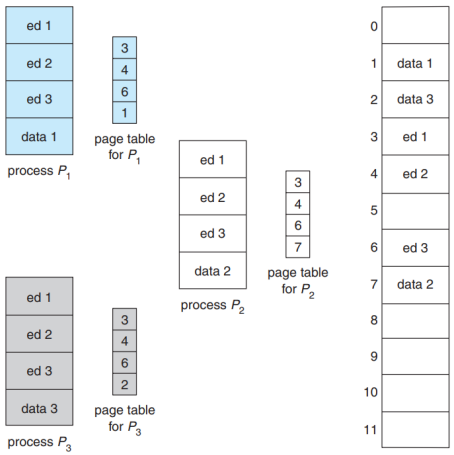
OPERATING SYSTEMS
Shared Pages

- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
  - Reentrant (multi-instance) code is a reusable routine that multiple programs can invoke, interrupt, and reinvoke simultaneously.
  - Reentrant code is non-self-modifying code: it never changes during execution.
  - Similar to multiple threads sharing the same process space
  - Also useful for interprocess communication if sharing of read-write pages is allowed
- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

OPERATING SYSTEMS
Shared Pages Example



OPERATING SYSTEMS
Shared Pages Example

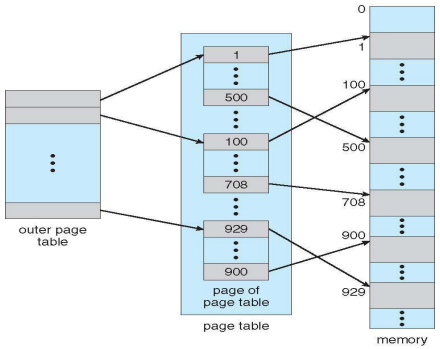


OPERATING SYSTEMS
Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB (2^{12})
  - Page table would have 1 million entries (2^{32} / 2^{12})
  - If each entry is 4 bytes -> 4 MB of physical memory for page table alone
    - That amount of memory used to cost a lot
    - Don't want to allocate that contiguously in main memory
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

OPERATING SYSTEMS
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



OPERATING SYSTEMS
Two-Level Paging Example

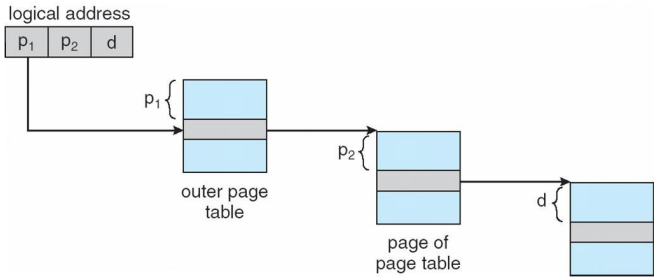
- A logical address (on 32-bit machine with 1K page size) is divided into:
  - a page number consisting of 22 bits
  - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
  - a 12-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

page number		page offset
p_1	p_2	d
12	10	10

where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table

OPERATING SYSTEMS
Address-Translation Scheme

- This scheme is known as **forward-mapped page table as** address translation works from the outer page table inward.



OPERATING SYSTEMS
64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
  - Then page table has 2^{52} entries
  - If two level scheme, inner page tables could be 2^{10} 4-byte entries
  - Address would look like

outer page	inner page	offset
p_1	p_2	d
42	10	12

- O
- One solution is to add a 2nd outer page table

OPERATING SYSTEMS
Three-level Paging Scheme

- We can divide the outer page table in various ways.
- For example, we can page the outer page table, giving us a three-level paging scheme. Suppose that the outer page table is made up of standard-size pages (2^{10} entries, or 2^{12} bytes).
- In this case, a 64-bit address space is still daunting:

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

The outer page table is still 2^{34} bytes (16 GB) in size. And possibly 4 memory access to get to one physical memory location

- The next step would be a four-level paging scheme, where the second-level outer page table itself is also paged, and so forth.
- The 64-bit UltraSPARC would require seven levels of paging—a prohibitive number of memory accesses— to translate each logical address. So, for 64-bit architectures, hierarchical page tables are generally considered inappropriate.

OPERATING SYSTEMS
Examples

Consider that the size of main memory (physical memory) is 512 MB and frame size of 4KB. Calculate

a.Page offset(d)

b.Bits to address page number/Frame number

c.What will be the total number of frames

d.Number of level of paging

OPERATING SYSTEMS
Hashed Page Tables

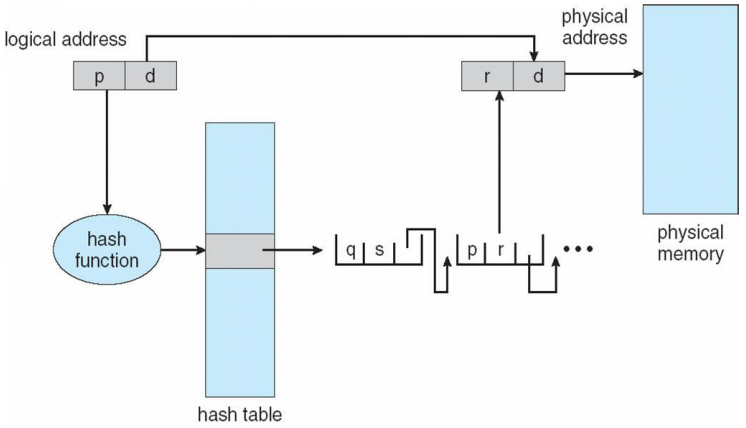
- Common in address spaces > 32 bits
- The virtual page number (VPN) is hashed into a page table
  - Each entry in the page table contains a linked list of elements that hash to the same location (to handle collisions)
- Each element contains three fields:

  (1) the virtual page number

  (2) the value of the mapped page frame

  (3) a pointer to the next element
- Virtual page numbers are compared (with field 1) in this chain searching for a match
  - If a match is found, the corresponding physical frame (field 2) is extracted
  - If there is no match, subsequent entries in the linked list are searched for a matching VPN

OPERATING SYSTEMS
Hashed Page Table



OPERATING SYSTEMS
Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
  - TLB can accelerate access
- But how to implement shared memory?
  - One mapping of a virtual address to the shared physical address

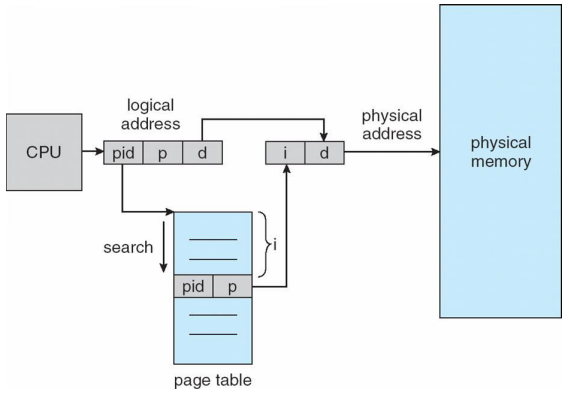
OPERATING SYSTEMS
Inverted Page Table

- A simplified version of the inverted page table used in the *IBM RT*.
- IBM was the first major company to use inverted page tables
  - starting with the IBM System 38 to RS/6000 and the current IBM Power CPUs.
  - For the IBM RT, each virtual address in the system consists of a triple:
    - <process-id, page-number, offset>.

OPERATING SYSTEMS
Inverted Page Table

- Each inverted page-table entry is a pair <process-id, page-number> where the process-id assumes the role of the address-space identifier.
- When a memory reference occurs, part of the virtual address, consisting of <process-id, page number>, *t*he inverted page table is then searched for a match.
- If a match is found—say, at entry *i*—then the physical address <*i*, offset> is generated.
- If no match is found, then an illegal address access has been attempted.

OPERATING SYSTEMS
Inverted Page Table Architecture



Used in 64-bit UltraSPARC (from Sun Micro Systems) and PowerPC (created by Apple-IBM-Motorola alliance)

OPERATING SYSTEMS
Inverted Page Table

- This scheme decreases the amount of memory needed to store each page table.
- It increases the amount of time needed to search the table when a page reference occurs.
- The inverted page table is sorted by physical address, but lookups occur on virtual addresses, the whole table might need to be searched before a match is found.
- Hash table is used to over come this problem where the number of search's are limited one or at most few.
- each access to the hash table adds a memory reference to the procedure.
- One virtual memory reference requires at least two real memory reads—one for the hash-table entry and one for the page table

OPERATING SYSTEMS
Inverted Page Table

- Systems that use inverted page tables have difficulty in implementing shared memory.
- Shared memory is usually implemented as multiple virtual addresses are mapped to one physical address.
- There is only one virtual page entry for every physical page
- One physical page cannot have two (or more) shared virtual addresses.
- A simple technique for addressing this issue is to allow the page table to contain only one mapping of a virtual address to the shared physical address.
- references to virtual addresses that are not mapped result in page faults

OPERATING SYSTEMS
Page table example

Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512 MB of physical memory. How many entries are there in each of the following?

a. A conventional single-level page table

b. An inverted page table

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Intel 32 and 64-bit Architectures

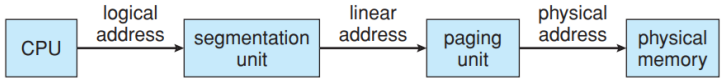
Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Logical to physical address translation in IA-32



- Memory management in IA-32 systems is divided into two components— segmentation and paging:
- The CPU generates logical addresses, which are given to the segmentation unit.
- The segmentation unit produces a linear address for each logical address.
- The linear address is then given to the paging unit, which in turn generates the physical address in main memory.
- The segmentation and paging units form the equivalent of the memory-management unit (MMU).

OPERATING SYSTEMS
IA-32 Segmentation

- The IA-32 architecture allows a segment to be as large as 4 GB
- Maximum number of segments per process is 16 K.
- The logical address space of a process is divided into two partitions. The first partition consists of up to 8K segments that are private to that process. The second partition consists of up to 8 K segments that are shared among all the processes.
- Information about the first partition is kept in the **local descriptor table (LDT)** and information about the second partition is kept in the **global descriptor table (GDT).**
- Each entry in the LDT and GDT consists of an 8-byte segment descriptor with detailed information about a particular segment, including the base location and limit of that segment.

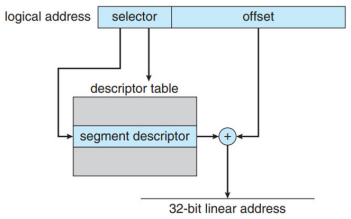
OPERATING SYSTEMS
IA-32 Segmentation (contd…)

- The logical address is a pair (selector, offset), where the selector is a 16-bit number
- s designates the segment number, g indicates whether the segment is in the GDT or LDT, and p deals with protection
- The offset is a 32-bit number specifying the location of the byte within the segment in question.
- The machine has **six segment registers**, allowing six segments to be addressed at any one time by a process.
- It also has **six 8-byte microprogram registers** to hold the corresponding descriptors from either the LDT or GDT. This cache lets the Pentium avoid having to read the descriptor from memory for every memory reference

s	g	p
13	1	2

OPERATING SYSTEMS
IA-32 Segmentation (contd…)

- The linear address on the IA-32 is 32 bits long and is formed as follows:
  - The segment register points to the appropriate entry in the LDT or GDT. The base and limit information about the segment in question is used to generate a linear address.
  - First, the limit is used to check for address validity. If the address is not valid, a memory fault is generated, resulting in a trap to the operating system.
  - If it is valid, then the value of the offset is added to the value of the base, resulting in a 32-bit linear address.



OPERATING SYSTEMS
IA-32 Paging

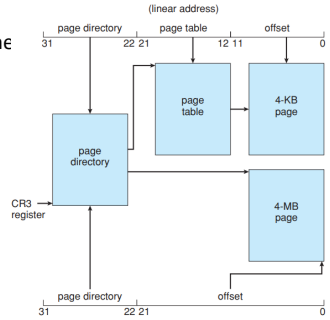
- The IA-32 architecture allows a page size of either 4 KB or 4 MB.
- For 4-KB pages, IA-32 uses a two-level paging scheme in which the division of the 32-bit linear address is as follows:

page number		page offset
p_1	p_2	d
10	10	12

- The 10 high-order bits reference an entry in the outermost page table, which IA-32 terms the page directory. (The CR3 register points to the page directory for the current process.)
- The page directory entry points to an inner page table that is indexed by the contents of the innermost 10 bits in the linear address.
- The low-order bits 0–11 refer to the offset in the 4-KB page pointed to in the page table.

OPERATING SYSTEMS
IA-32 Paging

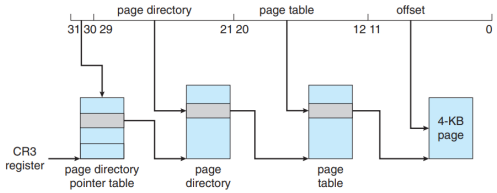
- One entry in the page directory is the Page Size flag, which if set, indicates that the size of the page frame is 4 MB and not the standard 4 KB.
- If this flag is set, the page directory points directly to the 4-MB page frame bypassing the inner page table; and the 22 low-order bits in the linear address refer to the offset in the 4-MB page frame



OPERATING SYSTEMS
IA-32 Paging

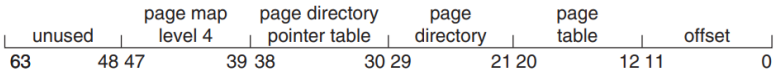
- To improve the efficiency of physical memory use, IA-32 page tables can be swapped to disk.
- An invalid bit is used in the page directory entry to indicate whether the table to which the entry is pointing is in memory or on disk.
- If the table is on disk, the operating system can use the other 31 bits to specify the disk location of the table.
- The table can then be brought into memory on demand.

OPERATING SYSTEMS
IA-32 Physical Address Extension



- Intel adopted a page address extension (PAE), which allows 32-bit processors to access a physical address space larger than 4 GB.
- The fundamental difference introduced by PAE support was that paging went from a two-level scheme to a **three-level scheme**, where the top two bits refer to a page directory pointer table.
- PAE also increased the page-directory and page-table entries from 32 to 64 bits in size, which allowed the base address of page tables and page frames to extend from 20 to 24 bits.
- Combined with the 12-bit offset, adding PAE support to IA-32 increased the address space to 36 bits, which supports up to 64 GB of physical memory.

OPERATING SYSTEMS
X86-64



- The x86-64 supported much larger logical and physical address spaces, as well as several other architectural advances.
- Support for a **64-bit address space yields an astonishing 264 bytes of addressable memory**—a number greater than 16 quintillion (or 16 exabytes).
- Even though 64-bit systems can potentially address this much memory, in practice far fewer than 64 bits are used for address representation in current designs.
- The x86-64 architecture currently provides a 48-bit virtual address with support for page sizes of 4 KB, 2 MB, or 1 GB using four levels of paging hierarchy
- Because this addressing scheme can use PAE, virtual addresses are 48 bits in size but support 52-bit physical addresses (4096 terabytes)

https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Virtual Memory

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Background

- Virtual memory is a technique that allows the execution of processes that are not completely in memory.
- One major advantage of this scheme is that programs can be larger than physical memory.
- This technique frees programmers from the concerns of memory-storage limitations.
- Virtual memory also allows processes to share files easily and to implement shared memory

OPERATING SYSTEMS
Background

- Code needs to be in memory to execute, but entire program rarely used
  - Error code, unusual routines, large data structures
- Entire program code not needed at same time
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running -> more programs run at the same time
    - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory -> each user program runs faster

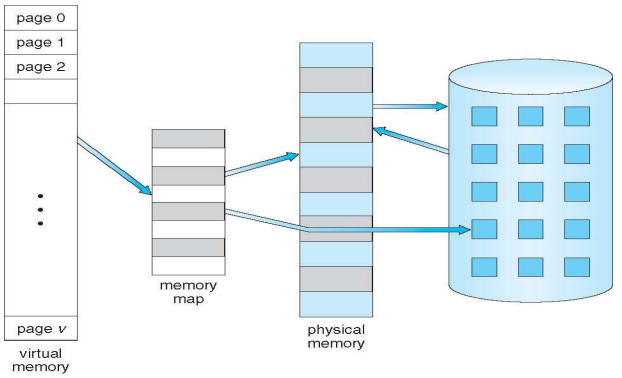
OPERATING SYSTEMS
Background (Cont.)

- **Virtual memory** – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes

OPERATING SYSTEMS
Background (Cont.)

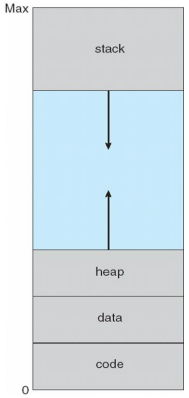
- **Virtual address space** – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

OPERATING SYSTEMS
Virtual Memory That is Larger Than Physical Memory

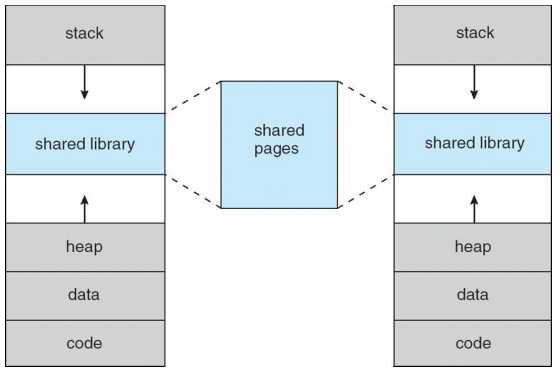


OPERATING SYSTEMS
Virtual-address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use
  - Unused address space between the two is hole
    - No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during fork(), speeding process creation

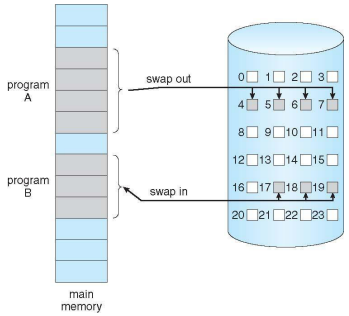


OPERATING SYSTEMS
Shared Library Using Virtual Memory



OPERATING SYSTEMS
Demand Paging

- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping (diagram on right)
- Page is needed ⇒ reference to it
  - invalid reference ⇒ abort
  - not-in-memory ⇒ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a **pager**

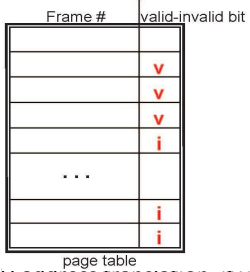


OPERATING SYSTEMS
Demand Paging – Basic Concepts

- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already **memory resident**
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load the page into memory from storage
    - Without changing program behavior
    - Without programmer needing to change code

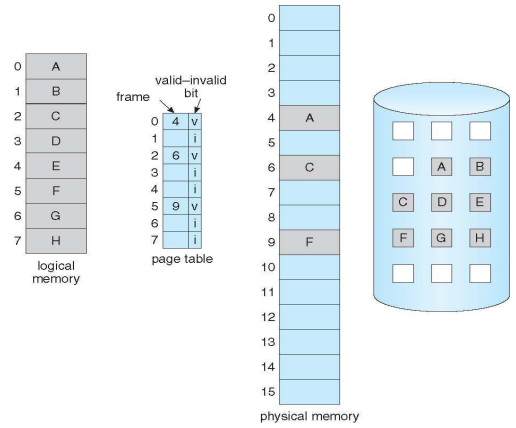
OPERATING SYSTEMS
Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:



- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

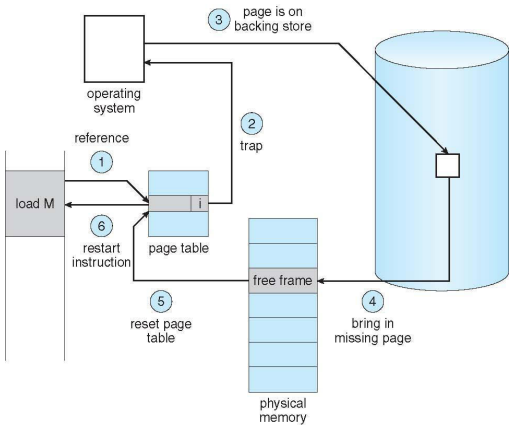
OPERATING SYSTEMS
Page Table When Some Pages Are Not in Main Memory



OPERATING SYSTEMS
Steps in Handling Page Fault

1. If there is a reference to a page, first reference to that page will trap to operating system
   - **Page fault**
2. Operating system looks at another table to decide:
   - Invalid reference ⇒ abort
   - Just not in memory
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory
   Set validation bit = **v**
6. Restart the instruction that caused the page fault

OPERATING SYSTEMS
Steps in Handling Page Fault (Cont.)



OPERATING SYSTEMS
Aspects of Demand Paging

- Extreme case – start process with *no* pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
  - And for every other process pages on first access
  - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
  - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
  - Pain decreased because of **locality of reference** - Process migrates from one locality (i.e., a set of pages that are actively used together) to another
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with **swap space**)
  - Instruction restart

OPERATING SYSTEMS
Instruction Restart

- Consider an instruction that could access several different locations (Ex: move some bytes from one location to another possibly overlapping location)
  - Source and destination blocks overlap i.e straddle a page boundary
    - Page fault might occur after the move is partially done
    - Source block may have been modified so we cannot simply restart the instruction
    - In one solution, the microcode computes and attempts to access both ends of both blocks.
      - Page fault can occur before anything is modified
    - The other solution uses temporary registers to hold the values of overwritten locations.
      - If Page fault occurs, all the old values are written back into memory before the trap occurs

OPERATING SYSTEMS
Performance of Demand Paging

- Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
   a) Wait in a queue for this device until the read request is serviced
   b) Wait for the device seek and/or latency time
   c) Begin the transfer of the page to a free frame

OPERATING SYSTEMS
Performance of Demand Paging (Cont.)

6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

OPERATING SYSTEMS
Performance of Demand Paging (Cont.)

- Three major activities
  - Service the interrupt – careful coding means just several hundred instructions needed
  - Read the page – lots of time
  - Restart the process – again just a small amount of time
- Page Fault Rate $0 \le p \le 1$ (p is the probability of a page fault)
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

EAT = (1 – *p*) x memory access
+ *p* x page-fault service time

OPERATING SYSTEMS
Demand Paging Example

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- EAT = (1 – p) x 200 + p (8 milliseconds)
  = (1 – p) x 200 + p x 8,000,000
  = 200 + p x 7,999,800

❑ EAT is directly proportional to page-fault rate

- If one access out of 1,000 causes a page fault i.e p = 0.001, then
  EAT = 8200 ns or 8.2 microseconds.
  This is a slowdown by a factor of 40!! (i.e. 8200/200)
- If we want performance degradation < 10 percent
  - 220 > 200 + 7,999,800 x p
    20 > 7,999,800 x p
  - p < .0000025
  - < one page fault in every 400,000 memory accesses

Demand Paging Optimizations

- Swap space I/O faster than file system I/O even if on the same device
  - Swap space is allocated in larger blocks, less management needed than file system
- Copy entire process image to swap space at process load time
  - Then page in and out of swap space
  - Used in older BSD Unix

Demand Paging Optimizations (Cont.)

- Demand page in from program binary on disk, but discard rather than paging out when freeing frame
  - Used in Solaris and current BSD
  - Still need to write to swap space
    - Pages not associated with a file (like stack and heap) – **anonymous memory**
    - Pages modified in memory but not yet written back to the file system
- Mobile systems
  - Typically don't support swapping
  - Instead, demand page from file system and reclaim read-only pages (such as code) from applications
    - if memory becomes constrained and demand page such data from file system later if needed

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering

sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

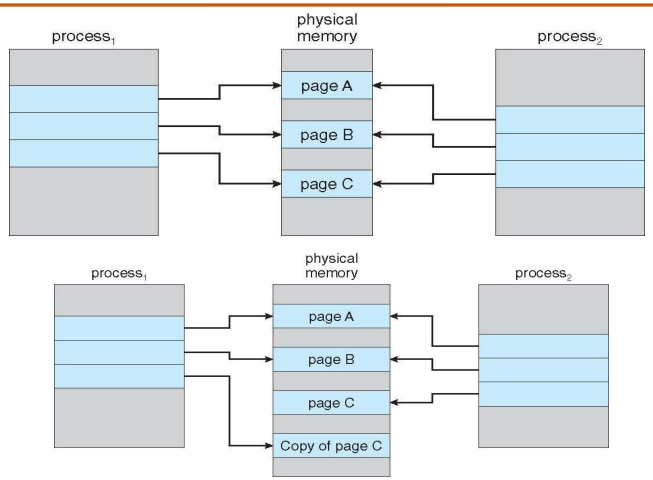
Virtual Memory

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

- **Copy-on-Write** (COW) allows both parent and child processes to initially ***share*** the same pages in memory
  - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a **pool** of **zero-fill-on-demand** pages
  - Pool should always have free frames for fast demand page execution
    - Don't want to have to free a frame as well as other processing on page fault
  - Why zero-out a page before allocating it?
- vfork() variation on fork() system call has parent suspend and child using copy-on-write address space of parent
  - Designed to have child call exec()
  - Very efficient



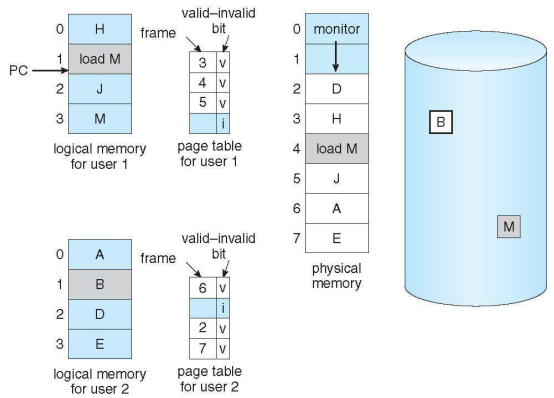
OPERATING SYSTEMS
What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
  - Algorithm – terminate? swap out? replace the page?
  - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

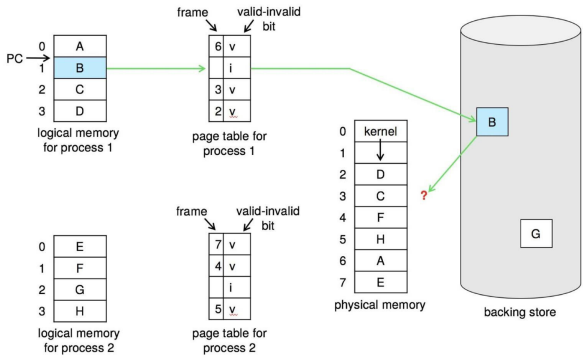
OPERATING SYSTEMS
Page Replacement

- Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement (vs simply increasing degree of multiprogramming)
- Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

OPERATING SYSTEMS
Need For Page Replacement – Example 1



OPERATING SYSTEMS
Need For Page Replacement –Example 2

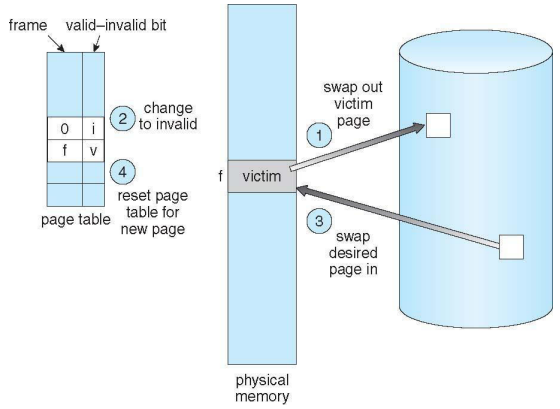


Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a **victim frame**
     - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing EAT

Basic Page Replacement (Cont.)



THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Virtual Memory

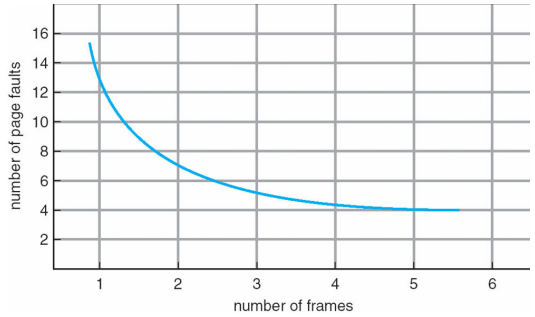
Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

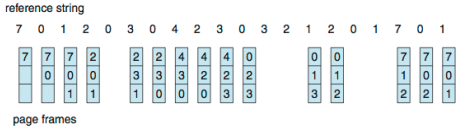
- **Frame-allocation algorithm** determines
  - How many frames to give each process
  - Which frames to replace
- **Page-replacement algorithm**
  - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
  - String is just page numbers, not full addresses
  - Repeated access to the same page does not cause a page fault
  - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

  7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1



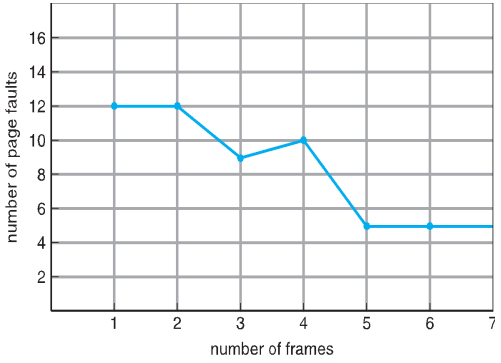
OPERATING SYSTEMS
First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



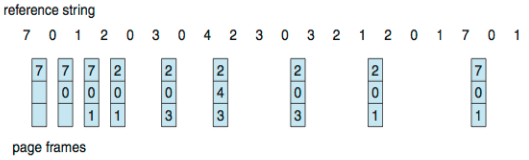
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - 4 **Belady's Anomaly**
- How to track ages of pages?
  - Just use a FIFO queue

OPERATING SYSTEMS
FIFO Illustrating Belady's Anomaly



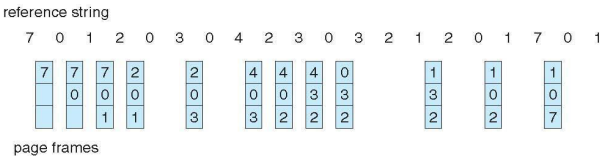
OPERATING SYSTEMS
Optimal Algorithm

- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs



OPERATING SYSTEMS
Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



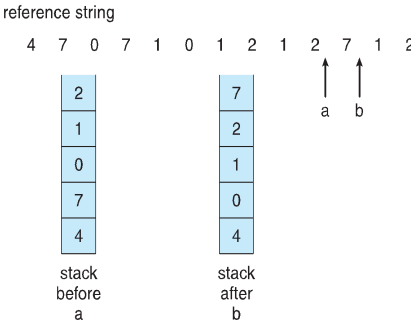
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

OPERATING SYSTEMS
LRU Algorithm (Cont.)

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value
    - Search through table needed
- Stack implementation
  - Keep a stack of page numbers in a double link form:
  - Page referenced:
    - move it to the top
    - requires 6 pointers to be changed
  - But each update more expensive
  - No search for replacement
- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

OPERATING SYSTEMS
Use Of A Stack to Record Most Recent Page References

- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly
- Use Of A Stack to Record Most Recent Page References



Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

THANK YOU

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Allocation of Frames

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

- How do we allocate the fixed amount of free memory among the various processes?

Example:

- Consider a single-user system with 128 KB of memory composed of pages 1 KB in size. Therefore, this system has 128 frames.
- The operating system may take 35 KB, leaving 93 frames for the user process.
- When a user process started execution, under pure demand paging, it would generate a sequence of page faults.
- The first 93 page faults would all get free frames from the free-frame list.
- When the free-frame list is exhausted, a page-replacement algorithm would be used.
- When the process is terminated, the 93 frames would once again be placed on the free-frame list

Minimum number of frames:

- One reason for allocating at least a minimum number of frames involves performance.
- As the number of frames allocated to each process decreases, the page-fault rate increases, slowing process execution
- When a page fault occurs before an executing instruction is complete, the instruction must be restarted. Consequently, we must have enough frames to hold all the different pages that any single instruction can reference
- The minimum number of frames is defined by the computer architecture.
- The maximum number is defined by the amount of available physical memory.
- In between, we are still left with significant choice in frame allocation

OPERATING SYSTEMS
Frame Allocation Algorithms

Equal allocation:

- The easiest way to split m frames among n processes is to give everyone an equal share, **m/n** frames (ignoring frames needed by the operating system)

  Example:

  - If there are 93 frames and 5 processes, each process will get 18 frames.
  - The three leftover frames can be used as a free-frame buffer pool.

OPERATING SYSTEMS
Frame Allocation Algorithms

Proportional allocation:

- Allocate available memory to each process according to its size
- Let the size of the virtual memory for process p_i be s_i and define $S = \sum s_i$
- If the total number of available frames is m, allocate a_i frames to process p_i , where a_i is approximately $a_i = s_i / S \times m$
- Adjust each a_i to be an integer that is greater than the minimum number of frames required by the instruction set, with a sum not exceeding m
- **Example**: To allocate 62 frames between two processes, one of 10 pages and one of 127 pages
  - $10/137 \times 62 \approx 4$
  - $127/137 \times 62 \approx 57$

OPERATING SYSTEMS
Global versus Local Allocation

- Page-replacement algorithms can be classified into two broad categories: global replacement and local replacement
- **Global replacement** allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; that is, one process can take a frame from another.
- **Local replacement** requires that each process select from only its own set of allocated frames.
- **Example**: Consider a global replacement scheme wherein high-priority processes are allowed to select frames from low-priority processes for replacement.
- A process can select a replacement from among its own frames or the frames of any lower-priority process. This approach allows a high-priority process to increase its frame allocation at the expense of a low-priority process
- With a local replacement strategy, the number of frames allocated to a process does not change.
- Global replacement generally results in greater system throughput and is therefore the more commonly used method.

OPERATING SYSTEMS
Non-uniform Memory Access

- In systems with multiple CPUs, a given CPU can access some sections of main memory faster than it can access others.
- These performance differences are caused by how CPUs and memory are interconnected in the system.
- Systems in which memory access times vary significantly are known collectively as **non-uniform memory access (NUMA)** systems, and without exception, they are slower than systems in which memory and CPUs are located on the same motherboard
- Managing which page frames are stored at which locations can significantly affect performance in NUMA systems. If we treat memory as uniform in such a system, CPUs may wait significantly longer for memory access than if we modify memory allocation algorithms to take NUMA into account.

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Virtual Memory

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

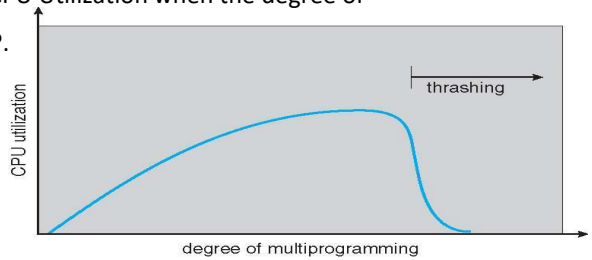
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Thrashing

- If a process does not have "enough" pages, the page-fault rate is very high
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - Operating system thinking that it needs to increase the degree of multiprogramming
    - Another process added to the system
- **Thrashing** ≡ a process is busy swapping pages in and out

OPERATING SYSTEMS
Cause of Thrashing

- Thrashing results in severe performance problems.
- Consider the scenario of the paging systems performance.
- As the degree of multiprogramming increases, CPU utilization also increases.
- Why there is a decrease in the CPU Utilization when the degree of multiprogramming is increased?.
- How this will be handled?

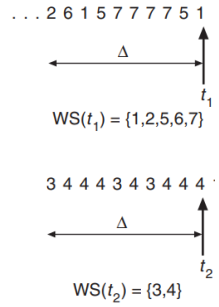


OPERATING SYSTEMS
Cause of Thrashing contd…

- We can Limit the effects of thrashing by using a **local replacement algorithm** (or priority replacement algorithm)
  - If process starts thrashing, it cannot steal frames from another processes
- how does demand paging work?
  Locality model
  - Process migrates from one locality to another
  - Localities may overlap
- Why does thrashing occur?
  Σ size of locality > total memory size
  - Limit effects by using local or priority page replacement

OPERATING SYSTEMS
Keeping Track of the Working Set

- The working-set model is based on the assumption of locality.
- The parameter, Δ , defines the working-set window.
- It is examine the most recent Δ page references.
- The set of pages in the most recent Δ page references is the working set.
- If a page is in active use, it will be in the working set.
- If it is no longer being used, it will drop from the working set time units after its last reference.
- Thus, the working set is an approximation of the program's locality
- Approximate the **W-S** model with interval timer + a reference bit



OPERATING SYSTEMS
Keeping Track of the Working Set

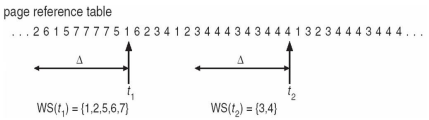
- Example: Δ = 10,000 references
  - Timer interrupts after every 5000 time units
  - Keep in memory 2 bits for each page
  - Whenever a timer interrupts copy and sets the values of all reference bits to 0
  - If one of the bits in memory = 1 ⇒ page in working set
- Why is this not completely accurate?
  - We cannot tell where, within an interval of 5000, a reference occurred.
- Improvement = 10 bits and interrupt every 1000 time units but overhead to service more frequent interrupts

OPERATING SYSTEMS
Working-Set Model

- Parameter Δ ≡ working-set window ≡ a fixed number of page references

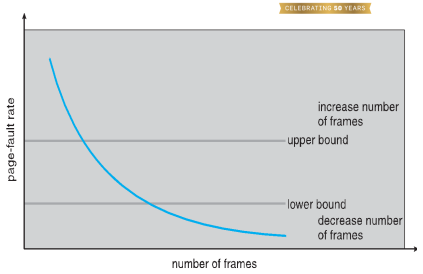
  Example: 10,000 instructions
- WSS_i (working set of Process P_i) =

  total number of pages referenced in the most recent Δ (parameter that varies in time)
  - if Δ too small will not encompass entire locality
  - if Δ too large will encompass several localities
  - if Δ = ∞ ⇒ will encompass entire program
- $D = \Sigma\ WSS_i$ ≡ total demand frames
  - Approximation of locality
- if $D > m$ *(available frames)* ⇒ Thrashing will occur
- Policy if $D >$ m, then suspend or swap out one of the processes



OPERATING SYSTEMS
Page-Fault Frequency

- More direct approach than W-S model
  - Control the page-fault rate
  - Looks clumsy to control thrashing
- Establish "acceptable" **page-fault frequency** (**PFF**) rate and use local replacement policy
  - If actual rate too low, process loses frame
  - If actual rate too high, process gains frame.
  - processes can be suspended if the free frames is zero
- With the working-set strategy, we may have to swap out a process.
  - If page fault Increases and no free frames available



THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Memory Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Virtual Memory – Case Study

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Virtual Memory Management in Windows

- Windows implements virtual memory using demand paging with clustering
- Clustering handles page faults by bringing in not only the faulting page but also several pages following the faulting page.
- When a process is first created, it is assigned a working-set minimum and maximum.
- The **working-set minimum** is the minimum number of pages the process is guaranteed to have in memory
- If sufficient memory is available, a process may be assigned as many pages as its **working-set maximum.**
- The virtual memory manager maintains a list of free page frames. Associated with this list is a threshold value that is used to indicate whether sufficient free memory is available.

OPERATING SYSTEMS
Virtual Memory Management in Windows

- If a page fault occurs for a process that is below its working-set maximum, the virtual memory manager allocates a page from this list of free pages.
- If a process that is at its working-set maximum incurs a page fault, it must select a page for replacement using a local LRU page-replacement policy
- When the amount of free memory falls below the threshold, the virtual memory manager uses a tactic known as **automatic working-set trimming** to restore the value above the threshold.
- Automatic working-set trimming works by evaluating the number of pages allocated to processes.
- If a process has been allocated more pages than its working-set minimum, the virtual memory manager removes pages until the process reaches its working-set minimum.
- A process that is at its working-set minimum may be allocated pages from the free-page-frame list once sufficient free memory is available. Windows performs working-set trimming on both user mode and system processes

OPERATING SYSTEMS
Virtual Memory Management in Linux Reference

- Linux uses demand paging to load executable images into a processes virtual memory.
- Whenever a command is executed, the file containing it is opened and its contents are mapped into the processes virtual memory. This is done by modifying the data structures describing this processes memory map and is known as *memory mapping*.
- However, only the first part of the image is actually brought into physical memory. The rest of the image is left on disk.
- As the image executes, it generates page faults and Linux uses the processes memory map in order to determine which parts of the image to bring into memory for execution.
- Linux on Alpha AXP systems uses 8 Kbyte pages and on Intel x86 systems it uses 4 Kbyte pages. Each of these pages is given a unique number; the **page frame number** (PFN).

OPERATING SYSTEMS
Virtual Memory Management in Linux

- In this paged model, a virtual address is composed of two parts; an offset and a virtual page frame number.
- If the page size is 4 Kbytes, bits 11:0 of the virtual address contain the offset and bits 12 and above are the virtual page frame number.
- Each time the processor encounters a virtual address it must extract the offset and the virtual page frame number.
- The processor must translate the virtual page frame number into a physical one and then access the location at the correct offset into that physical page. To do this the processor uses *page tables*.
- The page table is accessed using the virtual page frame number as an offset
- Linux uses a Least Recently Used (LRU) page aging technique to fairly choose pages which might be removed from the system.

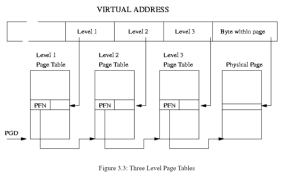


Figure 3.3: Three Level Page Tables

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

Storage Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File System

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
File Concept

- Computers can store information on various storage media, such as magnetic disks, magnetic tapes, and optical disks.
- File is logical storage where as disks are physical storage unit
- What is a File?
  - collection of related information
  - Data cannot to be written to disk unless they are in files
  - Data can be Numeric, alphabetic, alphanumeric, binary
- A file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user
- Contents of file can be source or executable programs, numeric or text data, photos, music, video, and so on.

File Attributes

Files are named and are referred by their names by the human users.

Attributes of files

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk.

File Attributes

- The information about all files is kept in the directory structure, which also resides on secondary storage.
- Directory entry consists of the file's name and its unique identifier.
- The identifier in turn locates the file attributes.

```
File: g.c
  Size: 218          Blocks: 0          IO Block: 4096   regular file
Device: 2h/2d   Inode: 3377699720577240  Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2020-09-15 15:18:57.338585500 +0530
Modify: 2021-03-20 10:40:37.416576700 +0530
Change: 2021-03-20 10:40:37.416576700 +0530
```


File Operations

File is an abstract data type.

File Operations

- **Create**
- **Write** – Use the system call to write to a file. A write pointer points to the location in the file where the next write will occur.
- **Read** – – Use the system call to read from a file. A read pointer points to the location in the file from where the next read is to take place
- **Reposition within file -** Repositioning within a file need not involve any actual I/O. It is done with system call lseek. Repositioning is also called seeking to location in a file
- **Delete:** Use the system call to delete/remove a file.
  - File to be deleted is searched in the directory.
  - Release the memory allocated after deletion

File Operations

- **Truncate:** The user may want to erase the contents of a file but keep its attributes.
  - Instead of deleting the file and then recreating it, use the function that sets the size of file to zero.
- **Appending** to the end of a existing file
- ***Open a file*** – use a system call open()
  - **Open-file table**: tracks open files
  - File pointer: pointer to last read/write location, per process that has the file open
  - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
- ***Close:*** *When a file is actively not used close it.*

OPERATING SYSTEMS
File Operations

- Several pieces of information are associated with an open file.
  - File pointer
  - File-open count.
  - Disk location of the file.
  - Access rights

OPERATING SYSTEMS
Open Files

- Provided by some operating systems and file systems
  - Similar to reader-writer locks
  - **Shared lock** similar to reader lock – several processes can acquire concurrently
  - **Exclusive lock** similar to writer lock
- OS mediates access to a file
- Mandatory or advisory:
  - **Mandatory** – access is denied depending on locks held and requested
  - **Advisory** – processes can find status of locks and decide what to do

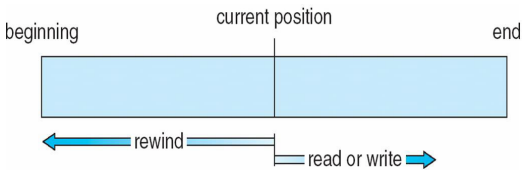
OPERATING SYSTEMS
File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

OPERATING SYSTEMS
File Structure

- None - sequence of words, bytes
- Simple record structure
  - Lines
  - Fixed length
  - Variable length
- Complex Structures
  - Formatted document
  - Relocatable load file
- Can simulate last two with first method by inserting appropriate control characters
- Who decides:
  - Operating system
  - Program

OPERATING SYSTEMS
Access Methods



OPERATING SYSTEMS
Sequential-access File

- **Sequential Access**
  - **read next**
  - **write next**
  - **reset**
- **Direct Access (or relative access)** – file is fixed length logical records
  - **read *n***
  - **write *n***
  - **position to *n***
    - **read next**
    - **write next**

  n = relative block number (i.e. index relative to the beginning of the file)
- Relative block numbers allow OS to decide where file should be placed

OPERATING SYSTEMS
Simulation of Sequential Access on Direct-access File

sequential access	implementation for direct access
reset	*cp* = 0;
read next	*read cp*; *cp* = *cp* + 1;
write next	*write cp*; *cp* = *cp* + 1;

- cp = current position
- Some systems allow only sequential file access; others allow only direct access

OPERATING SYSTEMS
Other Access Methods

- Can be built on top of base methods
- General involve creation of an index for the file
- Keep index in memory for fast determination of location of data to be operated on (consider UPC code plus record of data about that item with associated prices)
- If index file becomes too large, create index for the index file
  - Primary index file contains pointers to the secondary index files, which point to the actual data items
- IBM indexed sequential-access method (ISAM)
  - Small master index, points to disk blocks of secondary index (which points to the actual file blocks)
  - File kept sorted on a defined key
  - Binary search of the master index provides the block number of the secondary index and another binary search to find the block containing the desired record

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File System

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

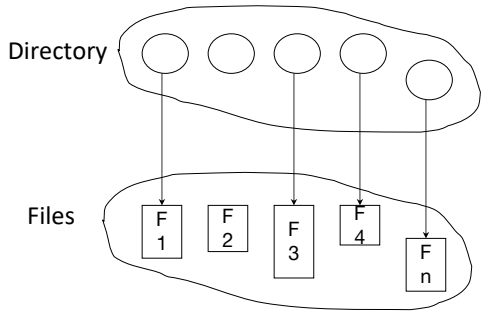
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Directory Structure

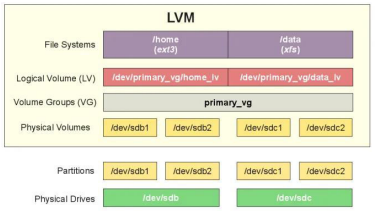
- A collection of nodes containing information about all files



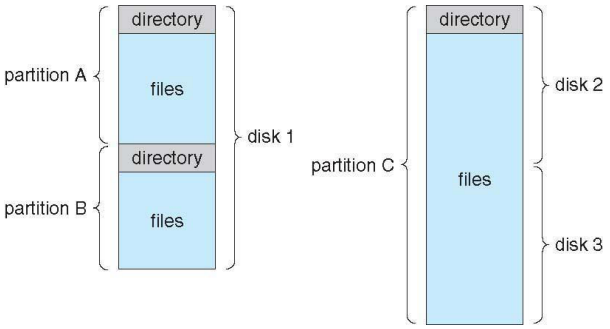
Both the directory structure and the files reside on disk

OPERATING SYSTEMS
Disk Structure

- Disk can be subdivided into **partitions**
- Disks or partitions can be **RAID** protected against failure
- Disk or partition can be used **raw** – without a file system, or **formatted** with a file system
- Partitions also known as minidisks, slices
- Entity containing file system known as a **volume**
- Each volume containing file system also tracks that file system's info in **device directory** or **volume table of contents**
- As well as **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer



OPERATING SYSTEMS
A Typical File-system Organization



OPERATING SYSTEMS
Types of File Systems

- We mostly talk of general-purpose file systems
- But systems frequently have may file systems, some general- and some special- purpose
- Consider Solaris has
  - tmpfs – memory-based volatile FS for fast, temporary I/O
  - objfs – interface into kernel memory to get kernel symbols for debugging
  - ctfs – contract file system for managing daemons
  - lofs – loopback file system allows one FS to be accessed in place of another
  - procfs – kernel interface to process structures
  - ufs, zfs – general purpose file systems

OPERATING SYSTEMS
Operations Performed on Directory

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

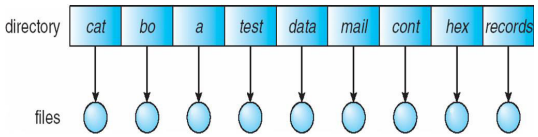
OPERATING SYSTEMS
Directory Organization

The directory is organized logically to obtain

- Efficiency – locating a file quickly
- Naming – convenient to users
  - Two users can have same name for different files
  - The same file can have several different names
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, …)

OPERATING SYSTEMS
Single-Level Directory

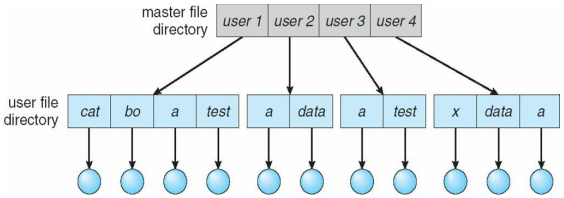
- A single directory for all users



- Naming problem
- Grouping problem

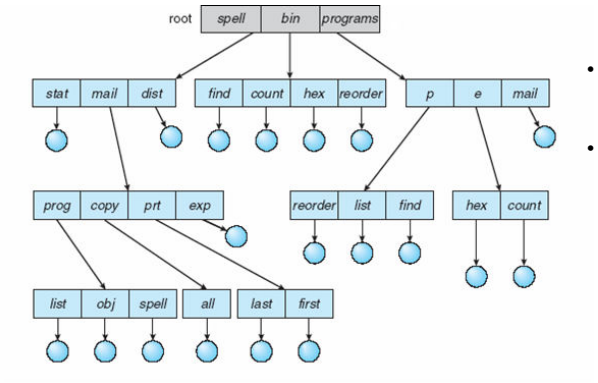
OPERATING SYSTEMS
Two-Level Directory

- Separate directory for each user



- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability

OPERATING SYSTEMS
Tree-Structured Directories



- A tree structure is the most common directory structure.
- The tree has a root directory, and every file in the system have a unique path.

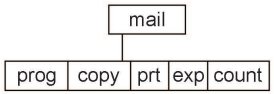
OPERATING SYSTEMS
Tree-Structured Directories (Cont.)

- Efficient searching
- Grouping Capability
- Current directory (working directory)
  - **cd /spell/mail/prog**
  - **type list**
- We cannot share files

OPERATING SYSTEMS
Tree-Structured Directories (Cont.)

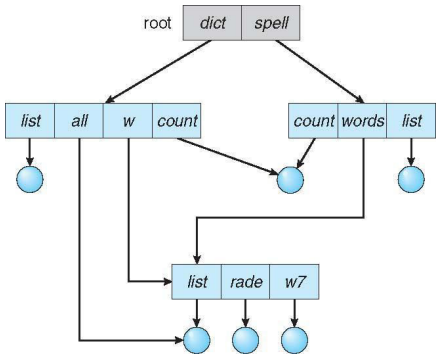
- **Absolute** or **relative** path name
- Creating a new file is done in current directory
- Delete a file
  rm <file-name>
- Creating a new subdirectory is done in current directory
  mkdir <dir-name>

❑ Example: if in current directory **/mail**
mkdir count



Deleting "mail" ⇒ deleting the entire subtree rooted by "mail"

OPERATING SYSTEMS
Acyclic-Graph Directories



- An acyclic graph is a graph with no cycle and allows to share subdirectories and files.
- The same file or subdirectories may be in two different directories
- It is used in a situation like when two programmers are working on a joint project and they need to access files.

OPERATING SYSTEMS
Acyclic-Graph Directories (Cont.)

- Two different names (aliasing)
- If ***dict*** deletes ***list*** ⇒ dangling pointer

  Solutions:
  - Backpointers, so we can delete all pointers
    Variable size records a problem
  - Backpointers using a daisy chain organization
  - Entry-hold-count solution
- New directory entry type
  - **Link** – another name (pointer) to an existing file
  - **Resolve the link** – follow pointer to locate the file

OPERATING SYSTEMS
General Graph Directory

Advantages:

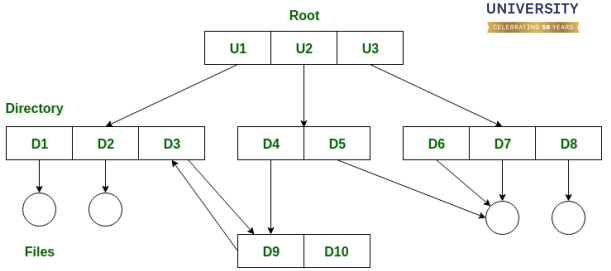
It allows cycles within a dir structure.

Multiple directories can be derived from more than one parent dir.

Disadvantages:

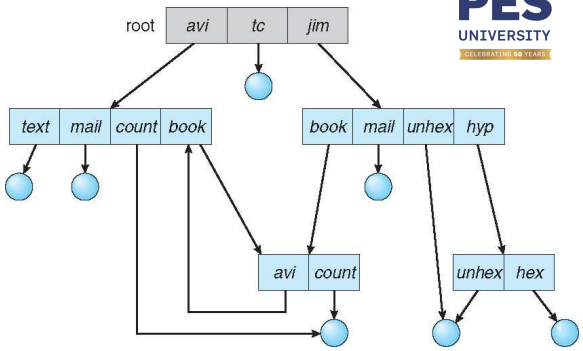
It is more costly than others.

It needs garbage collection (traversing the entire file system, marking everything that can be accessed)



OPERATING SYSTEMS
General Graph Directory (Contd.)

- How do we guarantee no cycles?
  - Allow only links to file not subdirectories
  - **Garbage collection**
  - Every time a new link is added use a cycle detection algorithm to determine whether it is OK



THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File System – File-System Mounting, File Sharing and File Protection

Suresh Jamadagni
Department of Computer Science

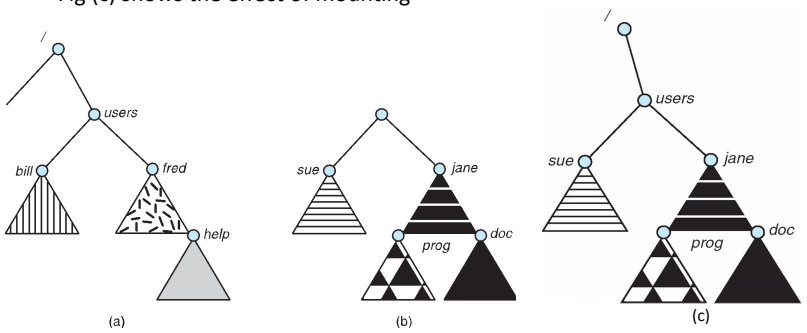
OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
File System Mounting

- A file system must be **mounted** before it can be accessed
- Fig (a) shows Existing File System
- A unmounted file system (i.e., Fig. (b)) is mounted at a **mount point**
- Fig (c) shows the effect of mounting



OPERATING SYSTEMS
File Sharing

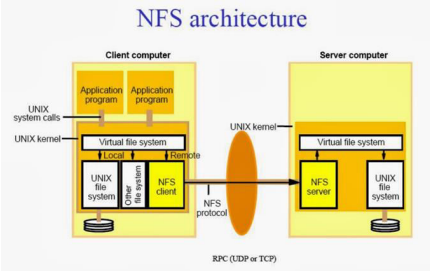
- Sharing of files on multi-user systems is desirable
- Sharing may be done through a **protection** scheme
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method
- If multi-user system
  - **User IDs** identify users, allowing permissions and protections to be per-user
    Group IDs allow users to be in groups, permitting group access rights
  - Owner of a file / directory
  - Group of a file / directory

OPERATING SYSTEMS
File Sharing – Remote File Systems

- Uses networking to allow file system access between systems
  - Manually via programs like FTP
  - Automatically, seamlessly using **distributed file systems**
  - Semi automatically via the **world wide web**

OPERATING SYSTEMS
File Sharing – Remote File Systems (Cont.)

- **Client-server** model allows clients to mount remote file systems from servers
  - Server can serve multiple clients
  - Client and user-on-client identification is insecure or complicated
  - **NFS** is standard UNIX client-server file sharing protocol
  - **CIFS** is standard Windows protocol
  - Standard operating system file calls are translated into remote calls
- Distributed Information Systems **(distributed naming services)** such as LDAP, DNS, NIS, Active Directory implement unified access to information needed for remote computing

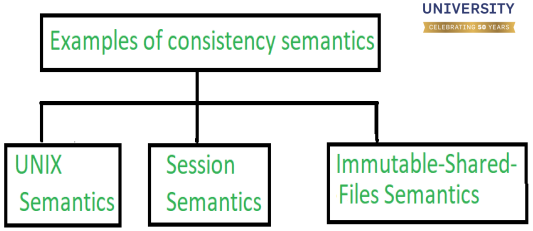


OPERATING SYSTEMS
File Sharing – Failure Modes

- All file systems have failure modes
  - For example corruption of directory structures or other non-user data, called **metadata**
- Remote file systems add new failure modes, due to network failure, server failure
- Recovery from failure can involve **state information** about status of each remote request
- **Stateless** protocols such as NFS v3 include all information in each request, allowing easy recovery but less security

File Sharing – Consistency Semantics

- **Consistency Semantics** is a concept which is used by users to check file systems which are supporting file sharing in their systems.
- Basically, it is a specification to check how in a single system multiple users are getting access to same file and at same time.
  - like when will modification by some user in some file is noticeable to others.



Used by AFS

File Sharing – Consistency Semantics (Contd.)

- Specify how multiple users are to access a shared file simultaneously
  - Similar to process synchronization algorithms
    - Tend to be less complex due to disk I/O and network latency (for remote file systems)
  - Andrew File System (AFS) implemented complex remote file sharing semantics
  - Unix file system (UFS) implements:
    - Writes to an open file visible immediately to other users of the same open file
    - Sharing file pointer to allow multiple users to read and write concurrently
  - AFS has session semantics
    - Writes only visible to sessions starting after the file is closed

Protection

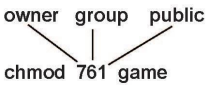
- File owner/creator should be able to control:
  - what can be done
  - by whom
- Types of access
  - **Read**
  - **Write**
  - **Execute**
  - **Append**
  - **Delete**
  - **List**

Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

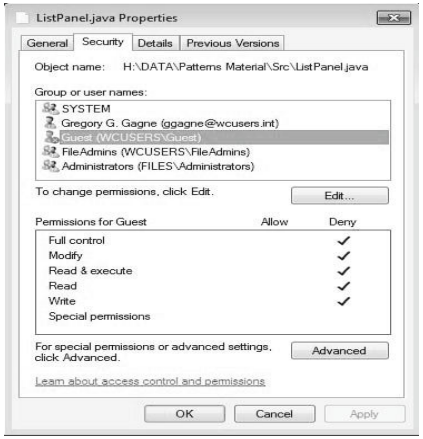
			RWX
a) **owner access**	7	⇒	1 1 1
b) **group access**	6	⇒	1 1 0
c) **public access**	1	⇒	0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.



Attach a group to a file

```
chgrp G game
```



THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Implementing File-Systems

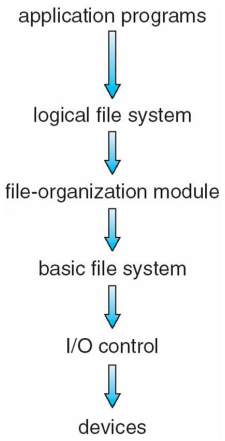
Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Layered File System



Manages metadata information, and the dir structure to provide the file-org module. Also responsible for protection

Translates logical to physical block addresses

Issues generic commands to the device driver, manages mem buffers

Device drivers and interrupt handlers

OPERATING SYSTEMS
File-System Structure

- File structure
  - Logical storage unit
  - Collection of related information
- **File system** resides on secondary storage (disks)
  - Provided user interface to storage, mapping logical to physical
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- Disk provides in-place rewrite and random access
  - I/O transfers performed in **blocks** of **sectors** (usually 512 bytes)
- **File control block (FCB)** – storage structure consisting of information about a file including ownership, permissions, and location of the file contents
- **Device driver** controls the physical device
- File system organized into layers

OPERATING SYSTEMS
File System Layers

- **Device drivers** manage I/O devices at the I/O control layer
  - Given commands like “read drive1, cylinder 72, track 2, sector 10, into memory location 1060” outputs low-level hardware specific commands to hardware controller
- **Basic file system** given command like “retrieve block 123” translates to device driver
- Also manages memory buffers and caches (allocation, freeing, replacement)
  - Buffers hold data in transit
  - Caches hold frequently used data

File System Layers (Cont.)

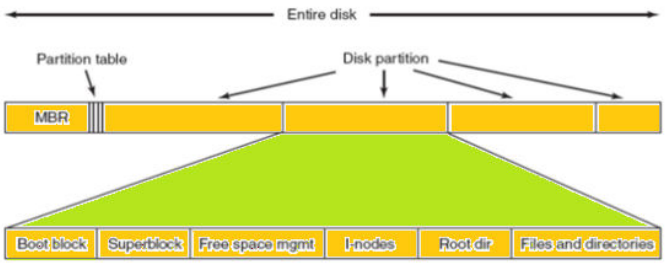
- **File organization module** understands files, logical address, and physical blocks
- Translates logical block # to physical block #
- Manages free space, disk allocation
- **Logical file system** manages metadata information
  - Translates file name into file number, file handle, location by maintaining file control blocks (**inodes** in UNIX)
  - Directory management
  - Protection

File System Layers (Cont.)

- Layering useful for reducing complexity and redundancy, but adds overhead and can decrease performance.
- Logical layers can be implemented by any coding method according to OS designer

- Many file systems, sometimes many within an operating system
  - Each with its own format (CD-ROM is ISO 9660; Unix has **UFS**, FFS; Windows has FAT, FAT32, NTFS as well as floppy, CD, DVD Blu-ray, Linux has more than 40 types, with **extended file system** ext2 and ext3 leading; plus distributed file systems, etc.)
  - New ones – ZFS, GoogleFS, Oracle ASM, FUSE

File-System Implementation



A possible file system layout.

File-System Implementation

- We have system calls at the API level, but how do we implement their functions?
  - On-disk and in-memory structures
- **Boot control block** contains info needed by system to boot OS from that volume
  - Needed if volume contains OS, usually first block of volume
- **Volume control block** (**superblock, master file table**) contains volume details
  - Total # of blocks, # of free blocks, block size, free block pointers or array
- Directory structure organizes the files
  - Names and inode numbers, master file table

OPERATING SYSTEMS
File-System Implementation (Cont.)

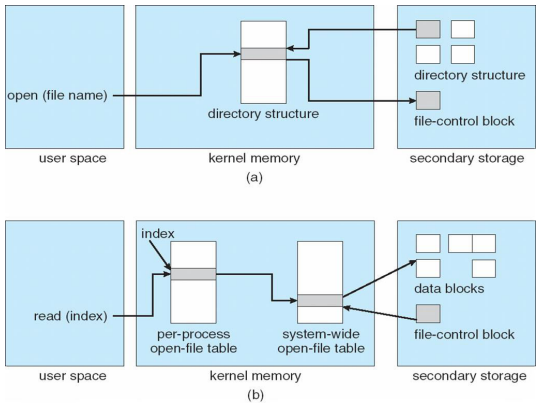
- Per-file **File Control Block (FCB)** contains many details about the file
  - inode number, permissions, size, dates
  - NFTS stores into in master file table using relational DB structures

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

OPERATING SYSTEMS
In-Memory File System Structures

- Mount table storing file system mounts, mount points, file system types
- The following figure illustrates the necessary file system structures provided by the operating systems
- Figure (a) refers to opening a file
- Figure (b) refers to reading a file
- Plus buffers hold data blocks from secondary storage
- Open returns a file handle for subsequent use
- Data from read eventually copied to specified user process memory address

OPERATING SYSTEMS
In-Memory File System Structures



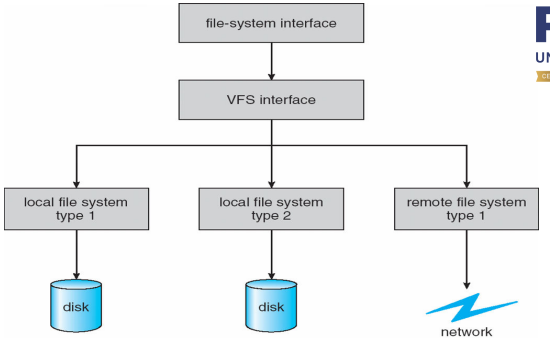
OPERATING SYSTEMS
Partitions and Mounting

- Partition can be a volume containing a file system (“cooked”) or **raw** – just a sequence of blocks with no file system
- Boot block can point to boot volume or boot loader set of blocks that contain enough code to know how to load the kernel from the file system
  - Or a boot management program for multi-os booting
- **Root partition** contains the OS, other partitions can hold other OS’s, other file systems, or be raw
  - Mounted at boot time
  - Other partitions can mount automatically or manually
- At mount time, file system consistency checked
  - Is all metadata correct?
    - If not, fix it, try again
    - If yes, add to mount table, allow access

- Virtual File Systems (VFS) on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
  - Separates file-system generic operations from implementation details
  - Implementation can be one of many file systems types, or network file system
    4 Implements vnodes which hold inodes or network file details
  - Then dispatches operation to appropriate file system implementation routines

OPERATING SYSTEMS
Virtual File Systems

- The API is to the VFS interface, rather than any specific type of file system (FS)
- VFS provides a single set of commands for the kernel and developers to access all types of FSs
- VFS software calls the specific device driver required to interface to various types of FSs
- Device driver interprets the standard set of FS commands to a specific type of FS on the partition or logical volume



OPERATING SYSTEMS
Virtual File Systems

- For example, Linux has four object types:
  - **Inode object, file object, superblock object, dentry object**
- VFS defines set of operations on the objects that must be implemented
  - Every object has a pointer to a function table
    Function table has addresses of routines to implement that function on that object
    For example:
    - **int open(. . .)**—Open a file
    - **int close(. . .)**—Close an already-open file
    - **ssize t read(. . .)**—Read from a file
    - **ssize t write(. . .)**—Write to a file
    - **int mmap(. . .)**—Memory-map a file

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Implementing File-Systems – Disk Space Allocation Methods

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

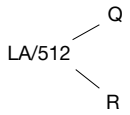
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Allocation Methods - Contiguous

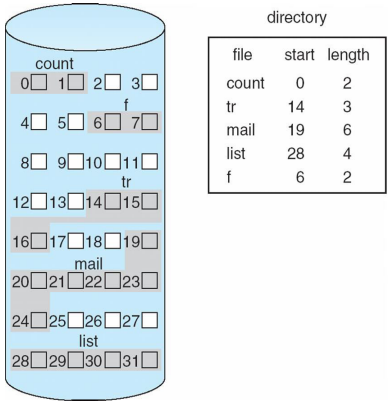
- An allocation method refers to how disk blocks are allocated for files:
- **Contiguous allocation** – each file occupies set of contiguous blocks
  - Best performance in most cases
  - Simple – only starting location (block #) and length (number of blocks) are required
  - Supports both sequential and direct access
  - Problems include finding space for file, knowing file size, external fragmentation, need for **compaction off-line** (**downtime**) or **on-line**

OPERATING SYSTEMS
Contiguous Allocation

- Mapping from logical to physical



- LA – Length of the area allocated for this file
- Q = displacement into index table
- Block to be accessed = Q + starting address
- Displacement into block = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

OPERATING SYSTEMS
Extent-Based Systems

- Many newer file systems (i.e., Veritas File System) use a modified contiguous allocation scheme
- Extent-based file systems allocate disk blocks in extents
- An **extent** is a contiguous block of disks
  - Extents are allocated for file allocation
  - A file consists of one or more extents

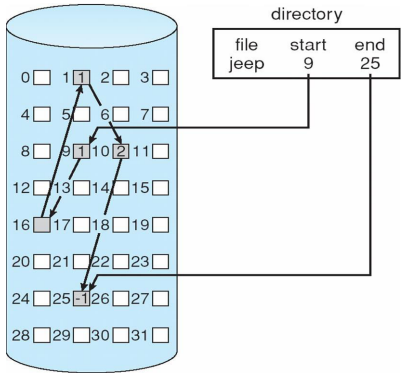
OPERATING SYSTEMS
Allocation Methods - Linked

- **Linked allocation** – each file a linked list of blocks
  - File ends with null pointer
  - No external fragmentation
  - Each block contains pointer to next block
  - No compaction, external fragmentation
  - Free space management system called when new block needed
  - Improve efficiency by clustering blocks into groups but increases internal fragmentation
  - Reliability can be a problem
  - Locating a block can take many I/Os and disk seeks

OPERATING SYSTEMS
Allocation Methods – Linked (Cont.)

- FAT (File Allocation Table) variation
  - Beginning of volume has table, indexed by block number
  - Much like a linked list, but faster on disk and cacheable
  - New block allocation simple

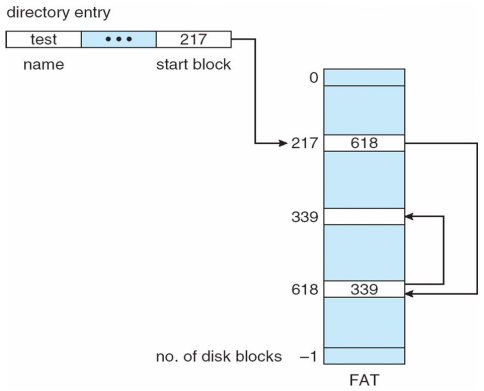
OPERATING SYSTEMS
Linked Allocation



- Each block contains a pointer to the next block.
- If each block is 512 bytes and a disk address (pointer) requires 4 bytes then the user sees blocks of 508 bytes (i.e. some disk space is wasted)
- Collect blocks into multiples called **clusters** and allocate clusters rather than blocks

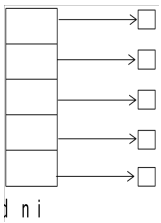
OPERATING SYSTEMS
File-Allocation Table (variation on linked allocation)

- Unused block is indicated by a table value of 0
- To allocate a new block to a file, find the first 0-valued table entry and replace the previous EOF value with the address of the new block.
- Then replace 0 with EOF value



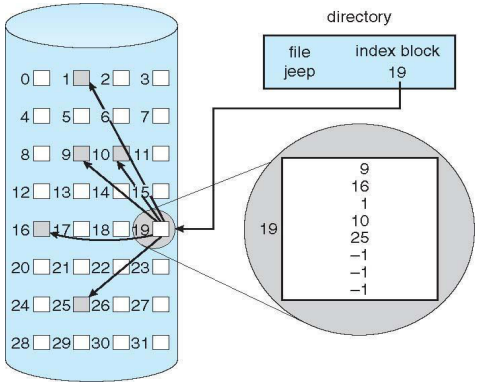
OPERATING SYSTEMS
Allocation Methods - Indexed

- **Indexed allocation**
  - Each file has its own **index block**(s) of pointers to its data blocks
- Logical view



OPERATING SYSTEMS
Example of Indexed Allocation

- In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file.
- Each file has its own index block. The ith entry in the index block contains the disk address of the ith file block.
- The directory entry contains the address of the index block as shown in the figure.



Allocation Methods – Indexed (Cont.)

Advantages:

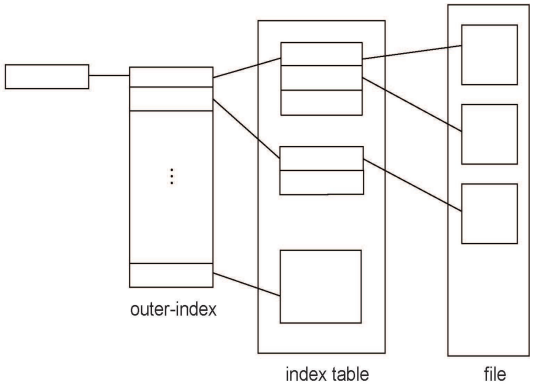
- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. (Note: In linked allocation we lose the space of only 1 pointer per block.)

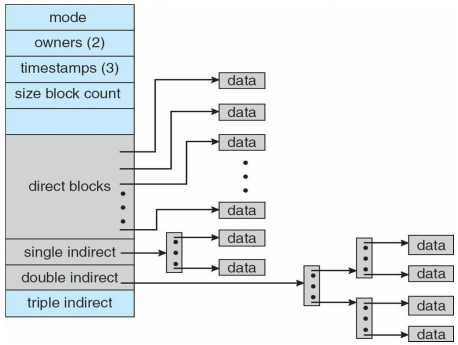
Indexed Allocation – Mapping (Cont.)

- For files that are very large, single index block may not be able to hold all the pointers.
- Other Schemes such as Linked scheme, Multilevel index and Combined Scheme are used.



Combined Scheme: UNIX UFS

4K bytes per block, 32-bit addresses



More index blocks than can be addressed with 32-bit file pointer

Performance

- Best method depends on file access type
  - Contiguous great for sequential and random
- Linked good for sequential, not random
- Declare access type at creation -> select either contiguous or linked
- Indexed more complex
  - Single block access could require 2 index block reads then data block read
  - Clustering can help improve throughput, reduce CPU overhead

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.du

OPERATING SYSTEMS
UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File Management – Free space management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

Free-Space Management

- Need to reuse the disk space from deleted files for new files
- To keep track of free disk space, the system maintains a **free-space list**
- The free-space list records all **free disk blocks**
- When a file is created, the free-space list is searched for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list.
- When a file is deleted, its disk space is added to the free-space list

Bit Vector

- The free-space list is implemented as a **bit map** or **bit vector**.
- Each block is represented by 1 bit.
- If the block is free, the bit is 1; if the block is allocated, the bit is 0
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The free-space bit map would be 00111100111111000110000001110000 ...
- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk

Bit Vector

- One technique for finding the first free block on a system that uses a bit-vector to allocate disk space is to sequentially check each word in the bit map to see whether that value is not 0, since a 0-valued word contains only 0 bits and represents a set of allocated blocks
- The first non-0 word is scanned for the first 1 bit, which is the location of the first free block.

Bit Vector

- The calculation of the block number is

  (number of bits per word) × (number of 0-value words) + offset of first 1 bit

  Example:

  Number of bits per word = 8

  Number of consecutive 0-value words = 1 (Word 0)

  Offset of the first 1 bit in the subsequent word (Word 1) = 5 (counting from 0)

  Applying the formula:

  Block Number = 8 × 1 + 5 = 13

  Thus, block 13 is the first free block.

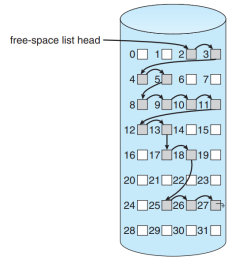
Word 0: 00000000
Word 1: 00000111
Word 2: 11110000

OPERATING SYSTEMS
Bit Vector

- Bit vectors are inefficient unless the entire vector is kept in main memory
- Keeping the bit vector in main memory is possible for smaller disks but not necessarily for larger ones
- A 1.3-GB disk with 512-byte blocks would need a bit map of over 332 KB to track its free blocks
- A 1-TB disk with 4-KB blocks requires 32 MB to store its bit map.

OPERATING SYSTEMS
Linked List

- Link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- The first free block contains a pointer to the next free disk block, and so on
- **Example:** Consider blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The pointer would point to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4 and so on
- This scheme is not efficient; to traverse the list, one must read each block, which requires substantial I/O time.



OPERATING SYSTEMS
Grouping

- A modification of the free-list approach stores the addresses of n free blocks in the first free block.
- The first n−1 of these blocks are actually free.
- The last block contains the addresses of another n free blocks, and so on.
- The addresses of a large number of free blocks can now be found quickly, unlike the situation when the standard linked-list approach is used.

OPERATING SYSTEMS
Counting

- Generally, several contiguous blocks may be allocated or freed simultaneously, particularly when space is allocated with the contiguous-allocation algorithm or through clustering.
- Rather than keeping a list of n free disk addresses, we can keep the address of the first free block and the number (n) of free contiguous blocks that follow the first block
- Each entry in the free-space list then consists of a disk address and a count.
- The entries can be stored in a balanced tree, rather than a linked list, for efficient lookup, insertion, and deletion.

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File Management – Efficiency and Performance

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Efficiency

Efficiency dependent on:

- Disk allocation and directory algorithms
- Types of data kept in file's directory entry
- Pre-allocation or as-needed allocation of metadata structures
- Fixed-size or varying-size data structures

OPERATING SYSTEMS
Performance

Performance dependent on:

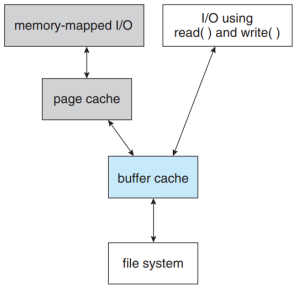
- Keeping data and metadata close together
- Buffer cache – separate section of main memory for frequently used blocks
- Synchronous writes sometimes requested by apps or needed by OS
  - No buffering / caching – writes must hit disk before acknowledgement
  - Asynchronous writes more common, buffer-able, faster
- Free-behind and read-ahead – techniques to optimize sequential access
- Reads frequently slower than writes

OPERATING SYSTEMS
Page cache

- Some systems maintain a separate section of main memory for a **buffer cache**, where blocks are kept under the assumption that they will be used again shortly.
- Other systems **cache file data using a page cache**. The page cache uses virtual memory techniques to cache file data as pages rather than as file-system-oriented blocks.
- Caching file data using virtual addresses is far more efficient than caching through physical disk blocks, as accesses interface with virtual memory rather than the file system.
- Several systems—including Solaris, Linux, and Windows —use page caching to cache both process pages and file data. This is known as **unified virtual memory**.

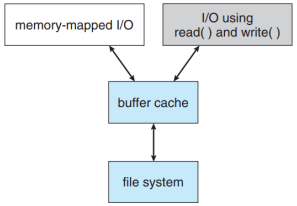
OPERATING SYSTEMS
I/O without a unified buffer cache

- Consider standard system calls read() and write()
- Without a unified buffer cache, the read() and write() system calls go through the buffer cache.
- The memory-mapping call, requires two caches—the page cache and the buffer cache.
- A memory mapping proceeds by reading in disk blocks from the file system and storing them in the buffer cache. Because the virtual memory system does not interface with the buffer cache, the contents of the file in the buffer cache must be copied into the page cache.
- This situation, known as **double caching**, requires caching file-system data twice. Not only does it waste memory but it also wastes significant CPU and I/O cycles due to the extra data movement within system memory.



OPERATING SYSTEMS
I/O using a unified buffer cache

- When a unified buffer cache is provided, both memory mapping and the read() and write() system calls use the same page cache.
- This has the benefit of avoiding double caching, and it allows the virtual memory system to manage file-system data



OPERATING SYSTEMS
Synchronous and Asynchronous writes

- Another issue that can affect the performance of I/O is whether writes to the file system occur synchronously or asynchronously.
- Synchronous writes occur in the order in which the disk subsystem receives them, and the writes are not buffered. Thus, the calling routine must wait for the data to reach the disk drive before it can proceed.
- In an asynchronous write, the data are stored in the cache, and control returns to the caller.
- Most writes are asynchronous, metadata writes can be synchronous

OPERATING SYSTEMS
Page replacement

- Sequential access can be optimized by techniques known as free-behind and read-ahead.
- **Free-behind** removes a page from the buffer as soon as the next page is requested. The previous pages are not likely to be used again and waste buffer space.
- With **read-ahead**, a requested page and several subsequent pages are read and cached. These pages are likely to be requested after the current page is processed.
- Retrieving these data from the disk in one transfer and caching them saves a considerable amount of time

OPERATING SYSTEMS
Disk cache

- When data is written to a disk file, the pages are buffered in the disk cache, and the disk driver sorts its output queue according to disk address to minimize disk-head seeks
- Unless synchronous writes are required, a process writing to disk simply writes into the cache, and the system asynchronously writes the data to disk when convenient
- The user process sees very fast writes.
- When data are read from a disk file, the block I/O system does some read-ahead
- Writes are much more nearly asynchronous than are reads. Thus, output to the disk through the file system is often faster than is input for large transfers, counter to intuition

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

Storage Management

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS

Mass-Storage Structure

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

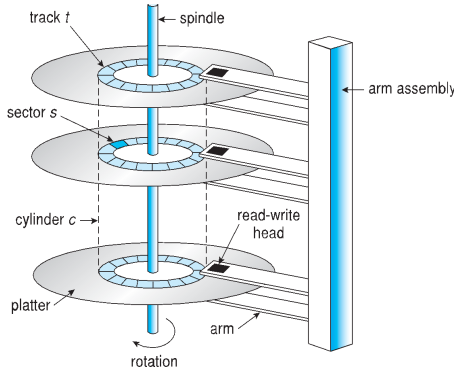
- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

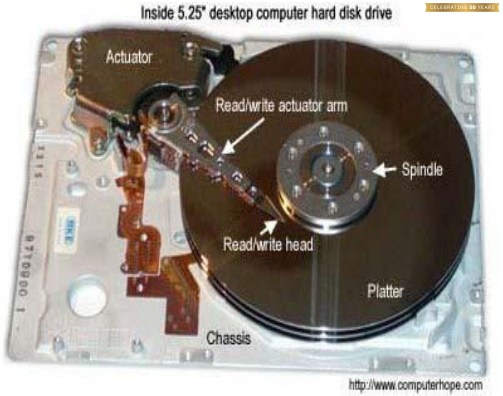
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Overview of Mass Storage Structure

- **Magnetic disks** provide bulk of secondary storage of modern computers.
- Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 3.5 inches.
- The two surfaces of a platter are covered with a magnetic material.
- The heads are attached to a disk arm that moves all the heads as a unit.
- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- The set of tracks that are at one arm position makes up a cylinder.

OPERATING SYSTEMS
Moving-head Disk Mechanism





OPERATING SYSTEMS
Overview of Mass Storage Structure

- When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 250 times per second.

Disk speed has two parts.

- The transfer rate is the rate at which data flow between the drive and the computer.
- The positioning time, or random-access time, consists of two parts
  - The time necessary to move the disk arm to the desired cylinder, called the **seek time**.
  - the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.

Disk Latency = Seek time + Rotational latency + Transfer time

OPERATING SYSTEMS
Magnetic Disk Specifications – Reference: www.seagate.com

Specifications	Barracuda®	Constellation®	Constellation ES	SV35 Series™
Primary Applications	Optimised for PC and personal external storage	Optimised for 2.5-inch business server and external storage arrays	Optimised for 3.5-inch business server and external storage arrays	Optimised for video surveillance applications
Capacity (GB)	250, 320, 500, 750, 1,000 1,500, 2,000, 3,000	250, 500, 1,000	500, 1,000, 2,000, 3,000	1,000, 2,000, 3,000
Spin Speed (RPM)	7,200	7,200	7,200	7,200
SATA interface (Gb/s)	1.5/3.0/6.0	1.5/3.0/6.0	1.5/3.0/6.0	1.5/3.0/6.0
SAS interface (Gb/s)	—	3.0/6.0	3.0/6.0	—
Rotational vibration (RV) (radians/s/s)	5.5 narrow spectrum up to 300Hz	16 broad spectrum up to 1,800Hz	12.5 broad spectrum up to 1,500Hz	5.5 narrow spectrum up to 300Hz
Seek time, average read/write (ms)	<8.5/<9.5	8.5/9.5	8.5/9.5	<8.5/<9.5
Cache (MB)[1]	16, 64	Up to 64	Up to 64	64
Non-recoverable read errors per bits read	1 sector per 10^{14}	1 sector per 10^{15}	1 sector per 10^{15}	1 sector per 10^{14}
Power-on hours (POH)	2,400 – 8x5	8,760 - 24x7	8,760 - 24x7	8,760- 24x7
Streaming capabilities	—	Multiple sequential streams	Multiple sequential streams	Up to 20 simultaneous HD streams[3]
POH usage profile	8x5 – On as needed	24x7 – Always on	24x7 – Always on	Up to 64 cameras 24x7 – Always on
MTBF (hours)	700,000	1.4 million	1.2 million	1 million
Power, average – idle (W)[2]	4.6	2.25 to 3.85	>3.74	—
Power, average – idle2 (W)[2]	3.4 to 5.4	—	—	3.4 to 5.4
Acoustics, typical – idling (bels)	2.2 to 2.4	2.2	1.9 to 2.7	2.2 to 2.4
Shock, operating/non-operating (Gs)	70 to 80/300 to 350	70/400	40 to 70/300	80/300 to 350
Ambient temperature, operating/ non-operarting (°C)	0 to 60/–40 to 70	5 to 60/–40 to 70	5 to 60/–40 to 70	0 to 70/–40 to 70
RAID support	0, 1	0, 1, 3, 4, 5, 6, 10	0, 1, 3, 4, 5, 6, 10	0, 1, 3, 4, 5, 6, 10
RAID Rebuild™ support		•	•	
Enterprise expert support		•	•	•

OPERATING SYSTEMS
Overview of Mass Storage Structure

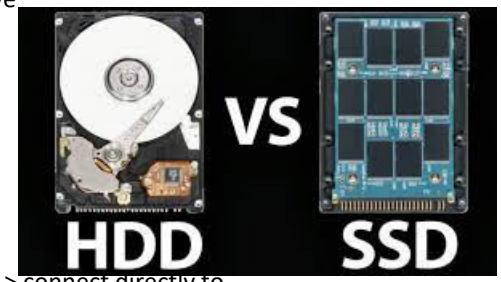
- Disk head flies on an extremely thin cushion of air.
- head will sometimes damage the magnetic surface when it makes contact with it.
  - Head crash
  - Not recoverable
- A disk can be **removable.**
- Removable magnetic disks generally consist of one platte
- Examples CDs, DVDs, and Blu-ray discs as well as removable flash-memory

OPERATING SYSTEMS
Mass Storage Structure (Cont.)

- Drive attached to computer via **I/O bus**
  - Busses vary, including **EIDE**, **ATA**, **SATA**, **USB**, **Fiber Channel**, **SCSI, SAS, Firewire**
    - SCSI is a set of parallel interface standards developed by ANSI
    - Allows more hard disks per computer compared to IDE
    - SCSI is more complicated to configure compared to SATA and IDE
    - EIDE has a 133 megabytes per second speed rate, while SATA has up to a 150 megabytes per second speed rate.
  - **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array

OPERATING SYSTEMS
Solid-State Disks

- Nonvolatile memory used like a hard drive
  - Many technology variations
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- But much faster
- Standard Bus interfaces can be too slow -> connect directly to the system bus (PCI, for example)
- No moving parts, so no seek time or rotational latency



OPERATING SYSTEMS
Magnetic Tape

- Was early secondary-storage medium
  - Evolved from open spools to cartridges
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems

Magnetic Tape (Cont.)

- Kept in spool and wound or rewound past read-write head
- Moving to the correct spot on a tape can take minutes, but once positioned, tape drives can write data at speeds comparable to disk drives.
- Tape capacities vary greatly, depending on the particular kind of tape drive, with current capacities exceeding several terabytes.
- Some tapes have built-in compression that can more than double the effective storage.
- Tapes and their drivers are usually categorized by width, including 4, 8, and 19 millimeters and 1/4 and 1/2 inch.
- Some are named according to technology, such as LTO-5 and SDLT.

THANK YOU

Chandravva Hebbi
Department of Computer Science Engineering
chandravvahebbi@pes.edu

OPERATING SYSTEMS

UE22CS242B

Storage Management

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS

Mass-Storage Structure – Disk Scheduling

FCFS, SSTF, SCAN, C-SCAN, LOOK

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time ≈ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer

OPERATING SYSTEMS
Disk Scheduling (Cont.)

- There are many sources of disk I/O request
  - OS
  - System processes
  - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
  - Optimization algorithms only make sense when a queue exists

OPERATING SYSTEMS
Disk Scheduling (Cont.)

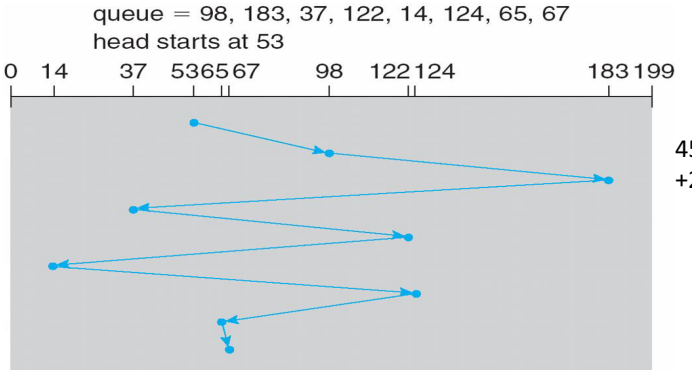
- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53

OPERATING SYSTEMS
FCFS

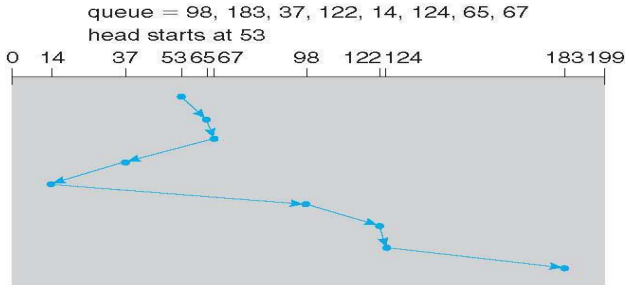
Illustration shows total head movement of 640 cylinders



45+85+146+85+108+110+59 +2 = 640 cylinders

OPERATING SYSTEMS
SSTF

- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Illustration shows total head movement of 236 cylinders

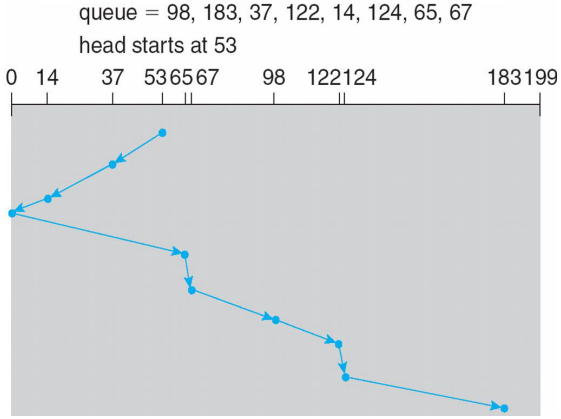


12+2+30+23+84+24+2+59 = 236 cylinders

OPERATING SYSTEMS
SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest

OPERATING SYSTEMS
SCAN (Cont.)

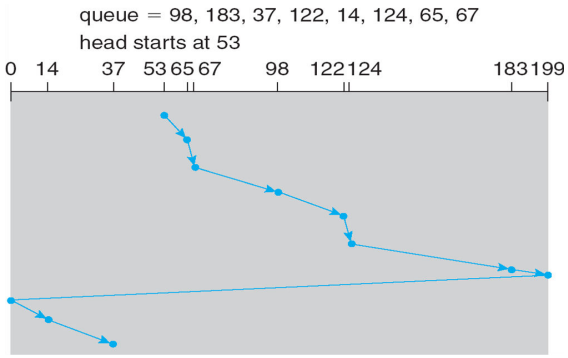


Number of cylinder moves= 16+23+79+2+31+24+2+59=236

OPERATING SYSTEMS
C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
  - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?

OPERATING SYSTEMS
C-SCAN (Cont.)

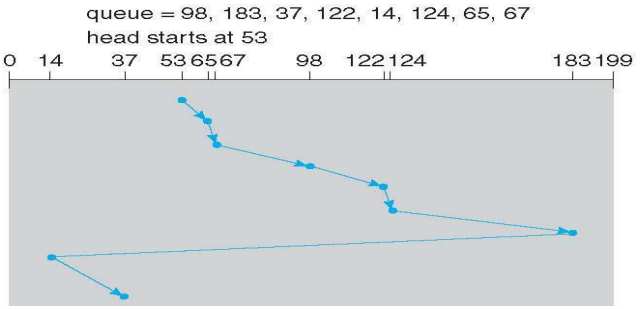


= (65-53)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)+(199-183)+(199-0)+(14-0)+(37-14)
=12+2+31+24+2+59+16+199+14+23
= 382

OPERATING SYSTEMS
LOOK & C-LOOK

- LOOK a version of SCAN, C-LOOK a version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk
- Total number of cylinders?

OPERATING SYSTEMS
C-LOOK (Cont.)



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (183 – 14) + (37 – 14)
= 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23
= 322

Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
  - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
  - And metadata layout
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm

Selecting a Disk-Scheduling Algorithm (Cont.)

- What about rotational latency?
  - Difficult for OS to calculate
  - The rotational latency can be nearly as large as the average seek time.
  - It is difficult for the operating system to schedule for improved rotational latency, though, because modern disks do not disclose the physical location of logical blocks.
  - Disk manufacturers have been alleviating this problem by implementing disk-scheduling algorithms in the controller hardware built into the disk drive.
- How does disk-based queueing effect OS queue ordering efforts?
  - If the OS sends a batch of requests to the controller, the controller can queue them and then schedule them to improve both the seek time and the rotational latency.

THANK YOU

Chandravva Hebbi
Department of Computer Science Engineering
chandravvahebbi@pes.edu

OPERATING SYSTEMS

UE22CS242B

Storage Management

Chandravva Hebbi
Department of Computer Science

OPERATING SYSTEMS

Mass-Storage Structure – Swap Space and RAID

Chandravva Hebbi
Department of Computer Science

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

Swap-Space Management

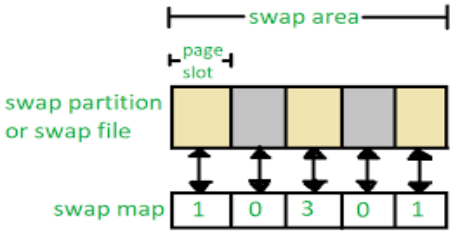
- Swap-space — Virtual memory uses disk space as an extension of main memory
  - Less common now due to memory capacity increases
- Swap-space can be carved out of the normal file system, or, more commonly, it can be in a separate disk partition (raw)
- Swap-space management
  - 4.3BSD allocates swap space when process starts; holds text segment (the program) and data segment
  - Kernel uses **swap maps** to track swap-space use
  - Some systems allow the use of multiple swap spaces – both files and dedicated swap partitions

Swap-Space Management (Cont.)

- Solaris 2 allocates swap space only when a dirty page is forced out of physical memory, not when the virtual memory page is first created
  - File data written to swap space until write to file system requested
  - Other dirty pages go to swap space due to no other home
  - Text segment pages thrown out and reread from the file system as needed
- What if a system runs out of swap space?
- Some systems allow multiple swap spaces

OPERATING SYSTEMS
Data Structures for Swapping on Linux Systems

- Linux allows one or more swap areas to be established.
- A swap area may be in either a swap file on a regular file system or a dedicated swap partition.
- Each swap area consists of a series of 4-KB **page slots**, which are used to hold swapped pages.
- Associated with each swap area is a **swap map**—an array of integer counters, each corresponding to a page slot in the swap area.
  - 0 => page slot is available
  - 3 => swapped page is mapped to 3 different processes



OPERATING SYSTEMS
RAID Basics

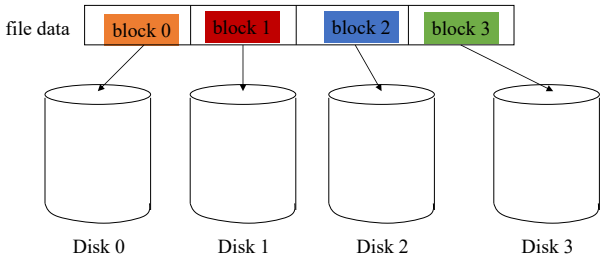
- RAID is a Disk Organization technique
  - Addresses performance and reliability issues
  - Multiple disk drives provide (or improve) reliability via **redundancy**
- Many systems today need to store many terabytes of data
- Don't want to use single, large disk
  - too expensive
  - failures could be catastrophic
- Would prefer to use many smaller disks
- **Redundant Array of Independent (inexpensive) Disks**

OPERATING SYSTEMS
RAID Basics (Cont.)

- Basic idea is to connect multiple disks together to provide
  - large storage capacity
  - faster access to reading data
  - redundant data
- Many different levels of RAID systems
  - differing levels of redundancy, error checking, capacity, and cost

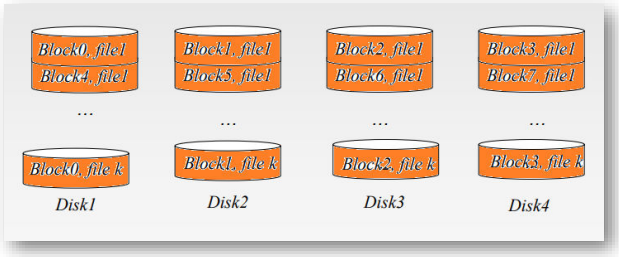
OPERATING SYSTEMS
Striping

- Take file data and map it to different disks
- Allows for reading data in parallel
- Transfer rate is improved by striping data across the disks
  - **Bit-level striping and block-level striping (most common)**



OPERATING SYSTEMS
Block-level Striping

❑With n disks, block i of a file goes to disk (i mod n) + 1
- Requests for different blocks can run in parallel if the blocks reside on different disks
- A request for a long sequence of blocks can utilize all disks in parallel



OPERATING SYSTEMS
Mirroring

- Keep two copies of data on two separate disks
- Gives good error recovery
  - if some data is lost, get it from the other source
- Expensive
  - requires twice as many disks
- Write performance can be slow
  - have to write data to two different spots
- Read performance is enhanced
  - can read data from file in parallel

OPERATING SYSTEMS
RAID Structure

- **Mean time to failure**
  - If MTTF of a single disk is 100,000 hours, MTTF of some disk in an array of 100 disks = 100000/100 = 1000 hours or 41.66 days
- **Mean time to repair –** time taken to replace a failed disk and to restore the data on it
- **Mean time to data loss** based on above factors
  - If mirrored disks fail independently (i.e., not related to power failures and natural disasters), consider disk with MTTF of 100,000 hours and MTTR is 10 hours
  - Mean time to data loss of a mirrored disk system is $100,000^2 / (2 * 10) = 500 * 10^6$ hours, or 57,000 years!

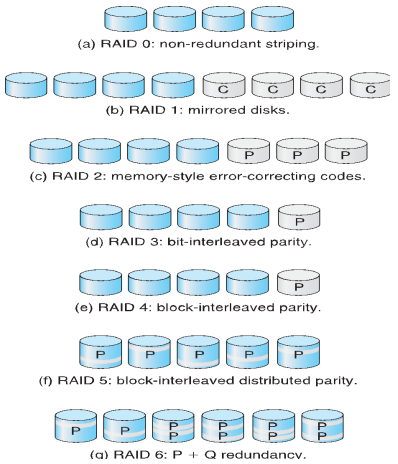
OPERATING SYSTEMS
RAID (Cont.)

- Frequently combined with **NVRAM** to improve write performance
- Several improvements in disk-use techniques involve the use of multiple disks working cooperatively
- Disk **striping** uses a group of disks as one storage unit
- RAID is arranged into six different levels
- RAID schemes improve performance and improve the reliability of the storage system by storing redundant data
  - **Mirroring** or **shadowing** (**RAID 1**) keeps duplicate of each disk
  - Striped mirrors (**RAID 1+0**) or mirrored stripes (**RAID 0+1**) provides high performance and high reliability
  - **Block interleaved parity** (**RAID 4, 5, 6**) uses much less redundancy

OPERATING SYSTEMS
RAID (Cont.)

- RAID within a storage array can still fail if the array fails, so automatic **replication** of the data between arrays is common
- Frequently, a small number of **hot-spare** disks are left unallocated, automatically replacing a failed disk and having data rebuilt onto them

OPERATING SYSTEMS
RAID Levels



P indicates error-correcting bits and C indicates a second copy of the data

P + Q redundancy scheme uses 2 parity values P and Q

OPERATING SYSTEMS
RAID Levels 0 and 1

- **RAID level 0** refers to disk arrays with striping at the level of blocks
  - No redundancy (such as mirroring or parity bits)
  - lots of disks means low Mean Time To Failure (MTTF)

- **RAID level 1** refers to disk mirroring.
  - A complete file is stored on a single disk
  - A second disk contains an exact copy of the file
  - Provides complete redundancy of data
  - Read performance can be improved
  - file data can be read in parallel
  - Write performance suffers
  - must write the data out twice
  - Most expensive RAID implementation
  - requires twice as much storage space

OPERATING SYSTEMS
RAID Level-2

- **RAID level 2** is also known as memory-style error correcting code (ECC) organization.

  - Stripes data across disks similar to Level-0
    - difference is data is **bit** interleaved instead of **block** interleaved
    - For ex, the first bit of each byte can be stored in disk 1, the second bit in disk 2, and so on until the eighth bit is stored in disk 8; the error-correction bits are stored in further disks.
  - Uses ECC to monitor correctness of information on disk
    - Multiple disks record the ECC information to determine which disk is in fault
    - A parity disk is then used to reconstruct corrupted or lost data
  - RAID level 2 requires only 3 disks' overhead for 4 disks of data, unlike RAID level 1, which requires 4 disks' overhead.

OPERATING SYSTEMS
RAID Level-3

- **RAID level 3**, or bit-interleaved parity organization;
  - One big problem with Level-2 is the number of extra disks needed to detect which disk had an error
  - Modern disks can already determine if there is an error
    - using ECC codes with each sector
  - So just need to include a parity disk
    - if a sector is bad, the disk itself tells us, and use the parity disk to correct it
  - Transfer rate for reading or writing a single block is faster than RAID level 1.
  - But supports fewer I/Os per second, since every disk has to participate in every I/O request.
  - Has performance problem due to the expense of computing and writing the parity.

(d) RAID 3: bit-interleaved parity.

OPERATING SYSTEMS
RAID Level-4

- **RAID level 4** interleaves file blocks
  - allows multiple small I/O's to be done at once

(e) RAID 4: block-interleaved parity.

- Consists of block-level striping with dedicated parity.
- Still use a single disk for parity
- Now the parity is calculated over data from multiple blocks
  - Level-2,3 calculate it over a single block
- If an error detected, need to read other blocks on other disks to reconstruct data
- Doing multiple small reads is now faster than before
- However, writes are still very slow
  - this is because of calculating and writing the parity blocks
- Also, only one write is allowed at a time
  - all writes must access the check disk so other writes have to wait

OPERATING SYSTEMS
RAID Level-5

- **RAID level 5** stripes file data and checks data over all the disks
  - no longer a single check disk
  - no more write bottleneck

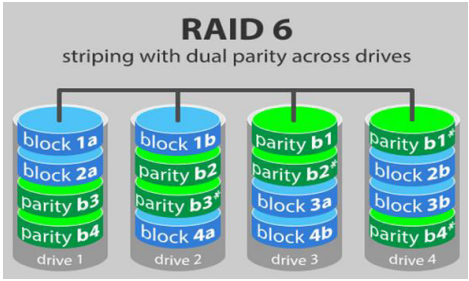
(f) RAID 5: block-interleaved distributed parity.

- Consists of block-level striping with distributed parity.
  - Unlike RAID 4, parity information is distributed among the drives
- Drastically improves the performance of multiple writes
  - they can now be done in parallel
- Slightly improves reads
  - one more disk to use for reading
- read and write performance close to that of RAID Level-1
- requires as much disk space as Levels-3,4

OPERATING SYSTEMS
RAID Level-6

- RAID 6 is like RAID 5, but the parity data are written to two drives.
  - That means it requires at least 4 drives and can withstand 2 drives dying simultaneously.
- Write data transactions are slower than RAID 5 due to the additional parity data that have to be calculated.

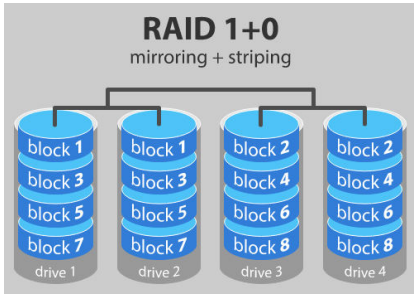
(g) RAID 6: P + Q redundancy.



OPERATING SYSTEMS

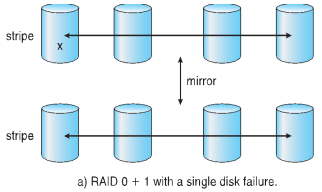
RAID level 10 - combining RAID 1 and RAID 0

- ❑ Provides security by mirroring all data on secondary drives while using striping across each set of drives to speed up data transfers.
- ❑ Rebuild time is very fast
- ❑ Half of the storage capacity goes to mirroring
  - ➢ so compared to large RAID 5 or RAID 6 arrays, this is an expensive way to have redundancy.



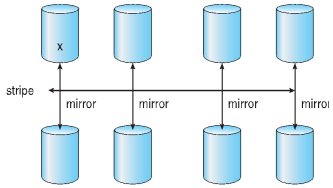
OPERATING SYSTEMS

RAID (0 + 1) and (1 + 0)



a) RAID 0 + 1 with a single disk failure.

Disks are striped and then the stripe is mirrored to another, equivalent stripe. If a single disk fails, an entire stripe is inaccessible.



b) RAID 1 + 0 with a single disk failure.

Disks are mirrored in pairs and then the resulting mirrored pairs are striped. This is better than 0+1 when a single disk fails

OPERATING SYSTEMS

Selecting a RAID level

- One consideration is rebuild performance.
  - If a disk fails, the time needed to rebuild its data can be significant.
  - This may be an important factor if a continuous supply of data is required, as it is in high-performance or interactive database systems.
  - Furthermore, rebuild performance influences the mean time to failure.
  - Rebuild performance varies with the RAID level used.
    - Rebuilding is easiest for RAID level 1, since data can be copied from another disk.

OPERATING SYSTEMS

Selecting a RAID level (Cont.)

    - For the other levels, we need to access all the other disks in the array to rebuild data in a failed disk.
    - Rebuild times can be hours for RAID 5 rebuilds of large disk sets.
  - RAID level 0 is used in high-performance applications where data loss is not critical.
  - RAID level 1 is popular for applications that require high reliability with fast recovery.
  - RAID 0 + 1 and 1 + 0 are used where both performance and reliability are important—for example, for small databases.
  - Due to RAID 1's high space overhead, RAID 5 is often preferred for storing large volumes of data.

Selecting a RAID level (Cont.)

- RAID system designers and administrators of storage have to make several other decisions as well.
  - How many disks should be in a given RAID set?
  - How many bits should be protected by each parity bit?
  - If more disks are in an array, data-transfer rates are higher, but the system is more expensive.
  - If more bits are protected by a parity bit, the space overhead due to parity bits is lower, but the chance that a second disk will fail before the first failed disk is repaired is greater, and that will result in data loss.

OPERATING SYSTEMS

Protection

Suresh Jamadagni
Department of Computer Science

THANK YOU

Chandravva Hebbi
Department of Computer Science Engineering
chandravvahebbi@pes.edu

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Protection - Introduction

- Protection refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by a computer system.
- This mechanism must provide a means for specifying the controls to be imposed, together with a means of enforcement
- Protection was originally conceived as an adjunct to multiprogramming operating systems, so that untrustworthy users might safely share a common logical name space, such as a directory of files, or share a common physical name space, such as memory.
- Modern protection concepts have evolved to increase the reliability of any complex system that makes use of shared resources.

OPERATING SYSTEMS
Goals of protection

- Need to prevent mischievous, intentional violation of an access restriction by a user
- Need to ensure that each program component active in a system uses system resources only in ways consistent with stated policies.
- A protection-oriented system should provides means to distinguish between authorized and unauthorized usage
- Improve reliability by detecting latent errors at the interfaces between component subsystems. Early detection of interface errors can often prevent contamination of a healthy subsystem by a malfunctioning subsystem.
- Provide a mechanism for the enforcement of the policies governing resource use. A protection system must have the flexibility to enforce a variety of policies.

OPERATING SYSTEMS
Principles of protection

- A key, time-tested guiding principle for protection is the **principle of least privilege**. It dictates that programs, users, and even systems be given just enough privileges to perform their tasks.
- An operating system following the principle of least privilege implements its features, programs, system calls, and data structures so that failure or compromise of a component does the minimum damage and allows the minimum damage to be done.
- An operating system should provide system calls and services that allow applications to be written with fine-grained access controls. It should provide mechanisms to enable privileges when they are needed and to disable them when they are not needed.

OPERATING SYSTEMS
Principles of protection

- Also beneficial is the creation of audit trails for all privileged function access.
- The audit trail allows the programmer, system administrator, or law-enforcement officer to trace all protection and security activities on the system
- Managing users with the principle of least privilege entails creating a separate account for each user, with just the privileges that the user needs.
- Some systems implement role-based access control (RBAC) to provide this functionality
- Computers implemented in a computing facility under the principle of least privilege can be limited to running specific services, accessing specific remote hosts via specific services, and doing so during specific times

OPERATING SYSTEMS
Domain of protection

- A computer system is a collection of processes and objects.
- ***Objects*** include both **hardware objects** (such as the CPU, memory segments, printers, disks, and tape drives) and **software objects** (such as files, programs, and semaphores).
- Each object has a unique name that differentiates it from all other objects in the system, and each can be accessed only through well-defined and meaningful operations.
- Objects are essentially abstract data types.
- The operations that are possible may depend on the object.
- Processes should be allowed to access only those resources for which it has authorization.
- At any time, a process should be able to access only those resources that it currently requires to complete its task. This requirement, commonly referred to as the **need-to-know principle**, is useful in limiting the amount of damage a faulty process can cause in the system

OPERATING SYSTEMS
Domain structure

- A process operates within a **protection domain**, which specifies the resources that the process may access.
- Each domain defines a set of objects and the types of operations that may be invoked on each object.
- The ability to execute an operation on an object is an **access right**.
- A domain is a collection of access rights, each of which is an ordered pair **<object-name, rights-set>**.
- For example, if domain *D* has the access right <file *F*, *{*read,write*}*>, then a process executing in domain *D* can both read and write file *F*. It cannot, however, perform any other operation on that object.
- Domains may share access rights

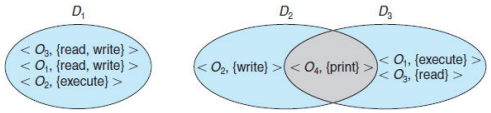
OPERATING SYSTEMS
Domain structure

- The association between a process and a domain may be either **static**, if the set of resources available to the process is fixed throughout the process's lifetime, or **dynamic**.
- Establishing dynamic protection domains is more complicated than establishing static protection domains.
- If the association between processes and domains is fixed, and we want to adhere to the need-to-know principle, then a mechanism must be available to change the content of a domain.
- The reason stems from the fact that a process may execute in two different phases and may, for example, need read access in one phase and write access in another.
- If a domain is static, we must define the domain to include both read and write access. However, this arrangement provides more rights than are needed in each of the two phases, since we have read access in the phase where we need only write access, and vice versa

OPERATING SYSTEMS
Domain structure

- Thus, the need-to-know principle is violated. We must allow the contents of a domain to be modified so that the domain always reflects the minimum necessary access rights.
- If the association is dynamic, a mechanism is available to allow **domain switching**, enabling the process to switch from one domain to another.
- We may also want to allow the content of a domain to be changed. If we cannot change the content of a domain, we can provide the same effect by creating a new domain with the changed content and switching to that new domain when we want to change the domain content

OPERATING SYSTEMS
Domain structure example



- We have three domains: *D*1, *D*2, and *D*3.
- The access right *<O*4, *{*print*}>* is shared by *D*2 and *D*3, implying that a process executing in either of these two domains can print object *O*4.
- A process must be executing in domain *D*1 to read and write object *O*1, while only processes in domain *D*3 may execute object *O*1

OPERATING SYSTEMS
Domain structure implementation

A domain can be realized in a variety of ways:

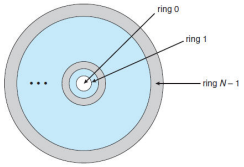
- Each ***user*** may be a domain. In this case, the set of objects that can be accessed depends on the identity of the user. Domain switching occurs when the user is changed—generally when one user logs out and another user logs in.
- Each ***process*** may be a domain. In this case, the set of objects that can be accessed depends on the identity of the process. Domain switching occurs when one process sends a message to another process and then waits for a response.
- Each ***procedure*** may be a domain. In this case, the set of objects that can be accessed corresponds to the local variables defined within the procedure. Domain switching occurs when a procedure call is made.

OPERATING SYSTEMS
Domain structure implementation - UNIX

- In the UNIX operating system, a domain is associated with the user.
- Switching the domain corresponds to changing the user identification temporarily.
- This change is accomplished through the file system as follows.
  - An owner identification and a domain bit (known as the **setuid bit**) are associated with each file.
  - When the setuid bit is on, and a user executes that file, the userID is set to that of the owner of the file. When the bit is off, however, the userID does not change.
  - For example, when a user *A* (that is, a user with userID = *A*) starts executing a file owned by *B,* whose associated domain bit is off, the userID of the process is set to *A.* When the setuid bit is on, the userID is set to that of the owner of the file: *B.*
  - When the process exits, this temporary userID change ends.

OPERATING SYSTEMS
Domain structure implementation - MULTICS

- In the MULTICS system, the protection domains are organized hierarchically into a ring structure. The rings are numbered from 0 to 7.
- MULTICS has a segmented address space; each segment is a file, and each segment is associated with one of the rings. A segment description includes an entry that identifies the ring number.
- Each ring corresponds to a single domain.
- Let *Di* and *Dj* be any two domain rings. If *j* < *i*, then *Di* is a subset of *Dj* . That is, a process executing in domain *Dj* has more privileges than does a process executing in domain *Di*. A process executing in domain D_0 has the most privileges



OPERATING SYSTEMS

Domain structure implementation - MULTICS

- A current-ring-number counter is associated with each process, identifying the ring in which the process is executing currently.
- When a process is executing in ring *i*, it cannot access a segment associated with ring *j* ($j < i$). It can access a segment associated with ring *k* ($k \geq i$).
- Domain switching in MULTICS occurs when a process crosses from one ring to another by calling a procedure in a different ring
- The main disadvantage of the ring (or hierarchical) structure is that it does not allow us to enforce the need-to-know principle.
- In particular, if an object must be accessible in domain *Dj* but not accessible in domain *Di* , then we must have $j < i$. But this requirement means that every segment accessible in *Di* is also accessible in *Dj* .

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Access Matrix

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Access Matrix

- A general model of protection can be viewed abstractly as a matrix, called an **access matrix**.
- The rows of the access matrix represent domains, and thecolumns represent objects. Each entry in the matrix consists of a set of access rights.
- Because the column defines objects explicitly, we can omit the object name from the access right.
- The entry access(*i,j*) defines the set of operations that a process executing in domain *Di* can invoke on object *Oj* .

OPERATING SYSTEMS
Access Matrix example

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

- There are four domains and four objects - three files (*F*1, *F*2, *F*3) and one laser printer.
- A process executing in domain *D*1 can read files *F*1 and *F*3.
- A process executing in domain *D*4 has the same privileges as one executing in domain *D*1; but in addition, it can also write onto files *F*1 and *F*3.
- The laser printer can be accessed only by a process executing in domain *D*2.

OPERATING SYSTEMS
Access Matrix

- The access-matrix scheme provides us with the mechanism for specifying a variety of policies.
- The mechanism consists of implementing the access matrix and ensuring that the semantic properties we have outlined hold.
- More specifically, we must ensure that a process executing in domain *Di* can access only those objects specified in row *i*, and then only as allowed by the access-matrix entries.
- The access matrix can implement policy decisions concerning protection.
- The policy decisions involve which rights should be included in the (*i, j*)*th* entry.
- We must also decide the domain in which each process executes. This last policy is usually decided by the operating system.

OPERATING SYSTEMS
Access Matrix – Domain switching

- The access matrix provides an appropriate mechanism for defining and implementing strict control for both static and dynamic association between processes and domains.
- When we switch a process from one domain to another, we are executing an operation (switch) on an object (the domain).
- We can control domain switching by including domains among the objects of the access matrix.
- Similarly, when we change the content of the access matrix, we are performing an operation on an object: the access matrix.
- Again, we can control these changes by including the access matrix itself as an object
- Processes can switch from domain *Di* to domain *Dj* if and only if the access right switch ∈ access(*i*, *j*)

OPERATING SYSTEMS
Access Matrix – Domain switching

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

- A process executing in domain *D*2 can switch to domain *D*3 or to domain *D*4.
- A process in domain *D*4 can switch to *D*1, and one in domain *D*1 can switch to *D*2.
- Allowing controlled change in the contents of the access-matrix entries requires three additional operations: copy, owner, and control
- The ability to copy an access right from one domain (or row) of the access matrix to another is denoted by an asterisk (*) appended to the access right.
- The copy right allows the access right to be copied only within the column (that is, for the object) for which the right is defined

OPERATING SYSTEMS
Implementation of the Access Matrix – Global table

- The simplest implementation of the access matrix is a global table consisting of a set of ordered triples <domain, object, rights-set>.
- Whenever an operation *M* is executed on an object *Oj* within domain *Di* , the global table is searched for a triple *<Di , Oj , Rk>*, with $M \in Rk$.
- If this triple is found, the operation is allowed to continue; otherwise, an exception (or error) condition is raised.
- This implementation suffers from several drawbacks. The table is usually large and thus cannot be kept in main memory, so additional I/O is needed.
- Virtual memory techniques are often used for managing this table.
- In addition, it is difficult to take advantage of special groupings of objects or domains.
- For example, if everyone can read a particular object, this object must have a separate entry in every domain.

OPERATING SYSTEMS
Implementation of the Access Matrix – Access list for objects

- Each column in the access matrix can be implemented as an access list for one object.
- Obviously, the empty entries can be discarded.
- The resulting list for each object consists of ordered pairs <domain, rights-set>, which define all domains with a nonempty set of access rights for that object.
- This approach can be extended easily to define a list plus a ***default*** set of access rights.
- When an operation *M* on an object *Oj* is attempted in domain *Di* , we search the access list for object *Oj* , looking for an entry *<Di , Rk>* with $M \in Rk$.
- If the entry is found, we allow the operation; if it is not, we check the default set.
- If *M* is in the default set, we allow the access. Otherwise, access is denied, and an exception condition occurs

OPERATING SYSTEMS
Implementation of the Access Matrix – capability list for domains

- Rather than associating the columns of the access matrix with the objects as access lists, we can associate each row with its domain.
- A **capability list** for a domain is a list of objects together with the operations allowed on those objects.
- An object is often represented by its physical name or address, called a **capability**.
- To execute operation *M* on object *Oj* , the process executes the operation *M*, specifying the capability (or pointer) for object *Oj* as a parameter.
- Simple **possession** of the capability means that access is allowed.
- The capability list is associated with a domain, but it is never directly accessible to a process executing in that domain. Rather, the capability list is itself a protected object, maintained by the operating system and accessed by the user only indirectly.

OPERATING SYSTEMS
Implementation of the Access Matrix – capability list for domains

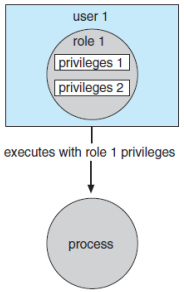
- Capability-based protection relies on the fact that the capabilities are never allowed to migrate into any address space directly accessible by a user process (where they could be modified).
- If all capabilities are secure, the object they protect is also secure against unauthorized access.

OPERATING SYSTEMS
Implementation of the Access Matrix – lock-key mechanism

- The **lock–key scheme** is a compromise between access lists and capability lists.
- Each object has a list of unique bit patterns, called **locks**.
- Similarly, each domain has a list of unique bit patterns, called **keys**.
- A process executing in a domain can access an object only if that domain has a key that matches one of the locks of the object.
- As with capability lists, the list of keys for a domain must be managed by the operating system on behalf of the domain.
- Users are not allowed to examine or modify the list of keys (or locks) directly.

OPERATING SYSTEMS
Role based access control

- A privilege is the right to execute a system call or to use an option within that system call (such as opening a file with write access).
- Privileges can be assigned to processes, limiting them to exactly the access they need to perform their work.
- Privileges and programs can also be assigned to **roles**.
- Users are assigned roles or can take roles based on passwords to the roles.
- A user can take a role that enables a privilege, allowing the user to run a program to accomplish a specific task.
- This implementation of privileges decreases the security risk associated with superusers and setuid programs.



OPERATING SYSTEMS
Revocation of access rights

In a dynamic protection system, we may need to revoke access rights to objects shared by different users

- **Immediate versus delayed**. Does revocation occur immediately or later?
- **Selective versus general**. When an access right to an object is revoked, does it affect all the users who have an access right to that object, or can we specify a select group of users whose access rights should be revoked?
- **Partial versus total**. Can a subset of the rights associated with an object be revoked, or must we revoke all access rights for this object?
- **Temporary versus permanent**. Can access be revoked permanently (that is, the revoked access right will never again be available), or can access be revoked and later be obtained again?

Revocation of access rights

- With an access-list scheme, revocation is easy.
- The access list is searched for any access rights to be revoked, and they are deleted from the list. Revocation is immediate and can be general or selective, total or partial, and permanent or temporary.
- Capabilities, however, present a much more difficult revocation problem.
- Since the capabilities are distributed throughout the system, we must find them before we can revoke them.
- Schemes that implement revocation for capabilities include the following:
  - **Reacquisition**. Periodically, capabilities are deleted from each domain. If a process wants to use a capability, it may find that that capability has been deleted. The process may then try to reacquire the capability. If access has been revoked, the process will not be able to reacquire the capability

OPERATING SYSTEMS

Revocation of access rights

- Schemes that implement revocation for capabilities include the following:
  - **Reacquisition**. Periodically, capabilities are deleted from each domain. If a process wants to use a capability, it may find that that capability has been deleted. The process may then try to reacquire the capability. If access has been revoked, the process will not be able to reacquire the capability
  - **Back-pointers**. A list of pointers is maintained with each object, pointing to all capabilities associated with that object. When revocation is required, we can follow these pointers, changing the capabilities as necessary. Implementation of this scheme is costly.
  - **Indirection**. The capabilities point indirectly, not directly, to the objects. Each capability points to a unique entry in a global table, which in turn points to the object. We implement revocation by searching the global table for the desired entry and deleting it

OPERATING SYSTEMS

Revocation of access rights

Schemes that implement revocation for capabilities include the following:

- **Keys**. A key is a unique bit pattern that can be associated with a capability. This key is defined when the capability is created, and it can be neither modified nor inspected by the process that owns the capability. A **master key** is associated with each object. Revocation replaces the master key with a new value via the set-key operation, invalidating all previous capabilities for this object

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

Security

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

- **Security**, requires not only an adequate protection system but also consideration of the ***external*** environment within which the system operates.
- Computer resources must be guarded against unauthorized access, malicious destruction or alteration, and accidental introduction of inconsistency.
- These resources include information stored in the system (both data and code), as well as the CPU, memory, disks, tapes, and networking that are the computer.
- In many applications, ensuring the security of the computer system is worth considerable effort
- We say that a system is **secure** if its resources are used and accessed as intended under all circumstances

OPERATING SYSTEMS
Security

- Security violations (or misuse) of the system can be categorized as intentional (malicious) or accidental.
- It is easier to protect against accidental misuse than against malicious misuse
- A **threat** is the potential for a security violation, such as the discovery of a vulnerability, whereas an **attack** is the attempt to break security.

- **Breach of confidentiality**. This type of violation involves unauthorized reading of data (or theft of information). Typically, a breach of confidentiality is the goal of an intruder
- **Breach of integrity**. This violation involves unauthorized modification of data.
- **Breach of availability**. This violation involves unauthorized destruction of data.
- **Theft of service**. This violation involves unauthorized use of resources.
- **Denial of service**. This violation involves preventing legitimate use of the system.

- Attackers use several standard methods in their attempts to breach security. The most common is **masquerading**, in which one participant in a communication pretends to be someone else (another host or another person).
- By masquerading, attackers breach **authentication**, the correctness of identification; they can then gain access that they would not normally be allowed or escalate their privileges—obtain privileges to which they would not normally be entitled.
- A **replay attack** consists of the malicious or fraudulent repeat of a valid data transmission. Sometimes the replay comprises the entire attack. But frequently it is done along with **message modification**, again to escalate privileges

- **man-in-the-middle attack**, in which an attacker sits in the data flow of a communication, masquerading as the sender to the receiver, and vice versa.
- In a network communication, a man-in-the-middle attack may be preceded by a **session hijacking**, in which an active communication session is intercepted.
- Absolute protection of the system from malicious abuse is not possible, but the cost to the perpetrator can be made sufficiently high to deter most intruders

To protect a system, we must take security measures at four levels:

1. **Physical** - The site or sites containing the computer systems must be physically secured against armed or surreptitious entry by intruders
2. **Human** - Authorization must be done carefully to assure that only appropriate users have access to the system
3. **Operating system** - The system must protect itself from accidental or purposeful security breaches.
4. **Network** - Much computer data in modern systems travels over private leased lines, shared lines like the Internet, wireless connections, or dial-up lines. Intercepting these data could be just as harmful as breaking into a computer, and interruption of communications could result in diminishing users' trust in the system

- Processes, along with the kernel, are the only means of accomplishing work on a computer.
- Therefore, writing a program that creates a breach of security, or causing a normal process to change its behavior and create a breach, is a common goal of attackers

Trojan Horse

- A code segment that misuses its environment is called a **Trojan horse**
- Many systems have mechanisms for allowing programs written by users to be executed by other users. If these programs are executed in a domain that provides the access rights of the executing user, the other users may misuse these rights

Trojan Horse

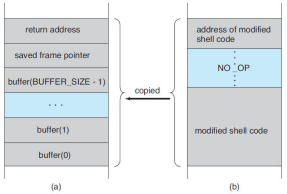
- A variation of the Trojan horse is a program that emulates a login program. An unsuspecting user starts to log in at a terminal and notices that he has apparently mistyped his password. He tries again and is successful. What has happened is that his authentication key and password have been stolen by the login emulator, which was left running on the terminal by the thief.
- Another variation on the Trojan horse is **spyware**. Spyware sometimes accompanies a program that the user has chosen to install. Most frequently, it comes along with freeware or shareware programs, but sometimes it is included with commercial software.
- The goal of spyware is to or capture information from the user's system and return it to a central Site
- Spyware is a micro example of a macro problem: violation of the principle of least privilege

Trap Door

- The designer of a program or system might leave a hole in the software that only he/she is capable of using
- For instance, the code might check for a specific user ID or password, and it might circumvent normal security procedures.
- A clever trap door could be included in a compiler. The compiler could generate standard object code as well as a trap door, regardless of the source code being compiled
- Trap doors pose a difficult problem because, to detect them, we have to analyze all the source code for all components of a system

Stack and Buffer overflow

- The stack- or buffer-overflow attack is the most common way for an attacker outside the system, on a network or dial-up connection, to gain unauthorized access to the target system.
- The attack exploits a bug in a program. The bug can be a simple case of poor programming, in which the programmer neglected to code bounds checking on an input field. In this case, the attacker sends more data than the program was expecting.



Hypothetical stack frame for (a) before and (b) after.

OPERATING SYSTEMS
Program Threats

- **Viruses**
- A virus is a fragment of code embedded in a legitimate program.
- Viruses are self-replicating and are designed to “infect” other programs.
- They can wreak havoc in a system by modifying or destroying files and causing system crashes and program malfunctions.
- As with most penetration attacks, viruses are very specific to architectures, operating systems, and applications.
- Once a virus reaches a target machine, a program known as a **virus dropper** inserts the virus into the system.
- The virus dropper is usually a Trojan horse, executed for other reasons but installing the virus as its core activity. Once installed, the virus may do any one of a number of things.

OPERATING SYSTEMS
System Threats

- System and network threats involve the abuse of services and network connections.
- System and network threats create a situation in which operating-system resources and user files are misused.
- Sometimes, a system and network attack is used to launch a program attack, and vice versa.
- The more ***open*** an operating system is—the more services it has enabled and the more functions it allows—the more likely it is that a bug is available to exploit

Worms

- A **worm** is a process that uses the **spawn** mechanism to duplicate itself.
- The worm spawns copies of itself, using up system resources and perhaps locking out all other processes

OPERATING SYSTEMS
Defenses against system attacks

- The broadest tool available to system designers and users to thwart attacks is **cryptography**.
- In an isolated computer, the operating system can reliably determine the sender and recipient of all inter-process communication, since it controls all communication channels in the computer.
- In a network of computers, the situation is quite different. A networked computer receives bits “from the wire” with no immediate and reliable way of determining what machine or application sent those bits.
- **Cryptography** is used to constrain the potential senders and/or receivers of a message.
- Modern cryptography is based on secrets called **keys** that are selectively distributed to computers in a network and used to process messages.
- Cryptography enables a recipient of a message to verify that the message was created by some computer possessing a certain key. Similarly, a sender can encode its message so that only a computer with a certain key can decode the message

OPERATING SYSTEMS
Defenses against system attacks

- The broadest tool available to system designers and users to thwart attacks is **cryptography**.
- In an isolated computer, the operating system can reliably determine the sender and recipient of all inter-process communication, since it controls all communication channels in the computer.
- In a network of computers, the situation is quite different. A networked computer receives bits “from the wire” with no immediate and reliable way of determining what machine or application sent those bits.
- **Cryptography** is used to constrain the potential senders and/or receivers of a message.
- Modern cryptography is based on secrets called **keys** that are selectively distributed to computers in a network and used to process messages.
- Cryptography enables a recipient of a message to verify that the message was created by some computer possessing a certain key. Similarly, a sender can encode its message so that only a computer with a certain key can decode the message

- Constraining the set of potential senders of a message is called **authentication**.
- Authentication is thus complementary to encryption
- Authentication is also useful for proving that a message has not been modified

SSL 3.0

- SSL 3.0 is a cryptographic protocol that enables two computers to communicate securely—that is, so that each can limit the sender and receiver of messages to the other.
- It is perhaps the most commonly used cryptographic protocol on the Internet today, since it is the standard protocol by which web browsers communicate securely with web servers.
- The SSL protocol is initiated by a client *c* to communicate securely with a server.
- Asymmetric cryptography is used so that a client and a server can establish a secure **session key** that can be used for symmetric encryption of the session between the two

User authentication

- A major security problem for operating systems is **user authentication**.
- The protection system depends on the ability to identify the programs and processes currently executing, which in turn depends on the ability to identify each user of the system
- The most common approach to authenticating a user identity is the use of **passwords**.

Securing passwords

- The UNIX system uses secure hashing to avoid the necessity of keeping its password list secret.
- Because the list is hashed rather than encrypted, it is impossible for the system to decrypt the stored value and determine the original password
- Some systems do not allow the use of dictionary words as passwords
- Multi-factor authentication requires users to provide more than one piece of information to authenticate successfully to an account or Linux host. The additional information may be a one-time password (OTP)

ssh

- SSH, or Secure Shell, constitutes a cryptographic network protocol designed to enable secure communication between two systems over networks that may not be secure.
- This protocol is widely employed for remote access to servers and the secure transmission of files between computers. In essence, SSH acts as a secure conduit, establishing a confidential channel for communication in scenarios where the network may pose security risks
- This technology is instrumental for professionals seeking a reliable and secure method of managing servers and transferring sensitive data across computers in a controlled and protected manner.
- ssh runs at TCP/IP port 22.
- https://www.geeksforgeeks.org/ssh-command-in-linux-with-examples/

Security policy

- The first step toward improving the security of any aspect of computing is to have a **security policy**.
- Policies vary widely but generally include a statement of what is being secured
- Without a policy in place, it is impossible for users and administrators to know what is permissible, what is required, and what is not allowed

Vulnerability Assessment

- The core activity of most vulnerability assessments is a **penetration test**, in which the entity is scanned for known vulnerabilities.
- Vulnerability scans typically focus on short or easy-to-guess passwords, unauthorized privileged programs such as setuid programs, unauthorized programs in system directories, Improper directory protections on user and system directories and changes to system programs detected with checksum values

Intrusion detection

- **Intrusion detection**, as its name suggests, strives to detect attempted or successful intrusions into computer systems and to initiate appropriate responses to the intrusions
- Intrusion Detection (IDS) systems raise an alarm when an intrusion is detected, while Intrusion Prevention (IDP) systems act as routers, passing traffic unless an intrusion is detected at which point that traffic is blocked
- **Anomaly detection**, attempts through various techniques to detect anomalous behavior within computer systems. Of course, not all anomalous system activity indicates an intrusion, but the presumption is that intrusions often induce anomalous behavior
- An example of an anomaly-detection tool is the **Tripwire file system** integrity checking tool for UNIX, developed at Purdue University. Tripwire operates on the premise that many intrusions result in modification of system directories and files.

Virus protection

- The best protection against computer viruses is prevention, or the practice of **safe computing**.
- Antivirus programs are often used to provide this protection.
- Some of these programs are effective against only particular known viruses. They work by searching all the programs on a system for the specific pattern of instructions known to make up the virus.
- When they find a known pattern, they remove the instructions, **disinfecting** the program.
- Antivirus programs may have catalogs of thousands of viruses for which they search

Auditing, Accounting, and Logging

- Auditing, accounting, and logging can decrease system performance, but they are useful in several areas, including security.
- Logging can be general or specific. All system-call executions can be logged for analysis of program behavior (or misbehavior). More typically, suspicious events are logged.
- Authentication failures and authorization failures can tell us quite a lot about break-in attempts.
- Accounting can be used to find performance changes, which in turn can reveal security problems.

Firewalling to Protect Systems and Networks

- A **firewall** is a computer, appliance, or router that sits between the trusted and the untrusted.
- A network firewall limits network access between the two **security domains** and monitors and logs all connections.
- It can also limit connections based on source or destination address, source or destination port, or direction of the connection.
- For instance, web servers use HTTP to communicate with web browsers. A firewall therefore may allow only HTTP to pass from all hosts outside the firewall to the web server within the firewall.
- Of course, a firewall itself must be secure and attack-proof. Otherwise, its ability to secure connections can be compromised
- **System-call firewalls** sit between applications and the kernel, monitoring system-call execution.

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

Storage Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Storage Management – System calls

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
File access permission

- There are nine permission bits for each file, divided into three categories
- The term user in the first three rows refers to the owner of the file
- The first rule is that to open any type of file by name, user must have execute permission in each directory mentioned in the name, including the current directory.
- The execute permission bit for a directory is often called the search bit
- For example, to open the file /usr/include/stdio.h, user would need execute permission in the directory /, execute permission in the directory /usr, and execute permission in the directory /usr/include and appropriate permission for the file stdio.h

st_mode mask	Meaning
S_IRUSR	user-read
S_IWUSR	user-write
S_IXUSR	user-execute
S_IRGRP	group-read
S_IWGRP	group-write
S_IXGRP	group-execute
S_IROTH	other-read
S_IWOTH	other-write
S_IXOTH	other-execute

OPERATING SYSTEMS
File access permission

- The read permission for a file determines whether user can open an existing file for reading: the O_RDONLY and O_RDWR flags for the open function.
- The write permission for a file determines whether user can open an existing file for writing: the O_WRONLY and O_RDWR flags for the open function.
- User must have write permission for a file to specify the O_TRUNC flag in the open function.
- User cannot create a new file in a directory unless they have write permission and execute permission in the directory.
- To delete an existing file, user needs write permission and execute permission in the directory containing the file.
- Users do not need read permission or write permission for the file itself

OPERATING SYSTEMS
Set-User-ID and Set-Group-ID

real user ID real group ID	who we really are
effective user ID effective group ID supplementary group IDs	used for file access permission checks
saved set-user-ID saved set-group-ID	saved by exec functions

- Every process has six or more IDs associated with it
- The real user ID and real group ID identify who the user really is. These two fields are taken from an entry in the password file when the user logs in
- The effective user ID, effective group ID, and supplementary group IDs determine file access permissions for the user
- The saved set-user-ID and saved set-group-ID contain copies of the effective user ID and the effective group ID, respectively, when a program is executed.

OPERATING SYSTEMS
Set-User-ID and Set-Group-ID

```
#include <unistd.h>
int setuid(uid_t uid);
int setgid(gid_t gid);
```
Both return: 0 if OK, −1 on error

- We can set the real user ID and effective user ID with the setuid function.
- We can set the real group ID and the effective group ID with the setgid function.
- If the process has superuser privileges, the setuid function sets the real user ID, effective user ID, and saved set-user-ID to uid.
- If the process does not have superuser privileges, but uid equals either the real user ID or the saved set-user-ID, setuid sets only the effective user ID to uid. The real user ID and the saved set-user-ID are not changed.
- If neither of these two conditions is true, errno is set to EPERM and −1 is returned

OPERATING SYSTEMS
access and faccessat

```
#include <unistd.h>
int access(const char *pathname, int mode);
int faccessat(int fd, const char *pathname, int mode, int flag);
                                        Both return: 0 if OK, –1 on error
```

mode	Description
R_OK	test for read permission
W_OK	test for write permission
X_OK	test for execute permission

- When a user tries to open a file, the kernel performs access tests based on the effective user and group IDs
- Sometimes, however, a process wants to test accessibility based on the real user and group IDs. This is useful when a process is running as someone else, using either the set-user-ID or the set-group-ID feature.
- The **access** and **faccessat** functions base their tests on the real user and group IDs.

OPERATING SYSTEMS
umask

```
#include <sys/stat.h>
mode_t umask(mode_t cmask);
                        Returns: previous file mode creation mask
```

- The umask function sets the file mode creation mask for the process and returns the previous value
- The cmask argument is formed as the bitwise OR of any of the nine constants
- The file mode creation mask is used whenever the process creates a new file or a new directory.

st_mode mask	Meaning
S_IRUSR	user-read
S_IWUSR	user-write
S_IXUSR	user-execute
S_IRGRP	group-read
S_IWGRP	group-write
S_IXGRP	group-execute
S_IROTH	other-read
S_IWOTH	other-write
S_IXOTH	other-execute

OPERATING SYSTEMS
chmod, fchmod, and fchmodat

```
#include <sys/stat.h>
int chmod(const char *pathname, mode_t mode);
int fchmod(int fd, mode_t mode);
int fchmodat(int fd, const char *pathname, mode_t mode, int flag);
                                        All three return: 0 if OK, –1 on error
```

- The chmod, fchmod, and fchmodat functions allow us to change the file access permissions for an existing file
- The chmod function operates on the specified file, whereas the fchmod function operates on a file that has already been opened.
- The fchmodat function behaves like chmod when the pathname argument is absolute or when the fd argument has the value AT_FDCWD and the pathname argument is relative
- To change the permission bits of a file, the effective user ID of the process must be equal to the owner ID of the file, or the process must have superuser permissions

OPERATING SYSTEMS
chown, fchown, fchownat, and lchown

```
#include <unistd.h>
int chown(const char *pathname, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
int fchownat(int fd, const char *pathname, uid_t owner, gid_t group,
             int flag);
int lchown(const char *pathname, uid_t owner, gid_t group);
                                        All four return: 0 if OK, –1 on error
```

- The chown functions allow users to change a file's user ID and group ID, but if either of the arguments owner or group is −1, the corresponding ID is left unchanged
- The fchown function changes the ownership of the open file referenced by the fd argument.
- lchown and fchownat (with the AT_SYMLINK_NOFOLLOW flag set) change the owners of the symbolic link itself, not the file pointed to by the symbolic link

File truncation

```
#include <unistd.h>
int truncate(const char *pathname, off_t length);
int ftruncate(int fd, off_t length);
                                    Both return: 0 if OK, –1 on error
```

- **truncate** and **ftruncate** functions truncate an existing file to *length* bytes.
- If the previous size of the file was greater than *length*, the data beyond *length* is no longer accessible.
- If the previous size was less than *length*, the file size will increase and the data between the old end of file and the new end of file will read as 0 (i.e., a hole is probably created in the file)

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Files and Directories – System calls

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Properties of a File

```
#include <sys/stat.h>
int stat(const char *restrict pathname, struct stat *restrict buf);
int fstat(int fd, struct stat *buf);
int lstat(const char *restrict pathname, struct stat *restrict buf);
int fstatat(int fd, const char *restrict pathname,
            struct stat *restrict buf, int flag);
                                    All four return: 0 if OK, –1 on error
```

- **stat** function returns a structure of information about the named file
- The **fstat** function obtains information about the file that is already open on the descriptor fd.
- The **lstat** function returns information about the symbolic link, not the file referenced by the symbolic link
- The **fstatat** function returns the file statistics for a pathname relative to an open directory represented by the fd argument.

OPERATING SYSTEMS
Structure that represents properties of a File

```
struct stat {
  mode_t          st_mode;    /* file type & mode (permissions) */
  ino_t           st_ino;     /* i-node number (serial number) */
  dev_t           st_dev;     /* device number (file system) */
  dev_t           st_rdev;    /* device number for special files */
  nlink_t         st_nlink;   /* number of links */
  uid_t           st_uid;     /* user ID of owner */
  gid_t           st_gid;     /* group ID of owner */
  off_t           st_size;    /* size in bytes, for regular files */
  struct timespec st_atim;    /* time of last access */
  struct timespec st_mtim;    /* time of last modification */
  struct timespec st_ctim;    /* time of last file status change */
  blksize_t       st_blksize; /* best I/O block size */
  blkcnt_t        st_blocks;  /* number of disk blocks allocated */
};
```

The timespec structure type defines time in terms of seconds and nanoseconds.

It includes at least the following fields:

time_t tv_sec;

long tv_nsec;

OPERATING SYSTEMS
File type macros

File type macros in <sys/stat.h>

Macro	Type of file
S_ISREG()	regular file
S_ISDIR()	directory file
S_ISCHR()	character special file
S_ISBLK()	block special file
S_ISFIFO()	pipe or FIFO
S_ISLNK()	symbolic link
S_ISSOCK()	socket

- Regular file - The most common type of file, which contains data of some form.
- Directory file - A file that contains the names of other files and pointers to information on these files
- Block special file - A type of file providing buffered I/O access in fixed-size units to devices such as disk drives
- Character special file - A type of file providing unbuffered I/O access in variable-sized units to devices.
- FIFO - A type of file used for communication between processes. It's sometimes called a named pipe
- Socket - A type of file used for network communication between processes
- Symbolic link - A type of file that points to another file

OPERATING SYSTEMS
File operations - creat, open and close

```
#include <fcntl.h>
int creat(const char *path, mode_t mode);
                    Returns: file descriptor opened for write-only if OK, –1 on error
```

- A new file can be created by calling the **creat** function.
- creat function is equivalent to **open** (path, O_WRONLY | O_CREAT | O_TRUNC, mode);

```
#include <fcntl.h>
int open(const char *path, int oflag, ... /* mode_t mode */ );
int openat(int fd, const char *path, int oflag, ... /* mode_t mode */ );
                    Both return: file descriptor if OK, –1 on error
```

- A file is opened or created by calling either the **open** function or the **openat** function

```
#include <unistd.h>
int close(int fd);
                    Returns: 0 if OK, –1 on error
```

- An open file is closed by calling the **close** function

OPERATING SYSTEMS
File operations - lseek

```
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence);
                    Returns: new file offset if OK, –1 on error
```

- Every open file has an associated ‘‘current file offset,’’ normally a non-negative integer that measures the number of bytes from the beginning of the file
- Read and write operations normally start at the current file offset and cause the offset to be incremented by the number of bytes read or written. By default, this offset is initialized to 0 when a file is opened, unless the O_APPEND option is specified.
- If whence is SEEK_SET, the file’s offset is set to offset bytes from the beginning of the file.
- If whence is SEEK_CUR, the file’s offset is set to its current value plus the offset. The offset can be positive or negative.
- If whence is SEEK_END, the file’s offset is set to the size of the file plus the offset. The offset can be positive or negative

OPERATING SYSTEMS
File operations - read

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t nbytes);
                    Returns: number of bytes read, 0 if end of file, –1 on error
```

- Data is read from an open file with the read function
- If the read is successful, the number of bytes read is returned. If the end of file is encountered, 0 is returned.
- The read operation starts at the file’s current offset. Before a successful return, the offset is incremented by the number of bytes actually read.

OPERATING SYSTEMS
File operations - write

```
#include <unistd.h>
ssize_t write(int fd, const void *buf, size_t nbytes);
                    Returns: number of bytes written if OK, –1 on error
```

- Data is written to an open file with the write function.
- The return value is usually equal to the nbytes argument; otherwise, an error has occurred
- A common cause for a write error is either filling up a disk or exceeding the file size limit for a given process
- For a regular file, the write operation starts at the file’s current offset.
- If the O_APPEND option was specified when the file was opened, the file’s offset is set to the current end of file before each write operation.
- After a successful write, the file’s offset is incremented by the number of bytes actually written

OPERATING SYSTEMS
Directories

```
#include <sys/stat.h>
int mkdir(const char *pathname, mode_t mode);
int mkdirat(int fd, const char *pathname, mode_t mode);
                                        Both return: 0 if OK, –1 on error
#include <unistd.h>
int rmdir(const char *pathname);
                                        Returns: 0 if OK, –1 on error
```

- Directories are created with the mkdir and mkdirat functions, and deleted with the rmdir function.
- The specified file access permissions, **mode**, are modified by the file mode creation mask of the process

OPERATING SYSTEMS
Reading Directories

```
#include <dirent.h>
DIR *opendir(const char *pathname);
DIR *fdopendir(int fd);
                                        Both return: pointer if OK, NULL on error
struct dirent *readdir(DIR *dp);
                                        Returns: pointer if OK, NULL at end of directory or error
void rewinddir(DIR *dp);
int closedir(DIR *dp);
                                        Returns: 0 if OK, –1 on error
long telldir(DIR *dp);
                                        Returns: current location in directory associated with dp
void seekdir(DIR *dp, long loc);
```

- Directories can be read by anyone who has access permission to read the directory. But only the kernel can write to a directory, to preserve file system sanity
- **fdopendir** provides a way to convert an open file descriptor into a DIR structure for use by the directory handling functions.
- The **dirent** structure defined in is implementation dependent.

```
ino_t  d_ino;          /* i-node number */
char   d_name[];       /* null-terminated filename */
```

OPERATING SYSTEMS
Hard links

```
#include <unistd.h>
int link(const char *existingpath, const char *newpath);
int linkat(int efd, const char *existingpath, int nfd, const char *newpath,
           int flag);
                                        Both return: 0 if OK, –1 on error
```

- **link** function or the **linkat** function can be used to create a link to an existing file.
- These functions create a new directory entry, newpath, that references the existing file existingpath.
- If the newpath already exists, an error is returned. Only the last component of the newpath is created. The rest of the path must already exist

```
#include <unistd.h>
int unlink(const char *pathname);
int unlinkat(int fd, const char *pathname, int flag);
                                        Both return: 0 if OK, –1 on error
```

- Use the **unlink** function to remove an existing link. If there are other links to the file, the data in the file is still accessible through the other links

OPERATING SYSTEMS
Symbolic links

```
#include <unistd.h>
int symlink(const char *actualpath, const char *sympath);
int symlinkat(const char *actualpath, int fd, const char *sympath);
                                        Both return: 0 if OK, –1 on error
```

- A symbolic link is created with either the symlink or symlinkat function
- A new directory entry, sympath, is created that points to actualpath. It is not required that actualpath exist when the symbolic link is created
- The symlinkat function is similar to symlink, but the sympath argument is evaluated relative to the directory referenced by the open file descriptor for that directory (specified by the fd argument)

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS

UE22CS242B

File Management

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

File-Systems – Case Study: Linux

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

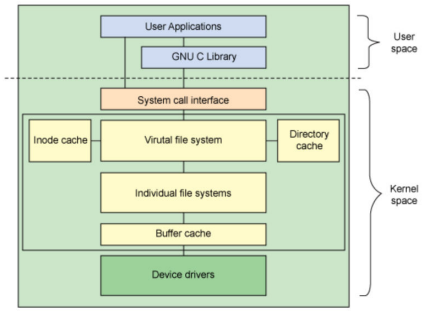
Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS
Linux File System - Architecture

- User space contains the applications which provides the user interface for the file system calls (open, read, write, close).
- The system call interface acts as a switch, funneling system calls from user space to the appropriate endpoints in kernel space.
- The VFS is the primary interface to the underlying file systems. This component exports a set of interfaces and then abstracts them to the individual file systems, which may behave very differently from one another.
- Two caches exist for file system objects (inodes and dentries). Each provides a pool of recently-used file system objects.



Reference: https://developer.ibm.com/tutorials/l-linux-filesystem/

OPERATING SYSTEMS
Linux File System - Architecture

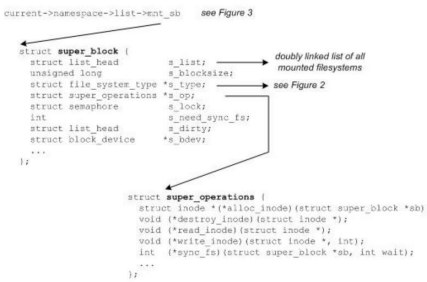
- Each individual file system implementation, such as ext2, JFS, and so on, exports a common set of interfaces that is used by the VFS.
- The buffer cache buffers requests between the file systems and the block devices that they manipulate. For example, read and write requests to the underlying device drivers migrate through the buffer cache. This allows the requests to be cached there for faster access (rather than going back out to the physical device).
- The buffer cache is managed as a set of least recently used (LRU) lists. You can use the sync command to flush the buffer cache out to the storage media (force all unwritten data out to the device drivers and, subsequently, to the storage device).

OPERATING SYSTEMS
Linux File System – Common set of objects

- Linux views all file systems from the perspective of a common set of objects.
- These objects are the **superblock, inode, dentry,** and **file**.
- At the root of each file system is the superblock, which describes and maintains state for the file system.
- Every object that is managed within a file system (file or directory) is represented in Linux as an inode.
- The inode contains all the metadata to manage objects in the file system (including the operations that are possible on it).
- Another set of structures, called dentries, is used to translate between names and inodes, for which a directory cache exists to keep the most-recently used around.
- The dentry also maintains relationships between directories and files for traversing file systems.
- A VFS file represents an open file (keeps state for the open file such as the write offset, and so on).

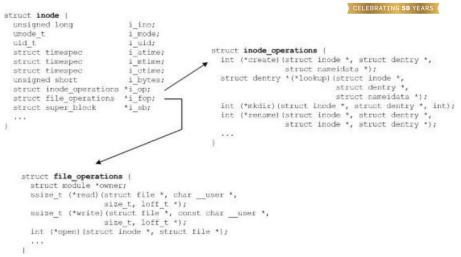
OPERATING SYSTEMS
Linux File System – Superblock

- The superblock is a structure that represents a file system.
- It includes the necessary information to manage the file system during operation.
- It includes the file system name (such as ext2), the size of the file system and its state, a reference to the block device, and metadata information (such as free lists and so on).
- The superblock is typically stored on the storage medium
- Superblock structure is available in ./linux/include/linux/fs.h.



OPERATING SYSTEMS
Linux File System – inode and dentry

- The inode represents an object in the file system with a unique identifier.
- The individual file systems provide methods for translating a filename into a unique inode identifier and then to an inode reference.
- inode_operations and file_operations structures refers to the individual operations that may be performed on the inode, file and directories



Reference: https://opensource.com/article/18/7/how-check-free-disk-space-linux

OPERATING SYSTEMS
Linux File System – Buffer Cache

- Buffer cache keeps track of read and write requests from the individual file system implementations and the physical devices (through the device drivers).
- For efficiency, Linux maintains a cache of the requests to avoid having to go back out to the physical device for all requests. Instead, the most-recently used buffers (pages) are cached here and can be quickly provided back to the individual file systems.
- Linux supports a wide range of file systems such as MINIX, MS-DOS, and ext2. Linux also supports the new journaling file systems such as ext3, JFS, and ReiserFS. Additionally, Linux supports cryptographic file systems such as CFS and virtual file system such as /proc.
- Linux also supports **Filesystem in USErspace** (**FUSE**) filesystem that lets non-privileged users create their own file systems without editing kernel code. This is achieved by running file system code in user space while the FUSE module provides only a bridge to the actual kernel interfaces

THANK YOU

Suresh Jamadagni
Department of Computer Science and Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

I/O Management, System Protection and Security and Case Study

Kakoli Bora
Department of Computer Science

OPERATING SYSTEMS

Case Study – Windows File System

Kakoli Bora
Department of Computer Science

OPERATING SYSTEMS
Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau
5. Internet source

OPERATING SYSTEMS
File System - Basics

- FAT file system is still used for portability on other systems such as cameras, flash memory and external disks
- FAT file system does not restrict file access to authorized users.
- NTFS uses ACLs to control access to individual files and supports encryption.
- NTFS supports data recovery, fault tolerance, very large files and file systems, journaling, file compression, etc
- A file in NTFS is not a simple byte stream, as in MS-DOS or UNIX, rather, it is a structured object consisting of attributes
- Attributes like file name, creation time, descriptor, ACLs, etc

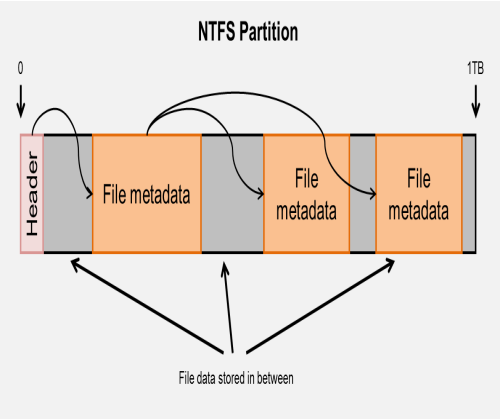
OPERATING SYSTEMS
File System – Basics (Cont.)

- The fundamental structure of Windows file system (NTFS) is a *volume*
  - Created by the disk administrator utility
  - Based on a logical disk partition
  - May occupy a portions of a disk, an entire disk, or span across several disks
- All *metadata*, such as information about the volume, is stored in a regular file
- NTFS uses ***clusters*** as the underlying unit of disk allocation
  - A cluster is a number of disk sectors that is a power of two
  - The default cluster size is based on the volume size - 4 KB for volumes > 2 GB
  - Because the cluster size is smaller than for the 16-bit FAT file system, the amount of internal fragmentation is reduced

OPERATING SYSTEMS
File System — NTFS vs FAT

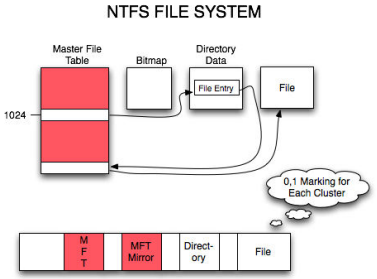
Features	NTFS	FAT32	FAT16	FAT12
Max Partition Size	2TB	32GB	4GB	16MB
Max File Size	16TB	4GB	2GB	Less than 16MB
Cluster Size	4KB	4KB to 32KB	2KB to 64KB	0.5KB to 4KB
Fault Tolerance	Auto Repair	No	No	No
Compression	Yes	No	No	No
Security	Local and Network	Only Network	Only Network	Only Network
Compatibility	Windows 10/8/7/XP/Vista/2000	Windows ME/2000/XP/7/8.1	Windows ME/2000/XP/7/8.1	Windows ME/2000/XP/7/8.1

OPERATING SYSTEMS
File System — NTFS (Cont.)



❖NTFS, the filesystem used by all modern versions of Windows
❖Instead of storing file metadata in tables scattered across the partition (usually to optimise hard disk seek times going from file metadata to actual file data), NTFS stores all the file metadata in a few large contiguous blocks.
❖These blocks are collectively known as the MFT (master file table).
❖Helps to quickly scan all the files in the partition to get each file's associated metadata. Instead of recursing through each directory manually and enumerating contents (like when computing summary statistics for directories)

OPERATING SYSTEMS
File System — NTFS (Cont.)



❖Every file in NTFS is described by one or more records in an array stored in a special file called the Master File Table (MFT)

❖Windows puts the Master File Table at the front of the drive, with a mirror file in the middle of the drive with only the first directory containing critical information.

❖The Bitmap is used to determine allocation/non-allocation of clusters.

OPERATING SYSTEMS
File System — Internal Layout

- NTFS uses logical cluster numbers (LCNs) as disk addresses
- Physical disk offset in bytes = LCN x cluster size
- A file in NTFS is not a simple byte stream, as in MS-DOS or UNIX, rather, it is a structured object consisting of attributes
- Each file on an NTFS volume has a unique ID called a file reference.
  - 64-bit quantity that consists of a 48-bit file number and a 16-bit sequence number
  - Sequence number is incremented every time an MFT entry is reused, can be used to perform internal consistency checks (to catch a stale reference to a deleted file)
- The NTFS name space is organized by a hierarchy of directories; the index root contains the top level of the B+ tree
  - B+ tree is an extension of the B tree

OPERATING SYSTEMS
File System—B+ Tree vs. B Tree

B+ Tree	B Tree
Data is only saved on the leaf nodes.	Both leaf nodes and internal nodes can store data
Data stored on the leaf node makes the search more accurate and faster.	Searching is slow due to data stored on Leaf and internal nodes.
Deletion is not difficult as an element is only removed from a leaf node.	Deletion of elements is a complicated and time-consuming process.
Linked leaf nodes make the search efficient and quick.	You cannot link leaf nodes.

The drawback of B-tree used for indexing, however is that it stores the data pointer (a pointer to the disk file block containing the key value), corresponding to a particular key value, along with that key value in the node of a B-tree. This technique, greatly reduces the number of entries that can be packed into a node of a B-tree, thereby contributing to the increase in the number of levels in the B-tree, hence increasing the search time of a record.
B+ tree eliminates the above drawback by storing data pointers only at the leaf nodes of the tree.

OPERATING SYSTEMS
File System - Recovery

- All file system data structure updates are performed inside transactions that are logged.
  - Before a data structure is altered, the transaction writes a log record that contains redo and undo information.
  - After the data structure has been changed, a commit record is written to the log to signify that the transaction succeeded.
  - After a crash, the file system data structures can be restored to a consistent state by processing the log records.

OPERATING SYSTEMS
File System – Recovery (Cont.)

- This scheme does not guarantee that all the user file data can be recovered after a crash, just that the file system data structures (the metadata files) are undamaged and reflect some consistent state prior to the crash.
- The log is stored in the third metadata file at the beginning of the volume.
- The logging functionality is provided by the *log-file service which keeps track of free space in the log file*
  - *halts new I/O operations in case free space gets too low*

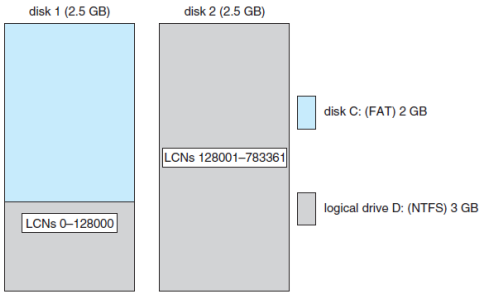
OPERATING SYSTEMS
File System - Security

- Security of an NTFS volume is derived from the Windows object model.
- Each file object has a security descriptor attribute stored in this MFT record.
- This attribute contains the access token of the owner of the file, and an access control list that states the access privileges that are granted to each user that has access to the file.

OPERATING SYSTEMS
Volume Management and Fault Tolerance

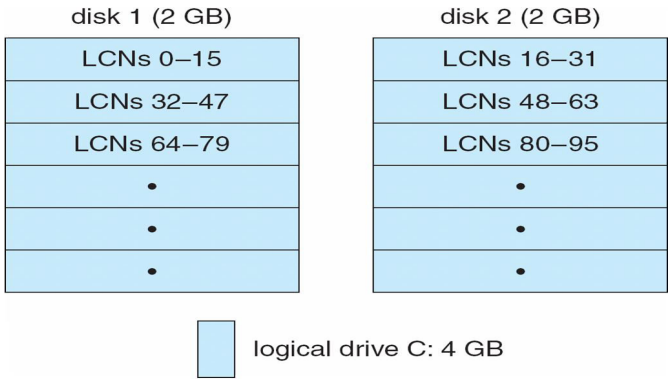
- **FtDisk**, the fault tolerant disk driver provides several ways to combine multiple SCSI disk drives into one logical volume
- Logically concatenate multiple disks to form a large logical volume, a *volume set*
- Interleave multiple physical partitions in round-robin fashion to form a ***stripe set*** (also called RAID level 0, or "disk striping")
  - FtDisk uses a stripe size of 64 KB – 1st 64 KB of the logical volume are stored in the 1st physical partition, 2nd 64 KB in the 2nd physical partition, and so on
  - Windows also supports RAID level 1 (mirroring) and RAID level 5 (*stripe set with parity)*
- Disk mirroring, or RAID level 1, is a robust scheme that uses a ***mirror set*** — two equally sized partitions on tow disks with identical data contents
- To deal with disk sectors that go bad, FtDisk, uses a hardware technique called ***sector sparing*** *(i.e. formatting a disk drive leaves extra sectors unmapped as spares)* and NTFS uses a software technique called ***cluster remapping*** *(i.e changes the pointers in the MFT from bad block to unallocated block)*

OPERATING SYSTEMS
Volume Set On Two Drives

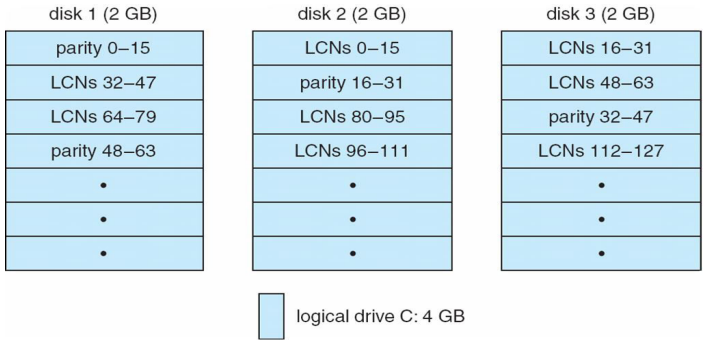


Logical volume or a *volume set can consist of up to 32 physical partitions*
A volume set can be extended by extending the bitmap metadata on the NTFS volume to cover the newly added space (bitmap contains information about free or used data blocks/clusters on a volume)
Fig 1==> 128000 x 4 KB = 0.5 GB
Fig 2 => 655360 x 4 KB = 2.5 GB

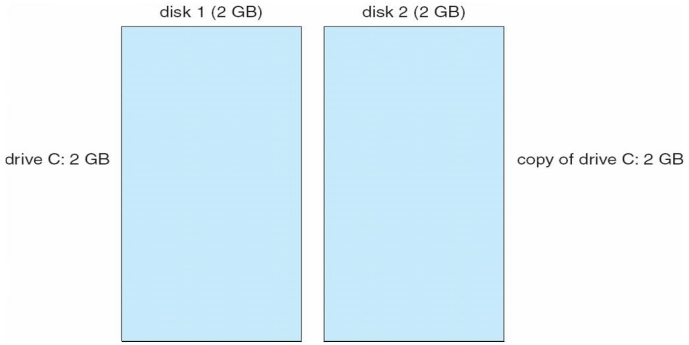
OPERATING SYSTEMS
Stripe Set on Two Drives



OPERATING SYSTEMS
Stripe Set With Parity on Three Drives



OPERATING SYSTEMS
Mirror Set on Two Drives



OPERATING SYSTEMS
File System — Compression

- To compress a file, NTFS divides the file's data into *compression units*, which are blocks of 16 contiguous clusters.
  - To improve performance when reading contiguous compression units, NTFS pre-fetches and decompresses ahead of the application requests.
- For sparse files, NTFS uses another technique to save space.
  - Clusters that contain all zeros (ie. never been written) are not actually allocated or stored on disk.
  - Instead, gaps are left in the sequence of virtual cluster numbers stored in the MFT entry for the file.
  - When reading a file, if a gap in the virtual cluster numbers is found, NTFS just zero-fills that portion of the caller's buffer.

OPERATING SYSTEMS
File System (Contd.)

- Mount Points, Symbolic links, Hard Links
  - Hard links: A single file has an entry in more than one directory of the same volume
- NTFS Journal – describes all changes that have been made to the file system.
  - Used by services such as User-mode (to identify what files have changed), search indexer (to re-index files) and file-replication (to replicate files across the network)
- Snapshots to create a shadow copy of volume for backup (and recovery of files if accidentally deleted)

THANK YOU

Kakoli Bora
Department of Computer Science Engineering
k_bora@pes.edu

OPERATING SYSTEMS
UE22CS242B

awk

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

awk

Suresh Jamadagni
Department of Computer Science

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau
5. Internet source

OPERATING SYSTEMS

AWK https://www.gnu.org/software/gawk/manual/gawk.html

- Awk is a scripting language used for manipulating data and generating reports.
- The awk command programming language requires no compiling and allows the user to use variables, numeric functions, string functions, and logical operators.
- Awk is a utility that enables a programmer to write tiny but effective programs in the form of statements that define text patterns that are to be searched for in each line of a document and the action that is to be taken when a match is found within a line.
- Awk is mostly used for pattern scanning and processing. It searches one or more files to see if they contain lines that matches with the specified patterns and then perform the associated actions.

OPERATING SYSTEMS
AWK

Operations:

- Scan a file line by line
- Split each input line into fields
- Compare input line/fields to pattern
- Perform action(s) on matched lines

Programming Constructs Provided:

- Format output lines
- Arithmetic and string operations
- Conditionals and loops

OPERATING SYSTEMS
AWK syntax

awk options 'selection _criteria {action }' input-file > output-file

Options:

- -f program-file : Reads the AWK program source from the file program-file, instead of from the first command line argument.
- -F fs : Use fs for the input field separator

OPERATING SYSTEMS
AWK examples

Print selected columns from a file

```
awk '{print $1,$4}' employee.txt
```

Print lines matching a criteria

```
awk '/manager/ {print}' employee.txt
awk '{if($4 >= "30000") print $1,$4;}' employee.txt
awk '{if($4 >= "30000" && $2 == "manager") print $1,$2,$4;}' employee.txt
awk '{if($4 >= "30000" && $2 != "manager") print $1,$2,$4;}' employee.txt
awk '{if($4 >= "30000" || $2 == "manager") print $1,$2,$4;}' employee.txt
```

OPERATING SYSTEMS
AWK examples

Print total number of fields in a csv file

```
awk 'BEGIN {FS=","} END {print NF}' employee.txt
```

Print total number of lines in a file

```
awk 'END { print NR }' employee.txt
```

Print total number of lines matching a criteria

```
awk '{if($4 >= "30000") count++} END {print count}' employee.txt
```

Using built-in functions to change the output

```
awk '/manager/ {print substr($1,1,4)}' employee.txt
awk '/manager/ {print toupper($1)}' employee.txt
awk '/manager/ {print tolower($1)}' employee.txt
awk '/manager/ {print toupper(substr($1,1,1)) substr($1,2)}' employee.txt
awk '{if($4 >= "30000" || $2 == "manager") print $1,$2,$4,$4*1.1;}' employee.txt
```

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering
sureshjamadagni@pes.edu

OPERATING SYSTEMS
UE22CS242B

sed

Suresh Jamadagni
Department of Computer Science

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau
5. Internet source

- sed is a stream editor.
- A stream editor is used to perform basic text transformations on an input stream (a file or input from a pipeline).
- While in some ways similar to an editor which permits scripted edits (such as ed), sed works by making only one pass over the input(s), and is consequently more efficient.
- But it is sed's ability to filter text in a pipeline which particularly distinguishes it from other types of editors.

OPERATING SYSTEMS
sed - syntax

- sed is invoked like this:

  sed SCRIPT INPUTFILE...

Example: To replace all occurrences of 'hello' to 'world' in the file input.txt:

  sed 's/hello/world/' input.txt > output.txt

- If ***INPUTFILE*** is not specified, sed filters the contents of the standard input
- sed writes output to standard output.
- Use -i to edit files in-place instead of printing to standard output

  sed -i 's/hello/world/' file.txt

- -f options are used to specify a *script*,

  sed -f myscript.sed input.txt > output.txt

  sed --expression='s/hello/world/' input.txt > output.txt

OPERATING SYSTEMS
sed - examples

- Use -n to suppress output, and the p command to print specific lines
- sed treats multiple input files as one long stream.
- The following example prints the first line of the first file (one.txt) and the last line of the last file (three.txt).

  sed -n '1p ; $p' one.txt two.txt three.txt

- Use -s to reverse this behavior.
- **Exit status:** An exit status of zero indicates success, and a nonzero value indicates failure
- Delete lines 30 to 35 in the input. 30,35 is an address range. d is the delete command

  sed '30,35d' input.txt > output.txt

OPERATING SYSTEMS
sed - examples

- Print all input until a line starting with the word 'foo' is found. If such line is found, sed will terminate with exit status 42. If such line was not found (and no other error occurred), sed will exit with status 0. /^foo/ is a regular-expression address. q is the quit command

  sed '/^foo/q42' input.txt > output.txt

- Commands within a *script* or *script-file* can be separated by semicolons (;) The following command performs two sed operations: deleting any lines matching the regular expression /^foo/, and replacing all occurrences of the string 'hello' with 'world':

  sed '/^foo/d ; s/hello/world/' input.txt > output.txt

OPERATING SYSTEMS
sed - examples

- Delete the second input line:

  seq 3 | sed 2d

- Print only the second input line

  seq 3 | sed -n 2p

- Perform substitution on every 3rd line

  seq 6 | sed 'n;n;s/./x/'

- Perform substitution then print the second input line

  seq 3 | sed -n '2{s/2/X/ ; p}'

- Add the word 'hello' after the second line:

  seq 3 | sed '2a hello'

sed - examples

- Insert the word ‘hello’ before the second line

  seq 3 | sed '2i hello‘

- Replace the 2nd to 9th lines with the word ‘hello’

  seq 10 | sed '2,9c hello‘

- Replace the word ‘hello’ with ‘world’ only on line 144

  sed '144s/hello/world/' input.txt > output.txt

- Replace the word ‘hello’ with ‘world’ only in lines containing the word ‘apple’

  sed '/apple/s/hello/world/' input.txt > output.txt

- Replace the word ‘hello’ with ‘world’ only in lines 4 to 17 (inclusive):

  sed '4,17s/hello/world/' input.txt > output.txt

OPERATING SYSTEMS

sed - examples

- An address range can be specified by specifying two addresses separated by a comma (,)

  seq 10 | sed -n '4,6p‘

- Prints lines containing the word ‘hello’

  sed -n '/hello/p‘

- Replace the words ‘gray’ or ‘grey’ with ‘blue’

  sed 's/gr[ae]y/blue/‘

- Joining specific lines (e.g. if lines 2 and 3 need to be joined)

  sed '2{N;s/\n//;}' lines.txt

THANK YOU

Suresh Jamadagni
Department of Computer Science Engineering

sureshjamadagni@pes.edu